

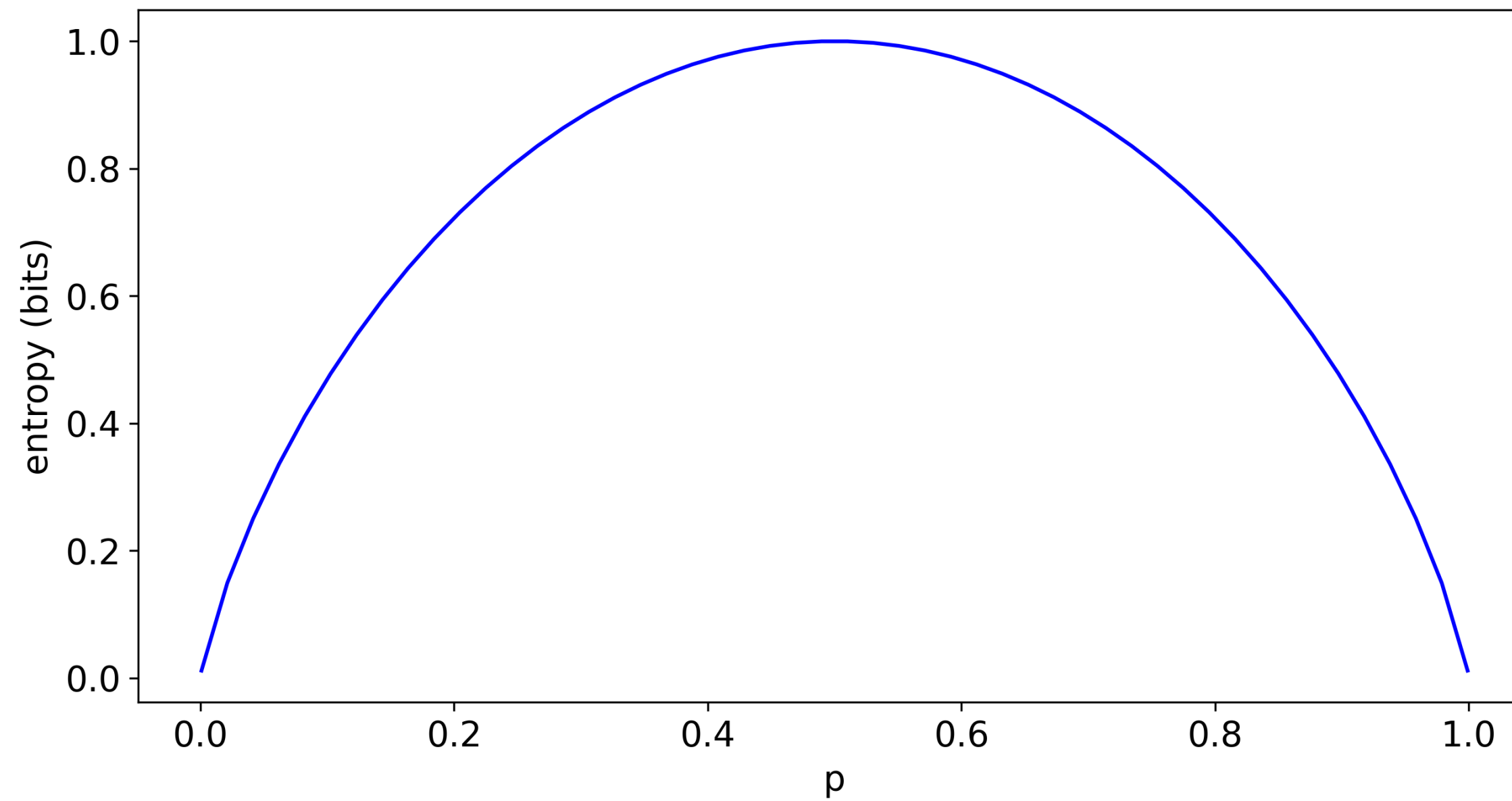
Overfitting

- Overfitting is what happens when we *estimate a complex* model's performance using *noisy, limited-size* training data, then *optimize* based on that performance estimate
- Two main effects:
 - ▶ we tend to be *too optimistic* about the selected model's performance (e.g., unrealistically low training error)
 - ▶ $\hat{P}(\text{selected model}) \ll P(\text{selected model})$
 - ▶ we'll tend to pick a *suboptimal* model
 - ▶ $P(\text{best model in } \mathcal{H}) \ll P(\text{selected model})$
- Typically can't check either of these (can't access true performance P , don't know best model in $\mathcal{H}$)
 - ▶ but $\hat{P}(\text{selected model}) \ll \hat{P}_{\text{test}}(\text{selected model})$ is close

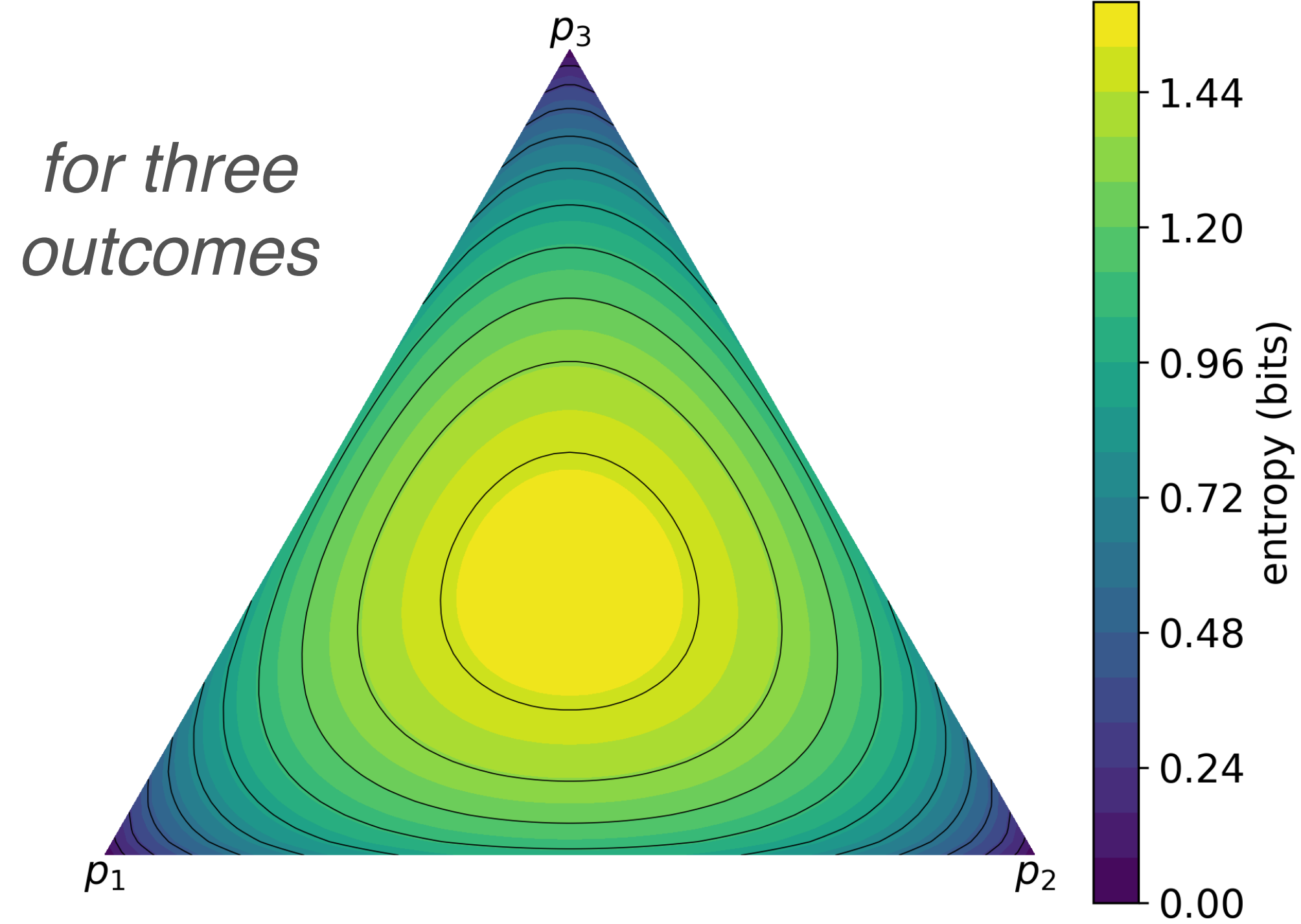
we'll say that (RHS – LHS) for any of these 3 expressions measures overfitting, but often we can only compute the last

*for a performance metric P
that we want to minimize*

Entropy



for two outcomes



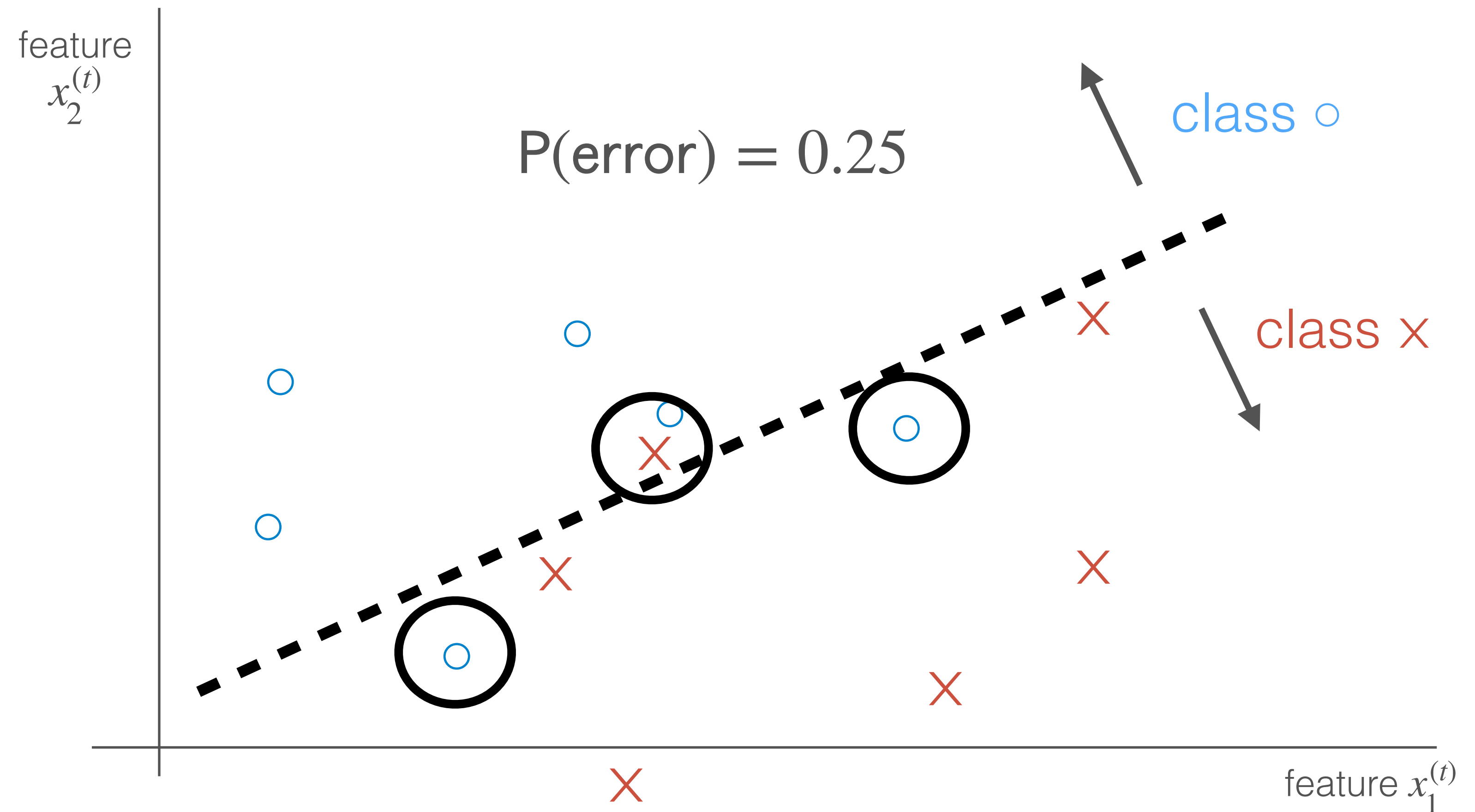
for three outcomes

- More certainty = less entropy

Connect back to learning

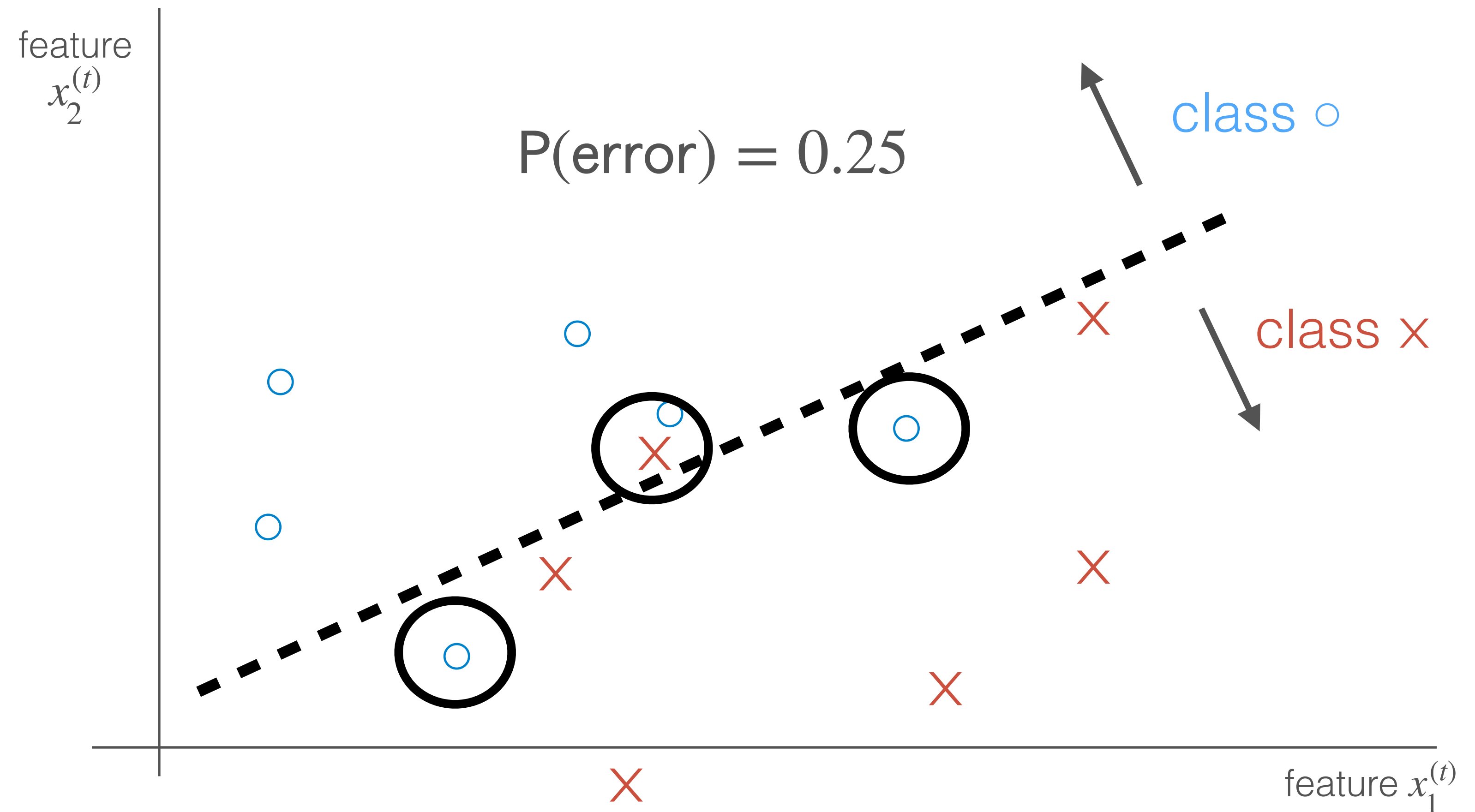
- Setup:
 - ▶ both Alice and Bob know $x^{(t)}$ for N examples
 - ▶ but only Bob knows $y^{(t)}$ — wants to send labels to Alice
- Naive strategy:
 - ▶ Bob sends $y^{(t)}$ for all examples, using 1 bit per example
- Possibly better strategy:
 - ▶ Bob learns a model $h \in \mathcal{H}$ (e.g., weight w in $f_w(x^{(t)})$)
 - ▶ first sends h to Alice, then uses h to help send $y^{(t)}$
- A good hypothesis is one that communicates lots of bits about our data using only a few bits of hypothesis
 - ▶ pay once up front, save on each new example

Using hypothesis to help send labels



- Encode the errors instead of the labels
 - ▶ Bob sends training error probability ϵ as part of h
 - ▶ coding cost is $\text{entropy}(\epsilon)$ per label — can be much less than naive cost of 1 bit

Using hypothesis to help send labels



- Encode the errors instead of the labels

send labels, L to R: 000101100111, $p=0.5$, entropy=1 bit/label

send errors, L to R: 001000101000, $p=0.25$, entropy=0.81 bits/label

Generalization

*preview of
upcoming lectures
on learning theory*

*notice we don't
need to look at a
test set to use this
bound*

- Can even make a bound: whatever compression we see for our training set, we're likely not to do much worse on unseen future test examples
- Whp, coding cost per example on training set satisfies

$$\begin{array}{ccccc} \text{cost under true} & & & & \text{terms that} \\ \text{error rate} & \leq & \text{observed cost} & + & \text{shrink w/ } N \end{array}$$

- Observed cost per example uses observed training error rate, and includes (amortized) cost of sending h
- LHS grows w/ true error, so this bound rules out true error being too much larger than observed
- More sophisticated versions of this result actually give nontrivial generalization bounds for modern LLMs — still an active research area

Learning as compression

- Weird but fairly decent learning algorithm: `gzip`
- Training:
 - ▶ collect positive-class examples into `pos.txt`
 - ▶ collect negative-class examples into `neg.txt`
- Prediction for a new example `test_ex.txt`:
 - ▶ get file size of `cat pos.txt test_ex.txt | gzip`
minus file size of `gzip pos.txt`
 - ▶ similarly, size of `cat neg.txt test_ex.txt | gzip`
minus size of `gzip neg.txt`
 - ▶ pick the class with the ***smaller*** increment in file size
- Learned hypothesis = internal data structures of `gzip`
after processing `pos.txt` and `neg.txt`

Learning objectives— information

- Why is measuring information important to ML?
- What are joint, marginal, and conditional distributions?
- What is the Shannon bound, and what does it tell us about codebooks / message length?
- What happens if we encode a message based on the wrong probability distribution?
- What are entropy, cross entropy, and relative entropy? What is mutual information?
- How does learning relate to communication and compression?

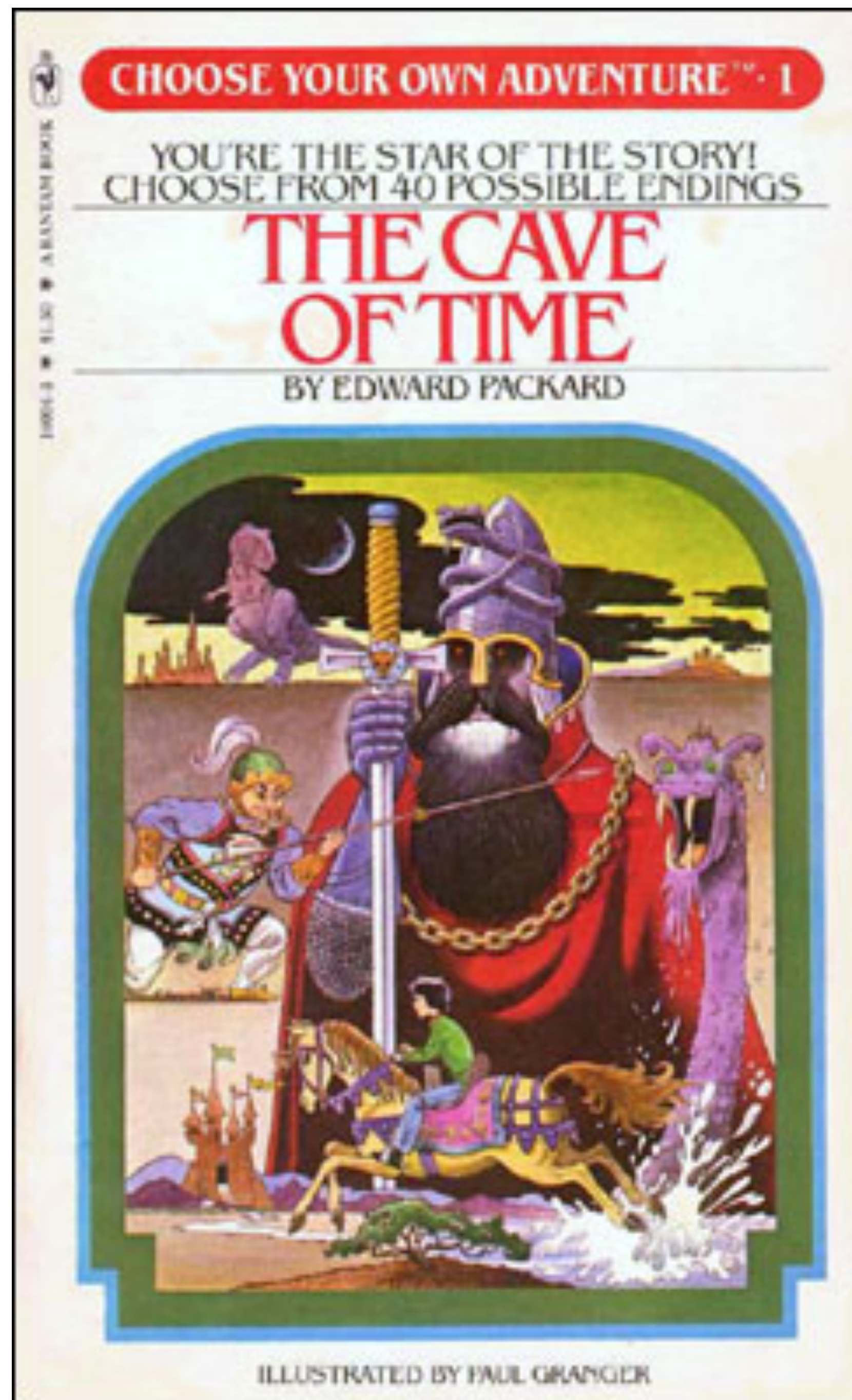
Decision trees, k -nearest neighbor, model selection



10-701 Introduction to Machine Learning
Geoff Gordon

(thanks to Matt Gormley and Henry Chai for some content)

Decision trees



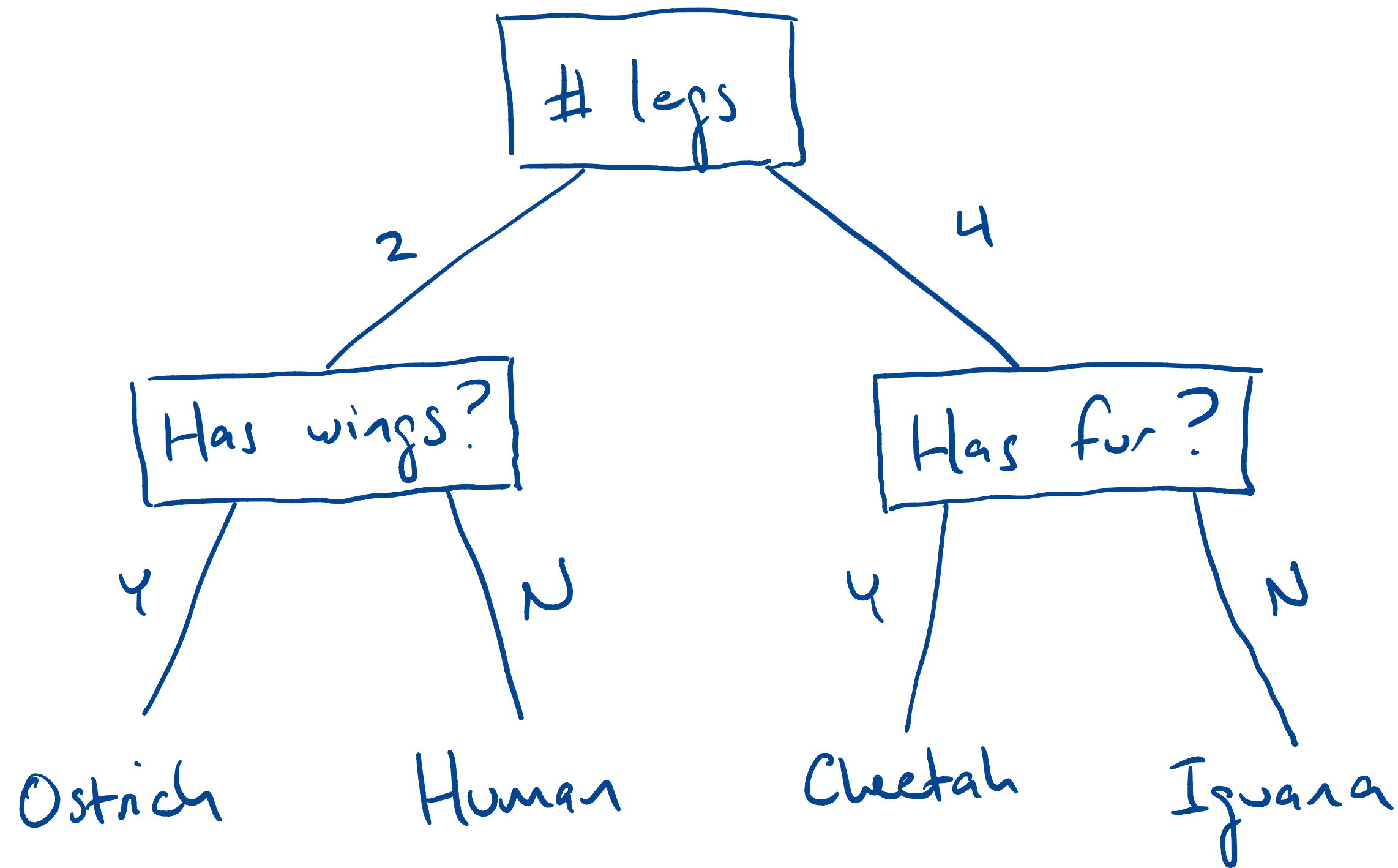
Choose your own adventure!

If you decide to open the creepy sarcophagus, turn to p63

If you decide to back slowly out of the room, turn to p17

If you decide to set up camp for the night, turn to p42

Decision trees

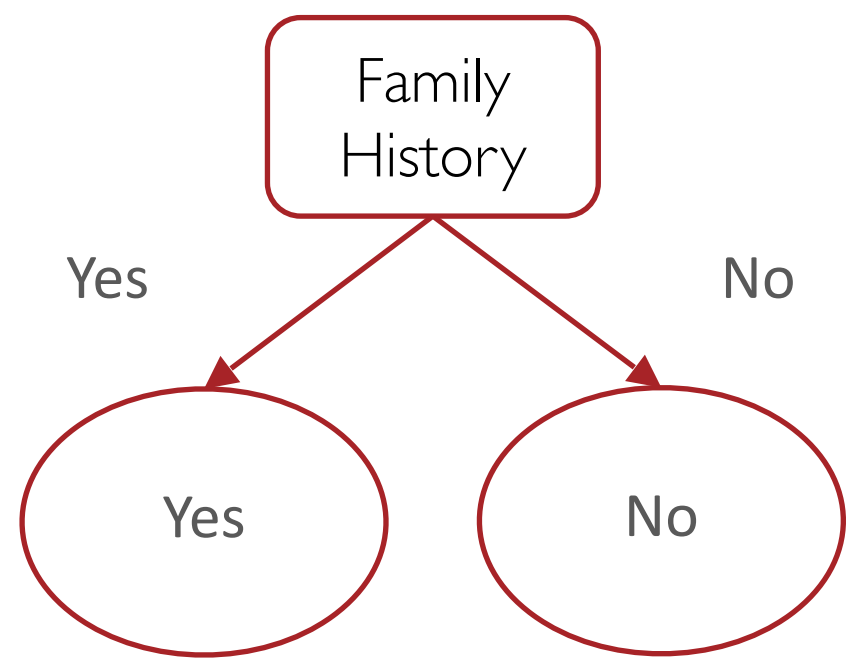


- A question in each node, an answer on each edge
 - ▶ answers mutually exclusive and exhaustive
- Each question splits the data into two (or more) pieces
- When we reach a leaf, it makes a prediction

Decision tree example

training data points

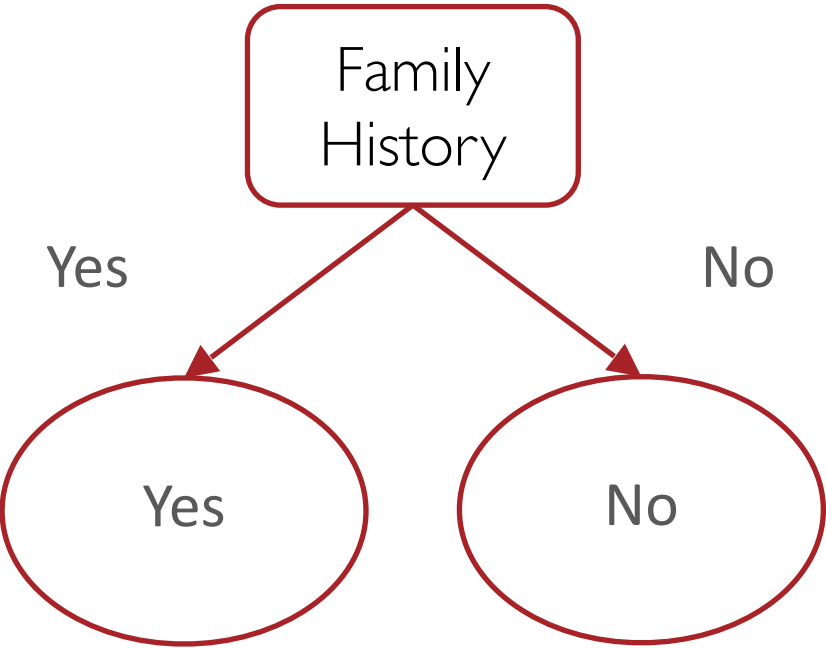
features			labels
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes



Decision tree example

training data points

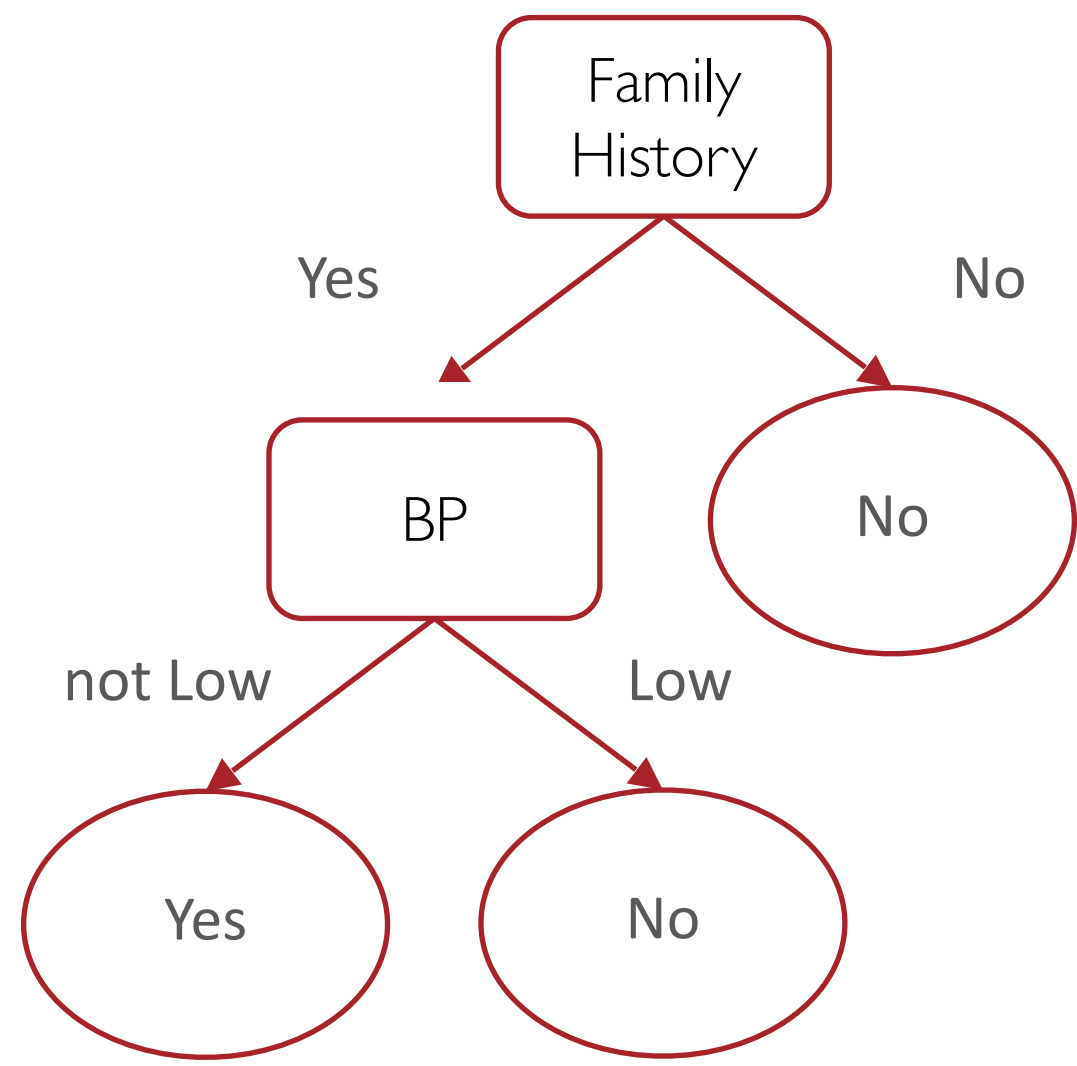
features			labels	predictions
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	Yes
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes



Decision tree example

training data points

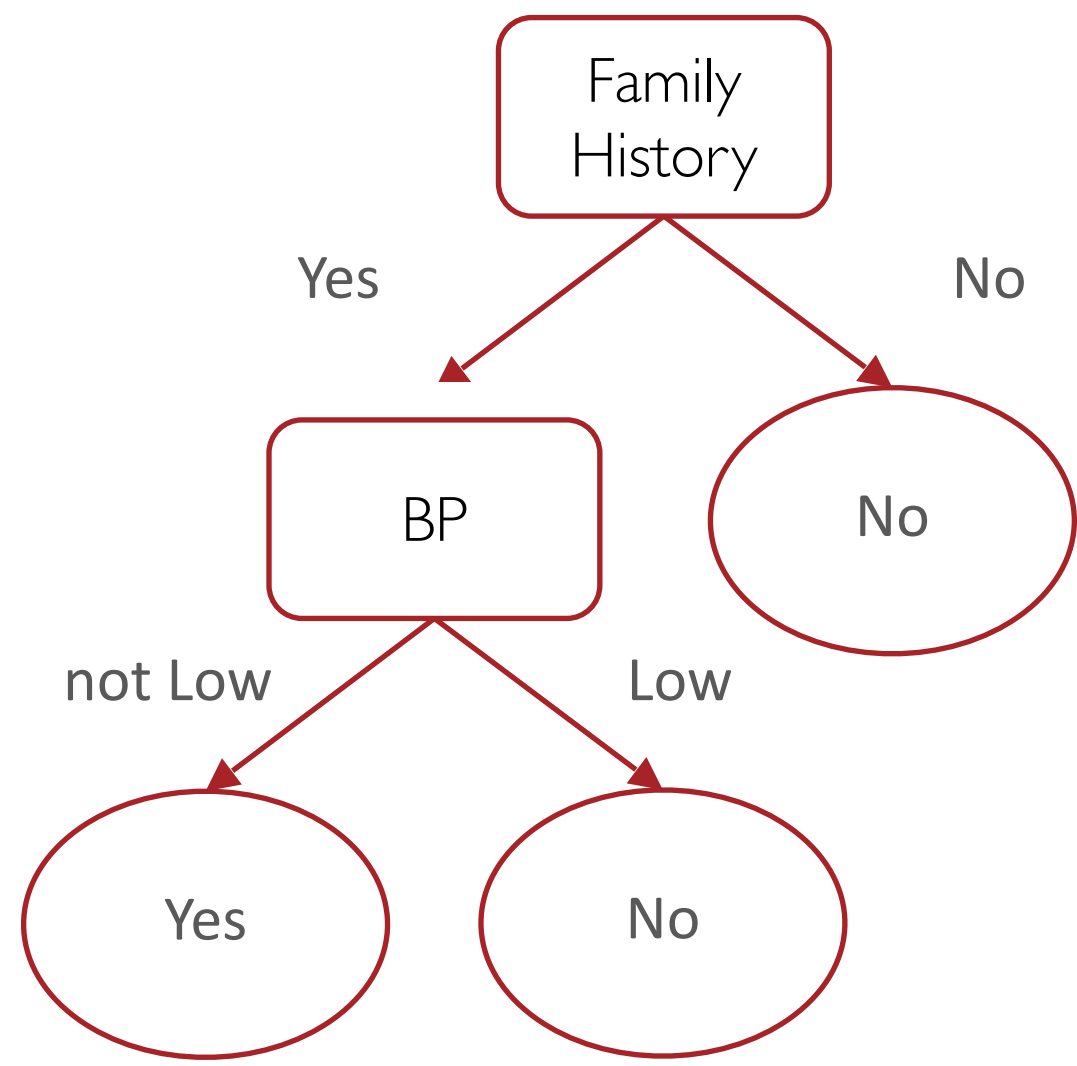
features			labels	predictions
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	Yes
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes



Decision tree example

training data points

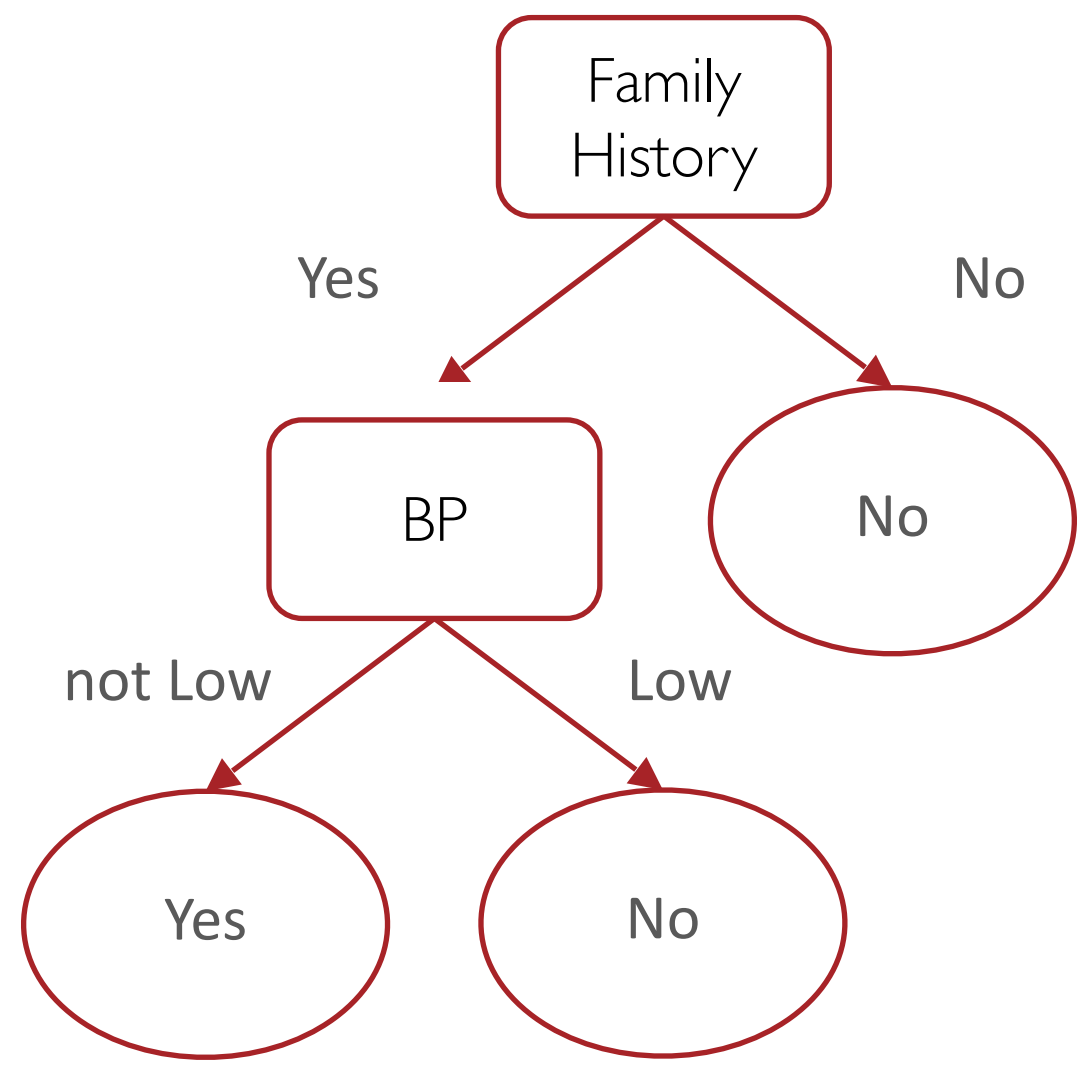
features			labels	predictions
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	No
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes



Decision tree example

training data points

features			labels	predictions
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	No
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes

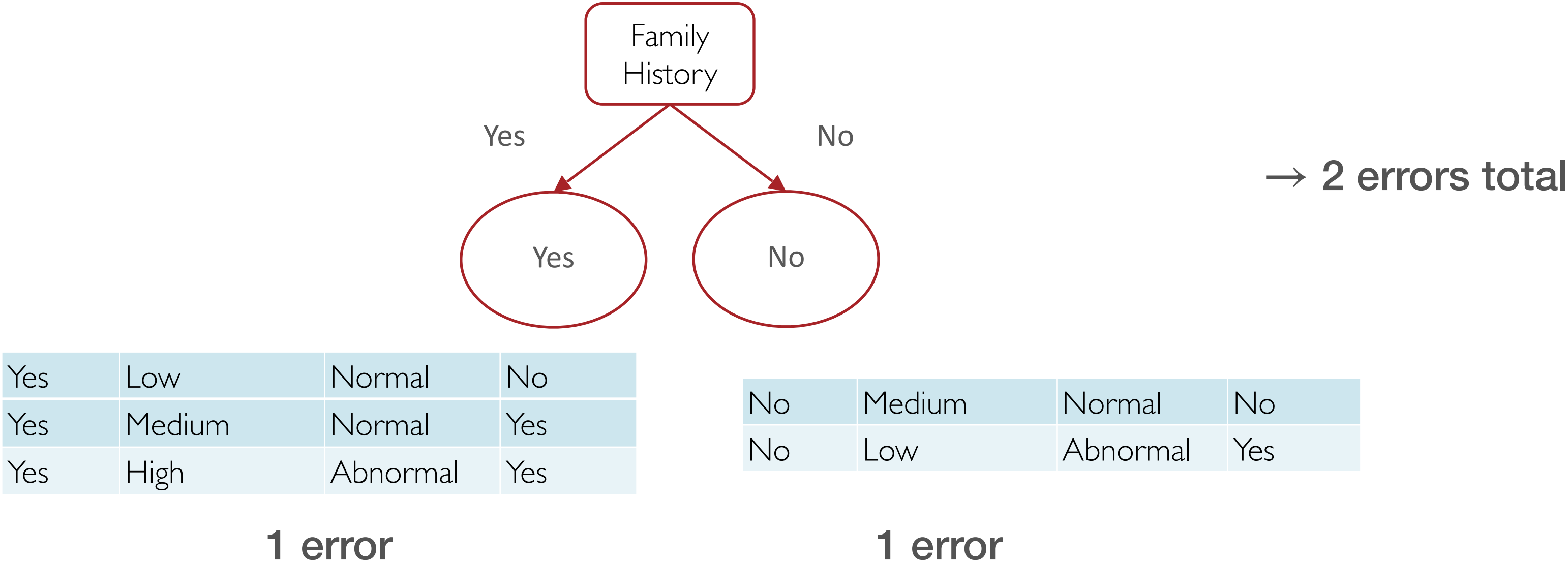


if just one split:
“decision stump”

Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

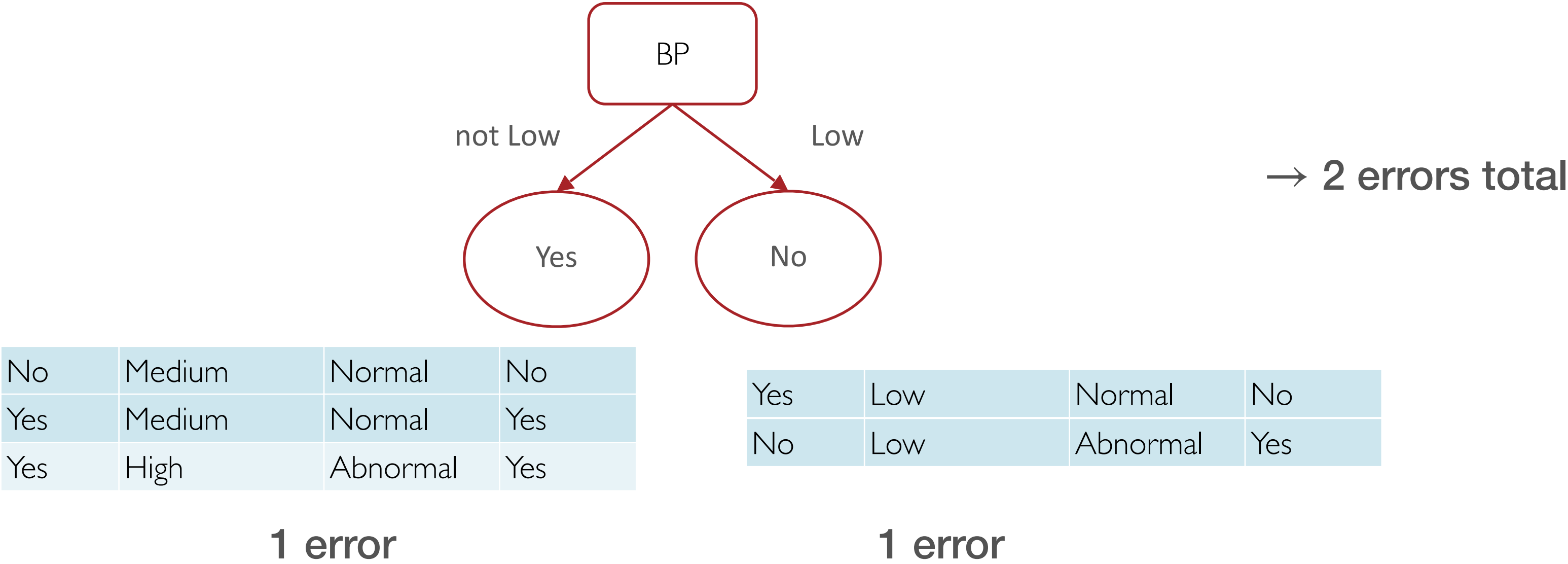
- How do we choose which features to split on?
 - example criterion: minimize training error



Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

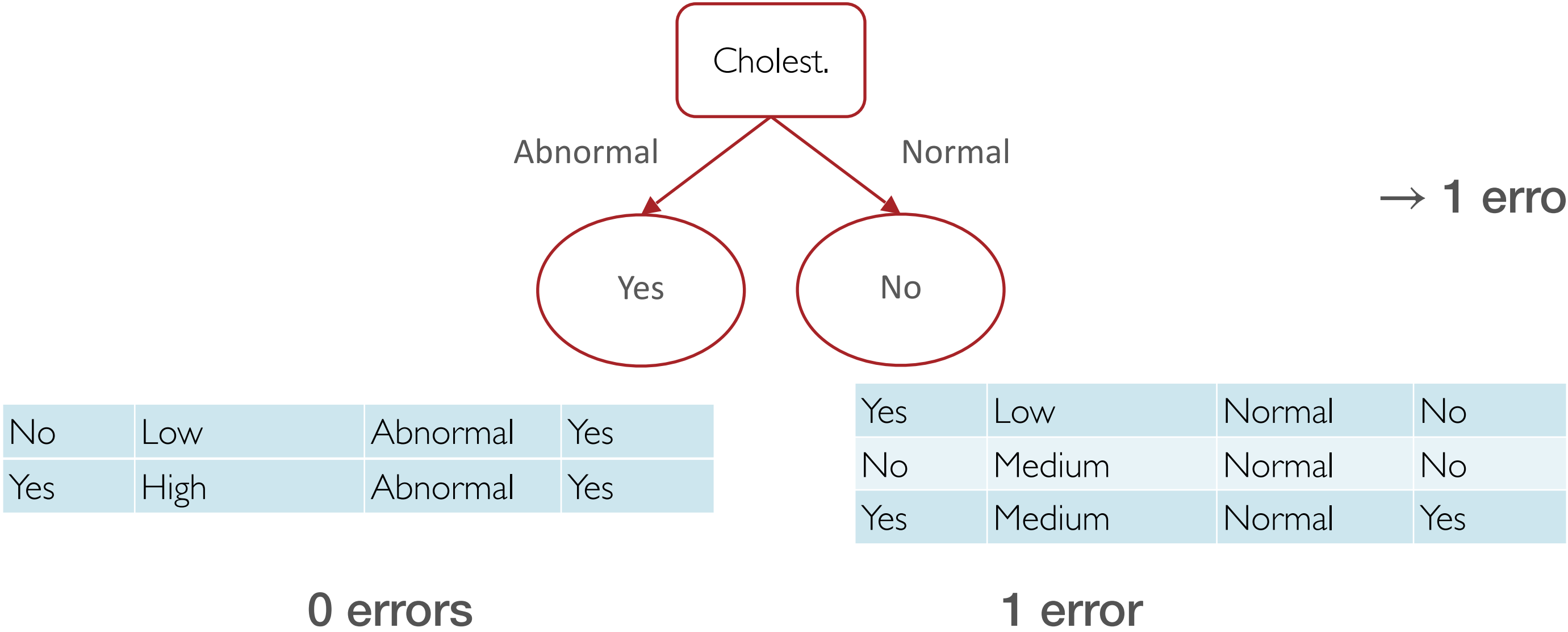
- How do we choose which features to split on?
 - ▶ example criterion: minimize training error



Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

- How do we choose which features to split on?
 - ▶ example criterion: minimize training error



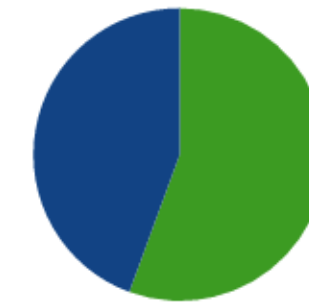
Decision tree learning

- We could ask for the decision tree that minimizes error on training set (subject to constraints, e.g., max depth)
 - ▶ but this is a difficult combinatorial optimization problem
- Instead, typically use a heuristic search
 - ▶ find a good tree but maybe not the absolute optimal one
- Most common: greedy top-down search
 - ▶ pick the split at the root according to some ***split criterion***
 - ▶ recursively learn left and right children, using the appropriate subset of the data for each
 - ▶ stop if ***stopping criterion*** (e.g., threshold on depth or number of examples in subtree)
 - ▶ use *majority* classifier within each leaf

Decision Tree — Training

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

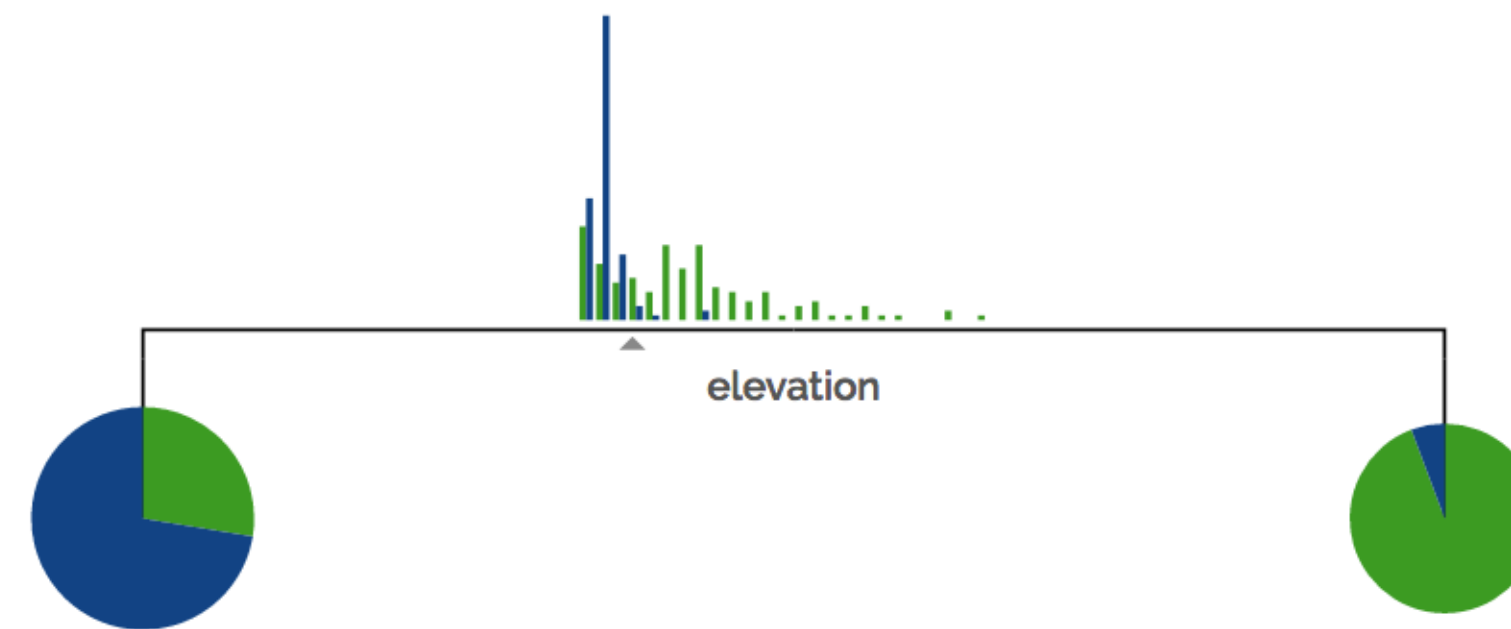
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Training

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

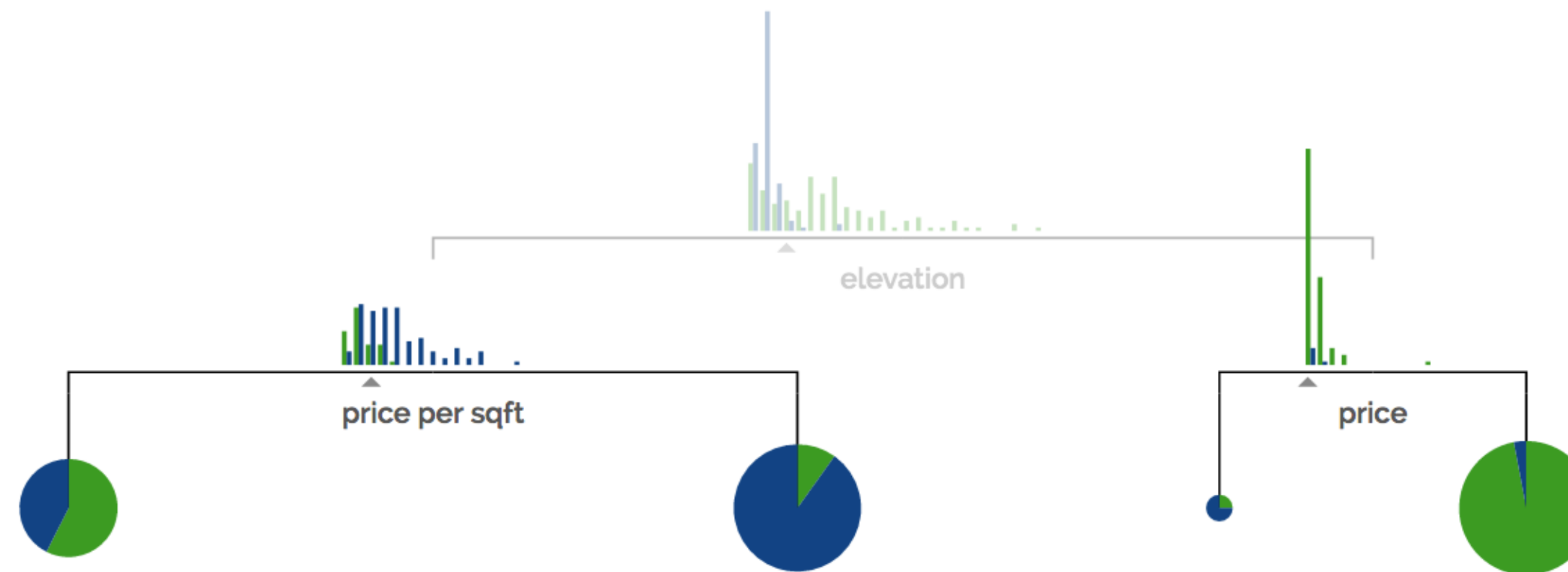
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Training

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

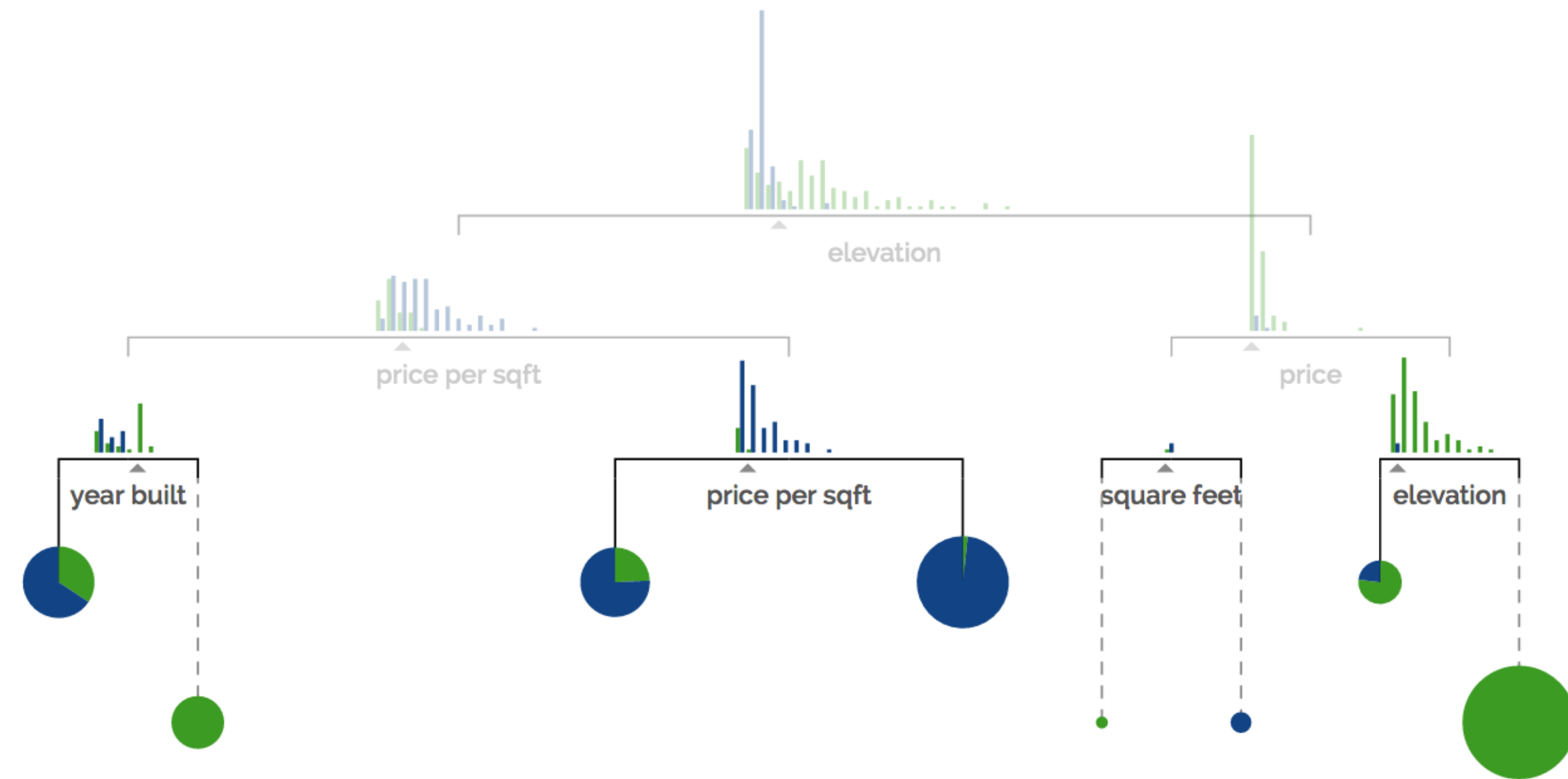
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Training

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

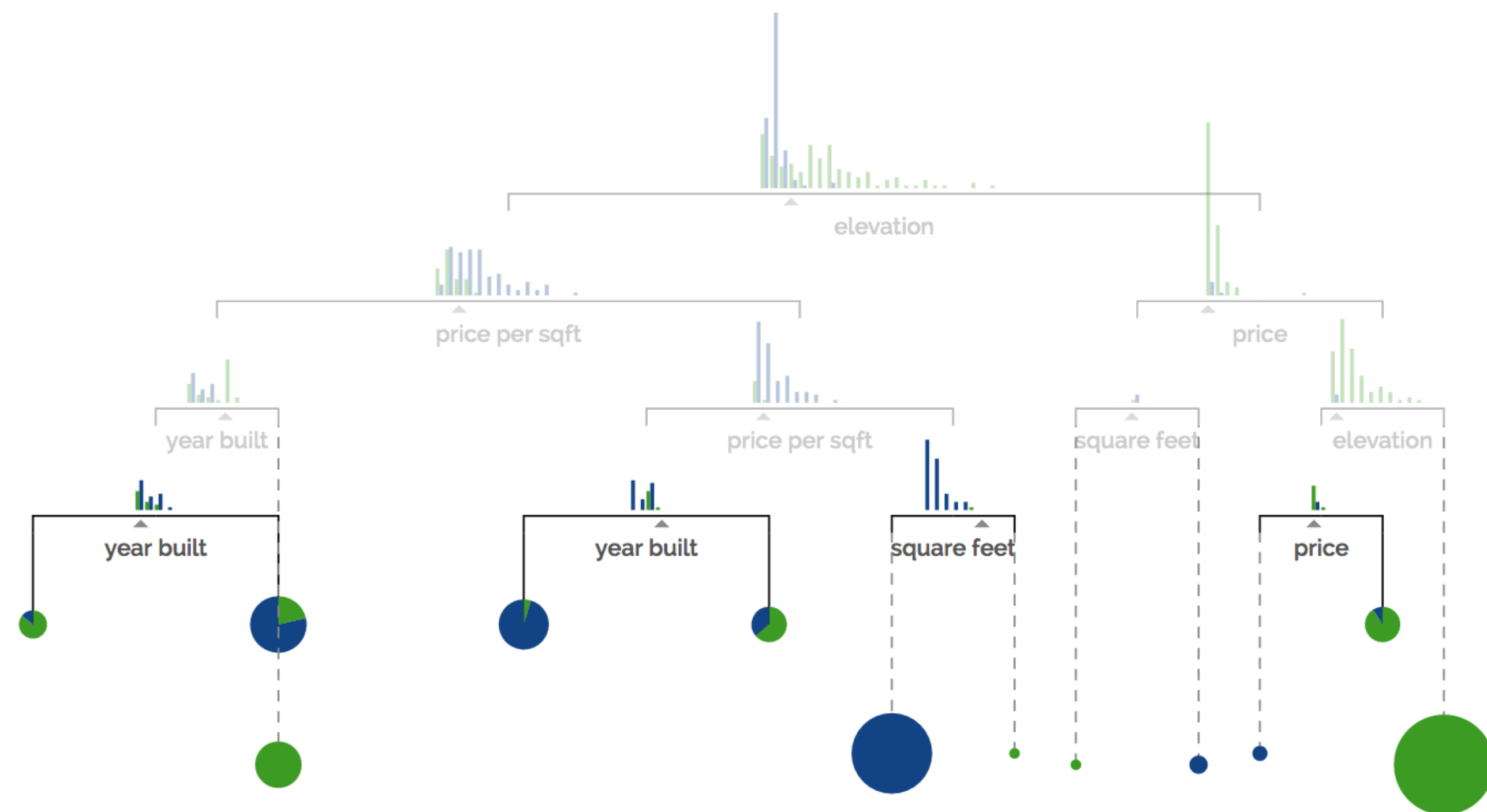
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Training

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

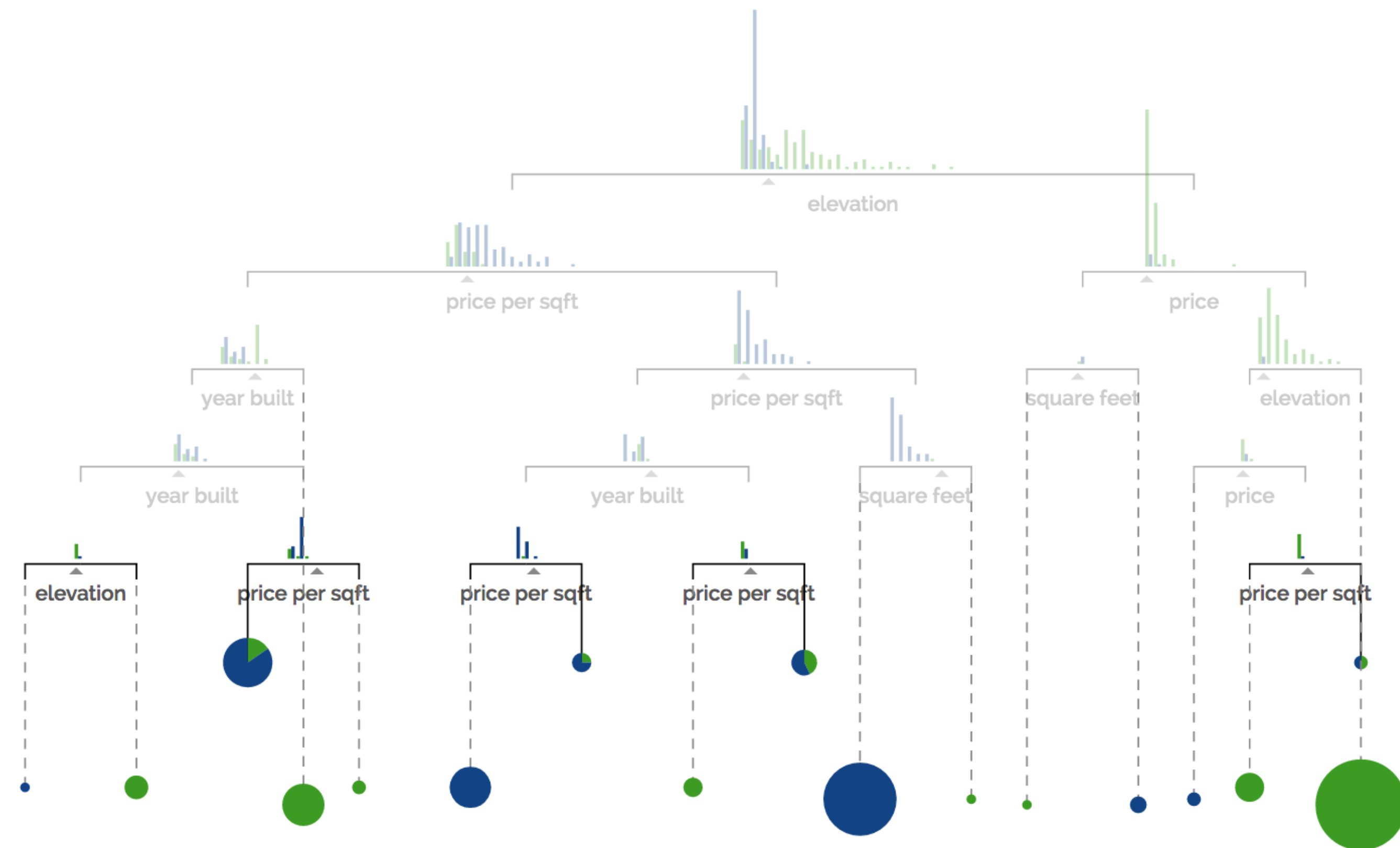
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Training

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

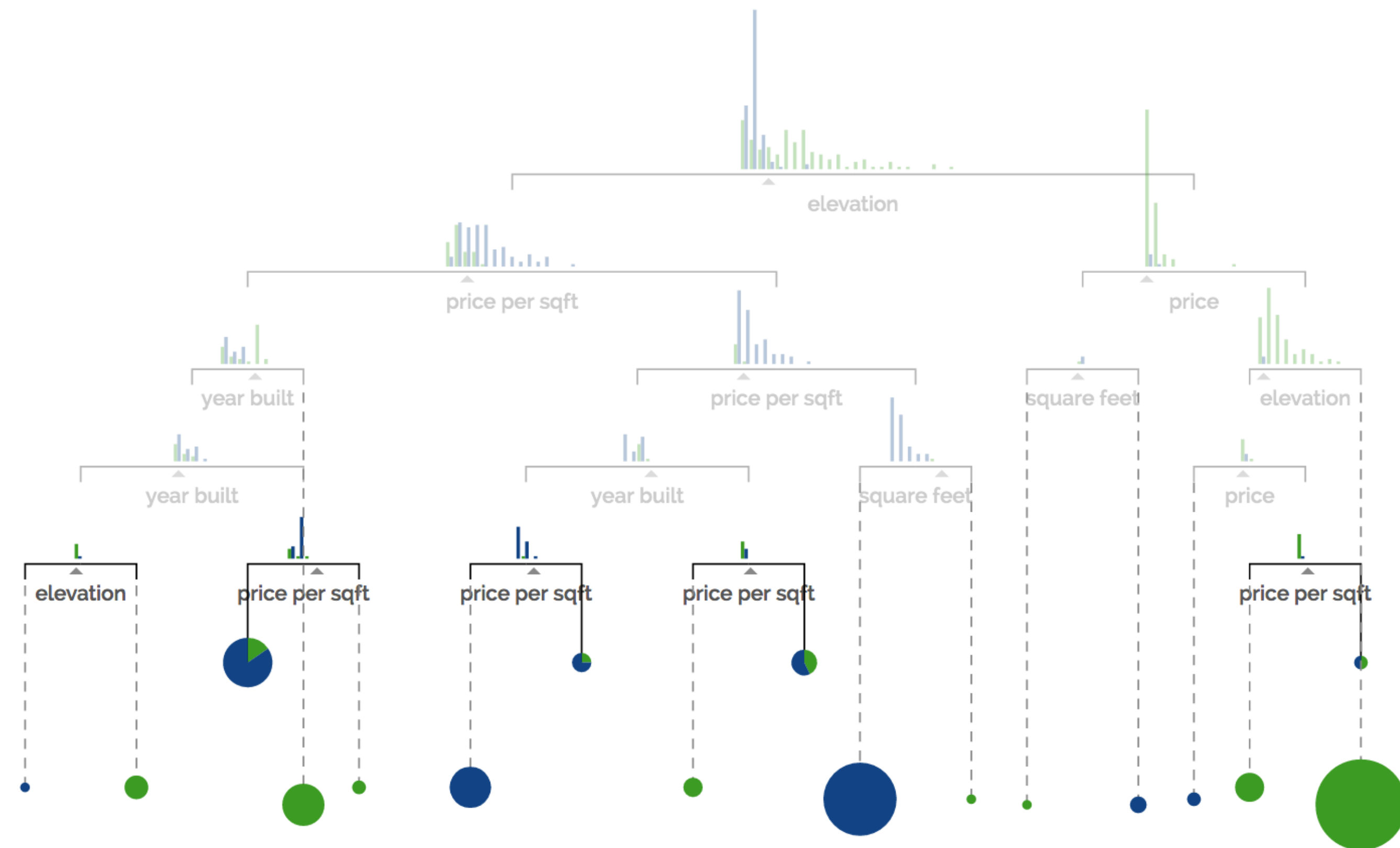
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

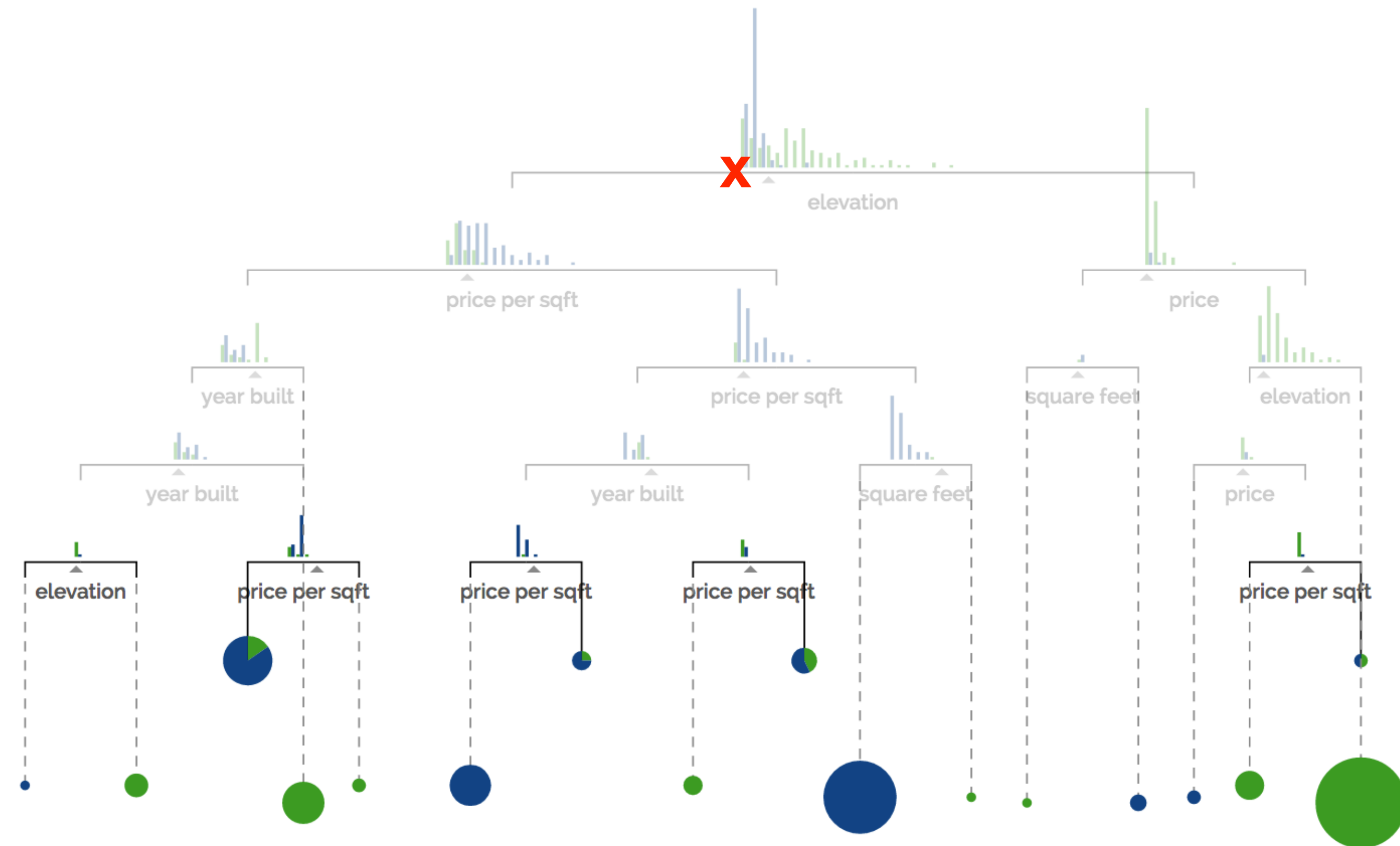
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

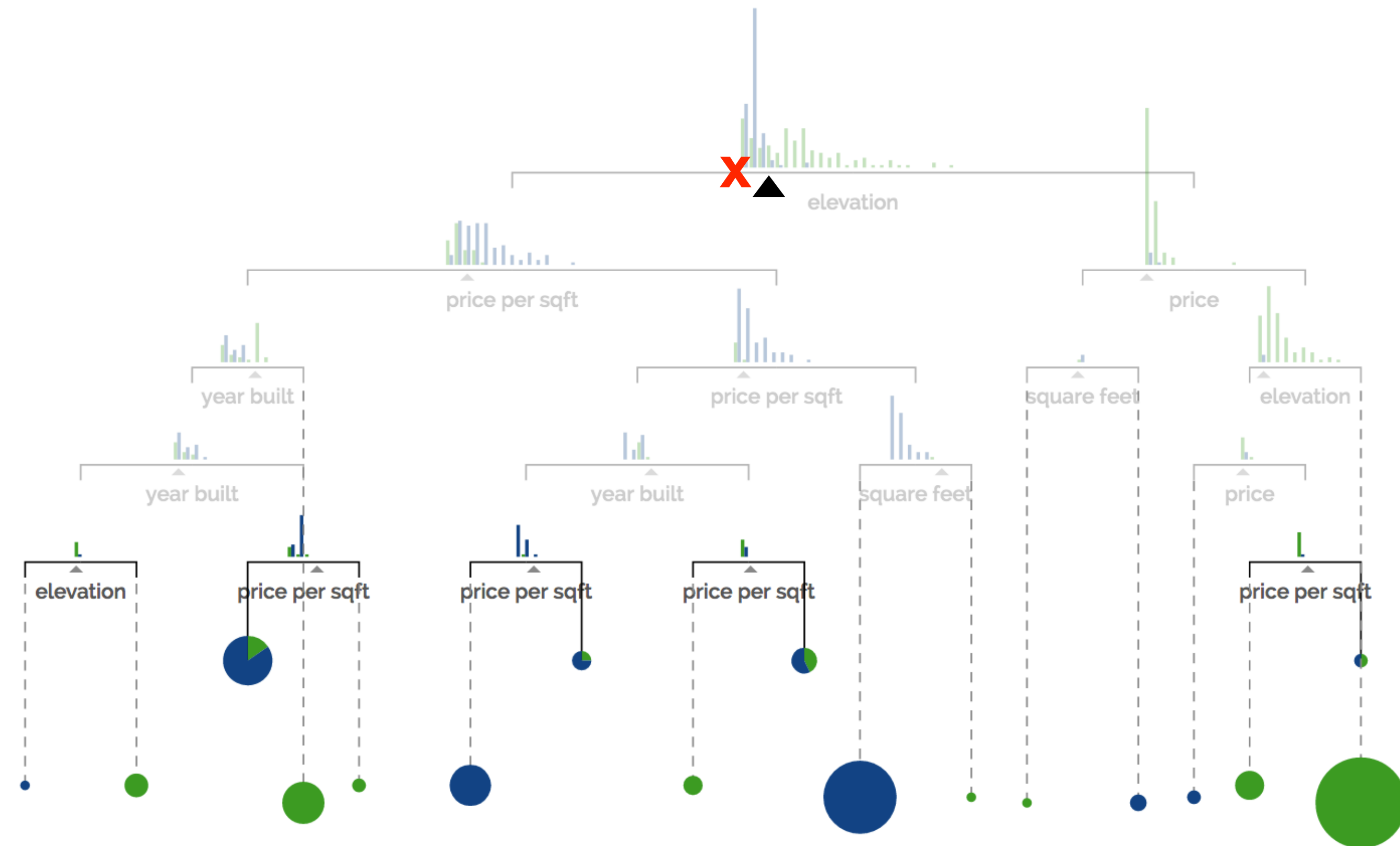
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

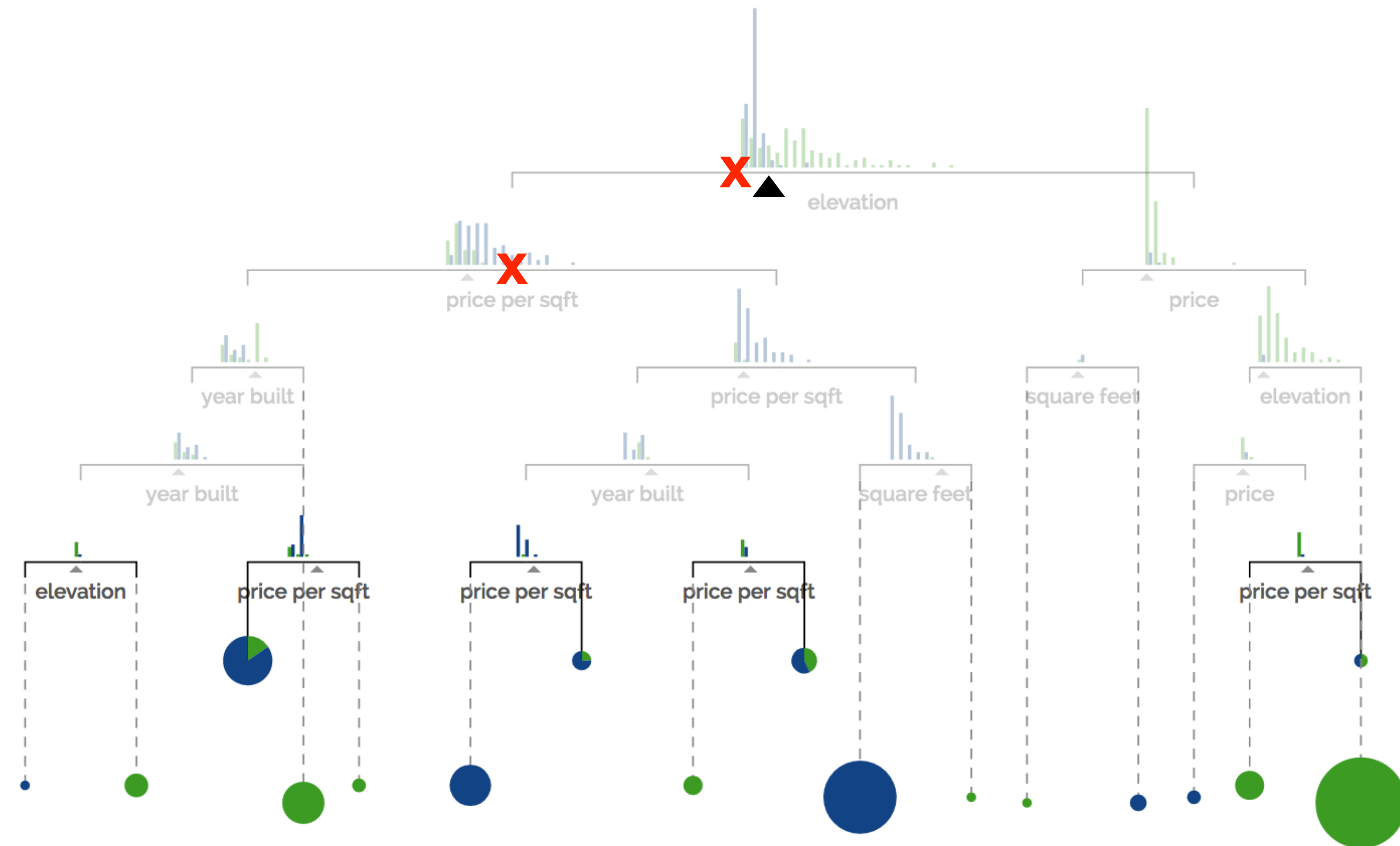
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

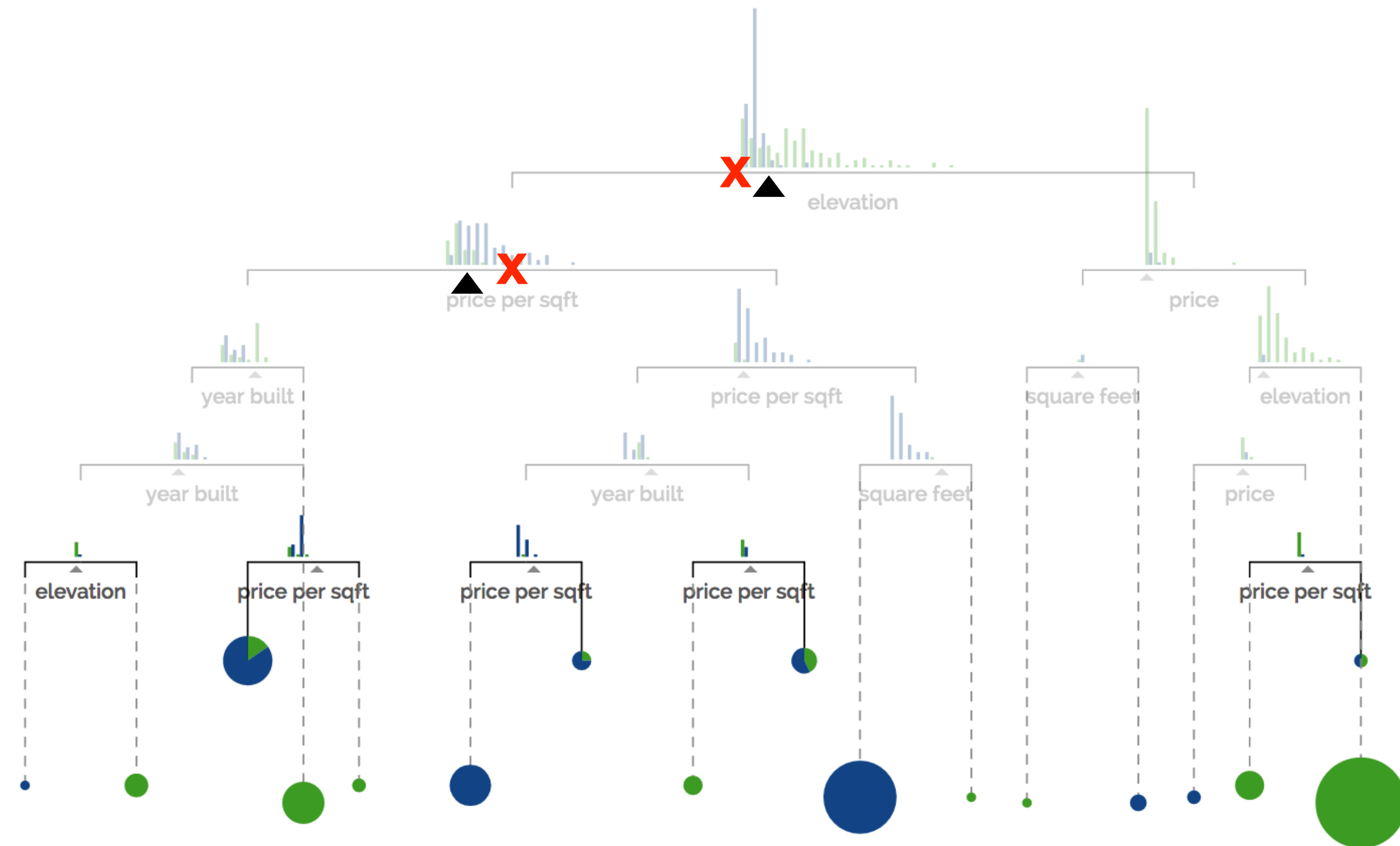
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

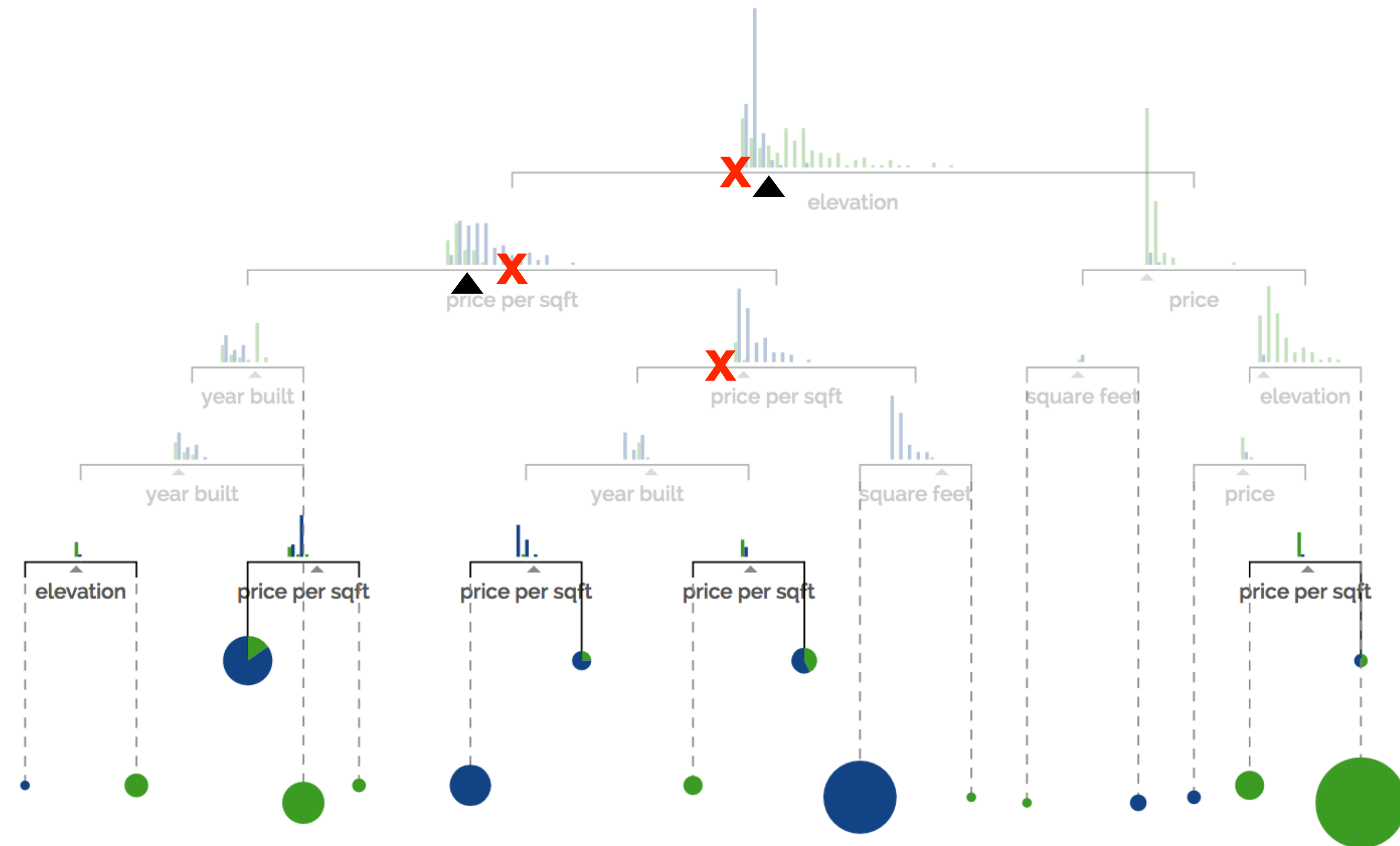
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

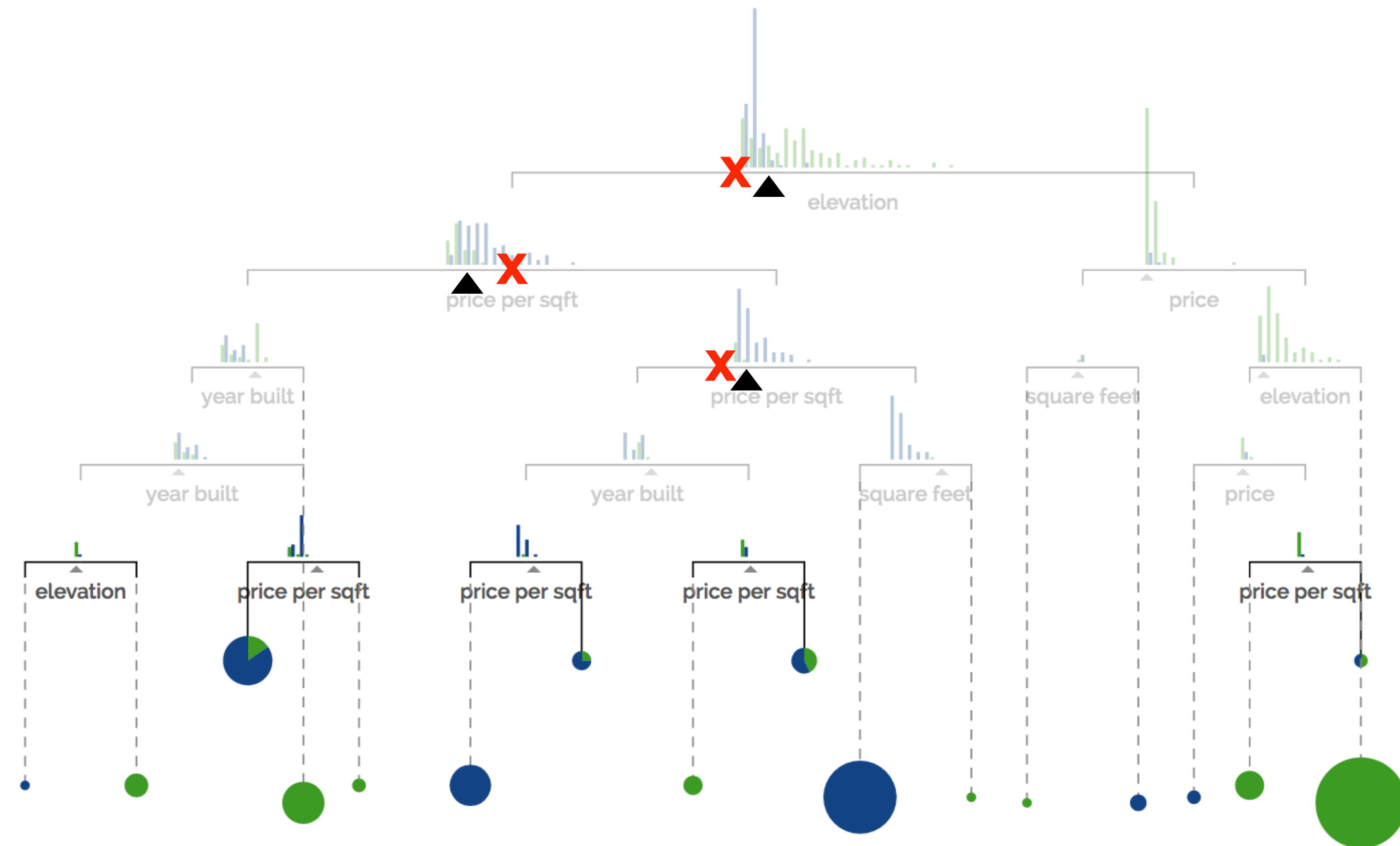
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

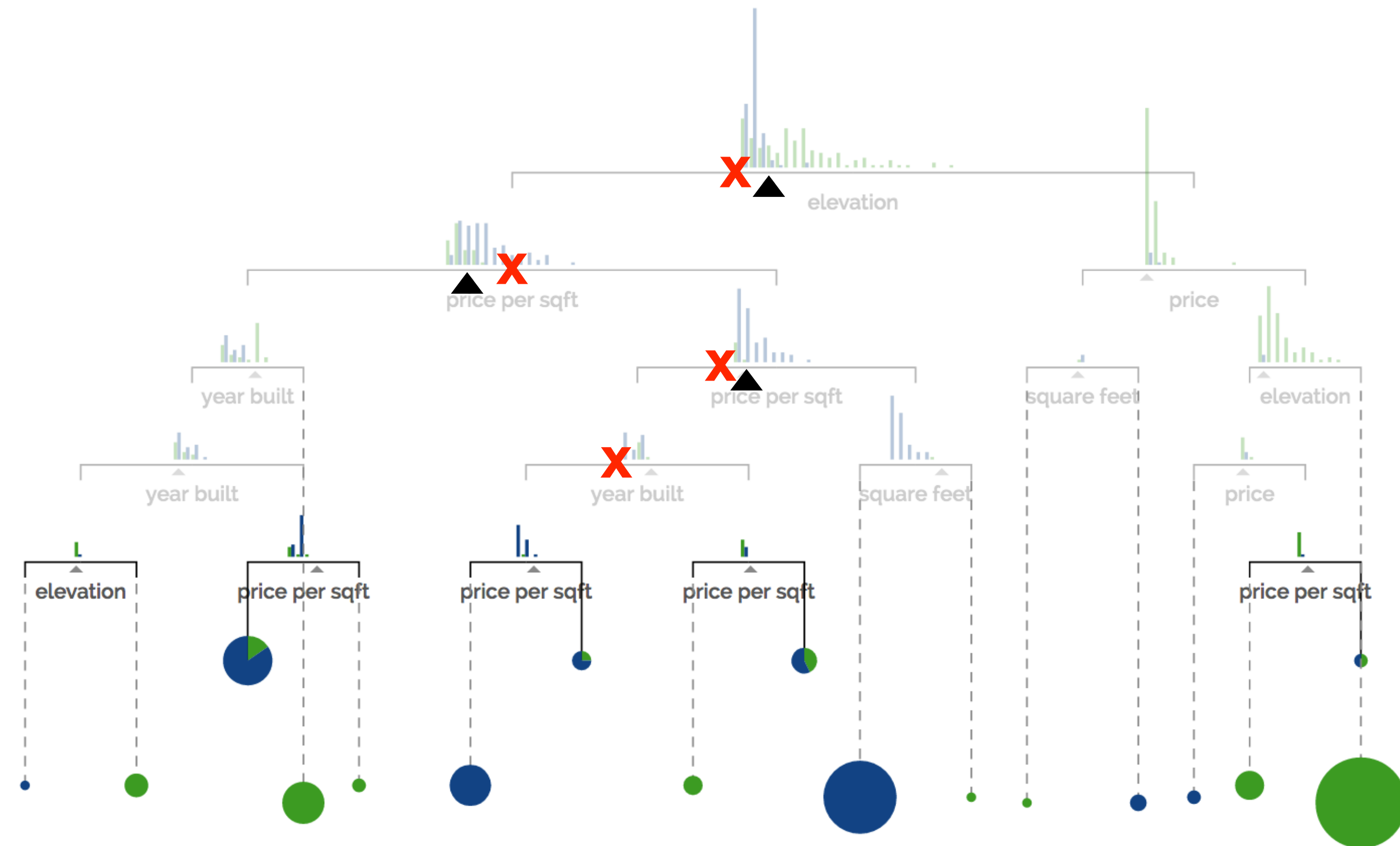
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

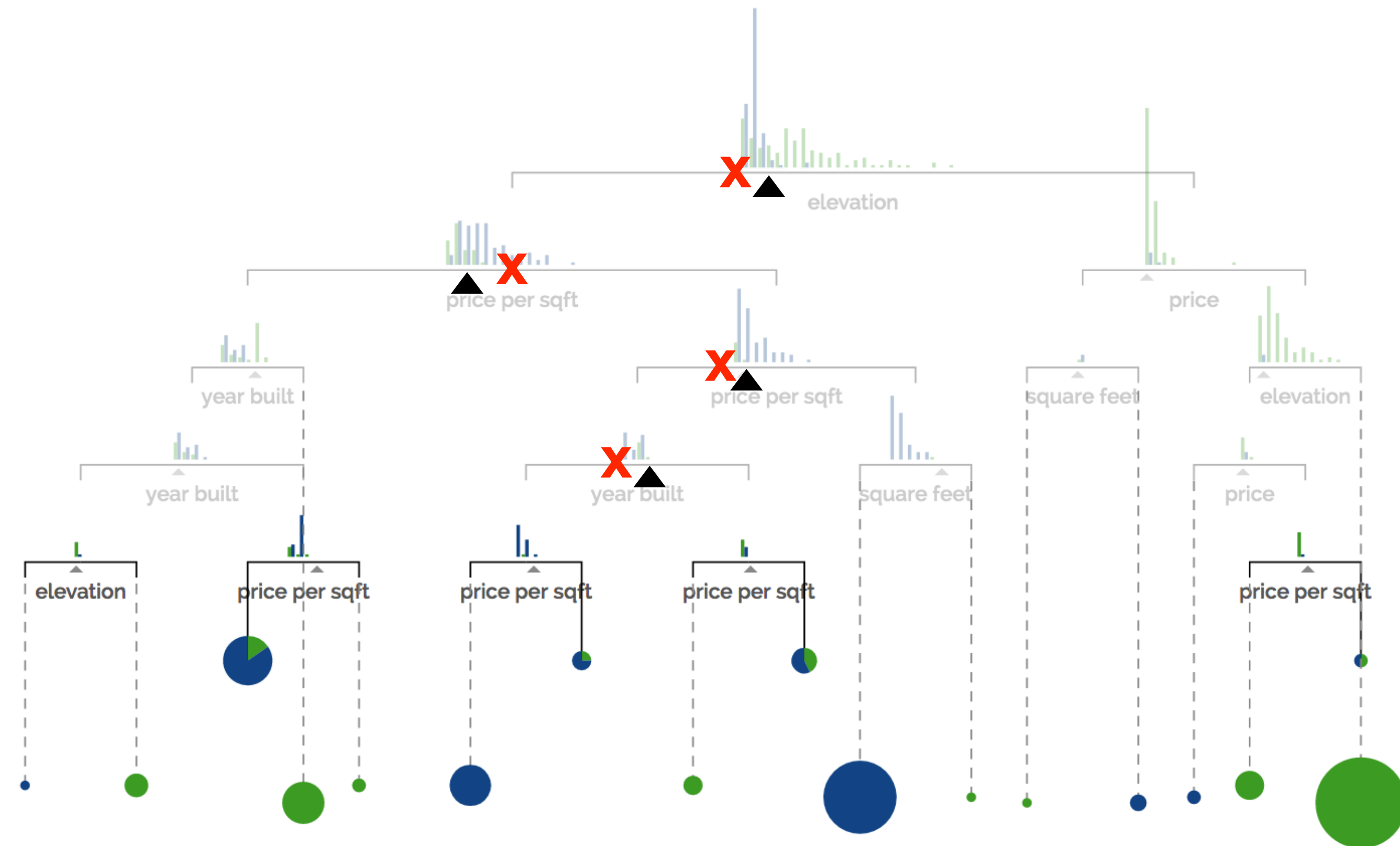
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

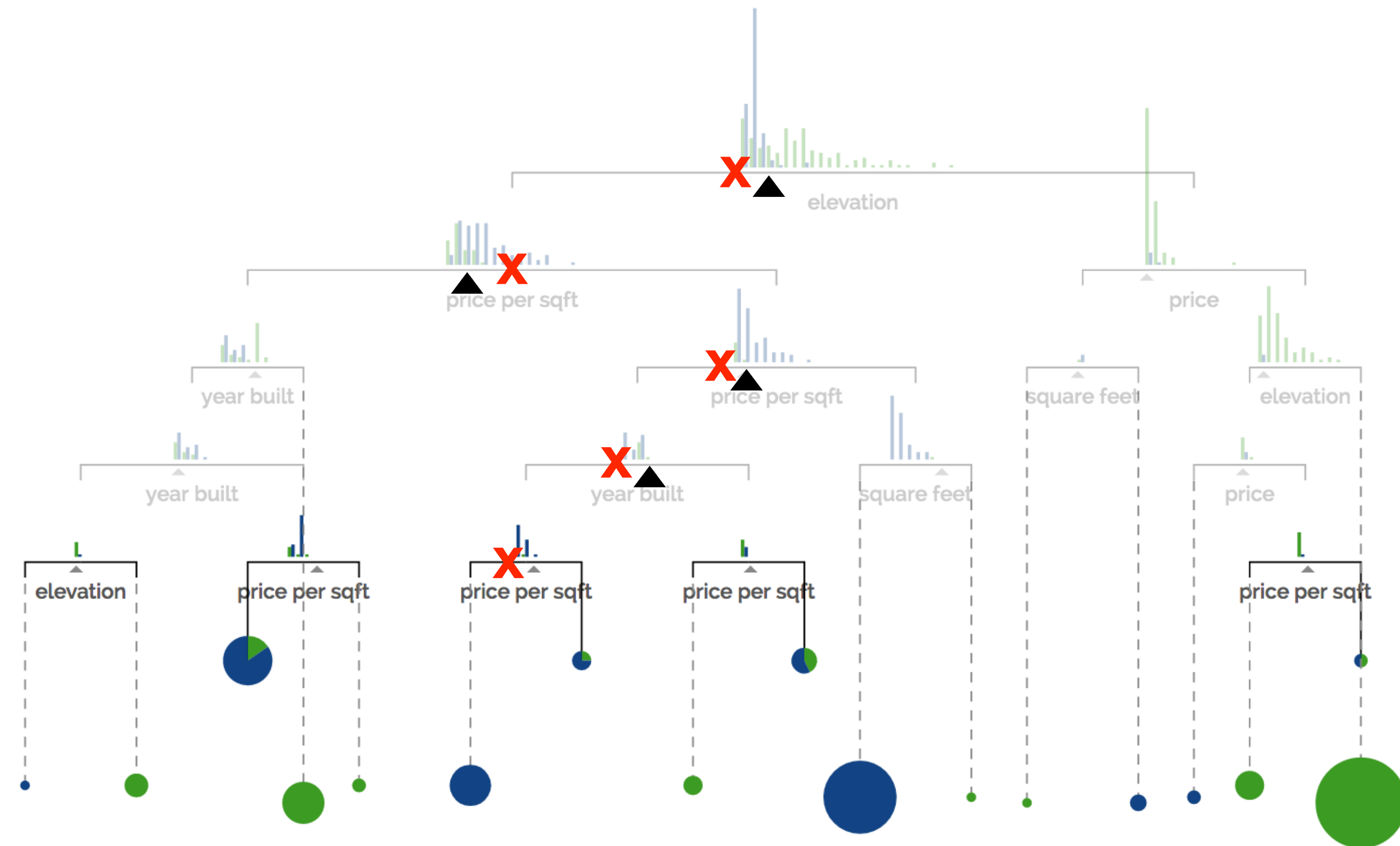
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

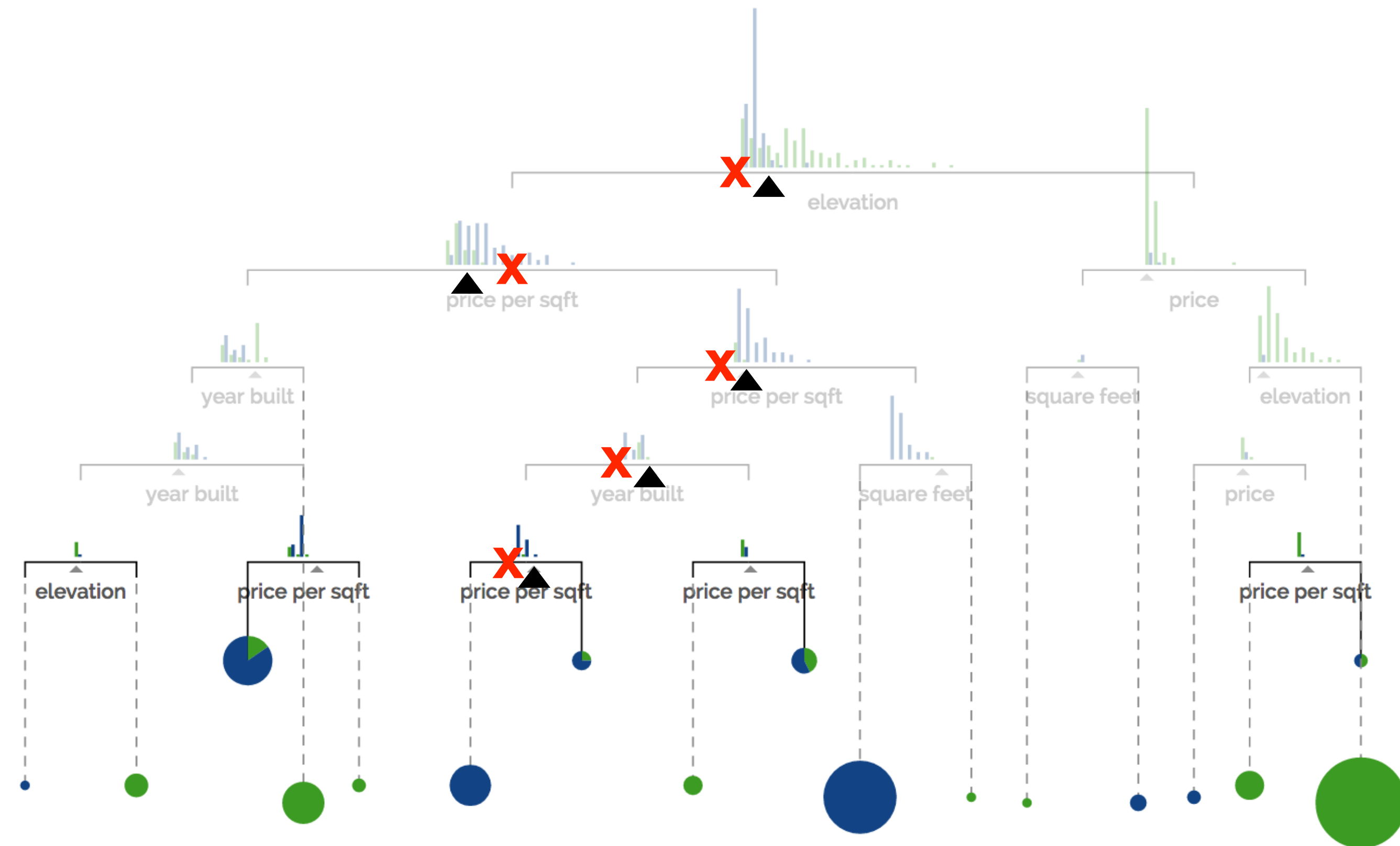
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

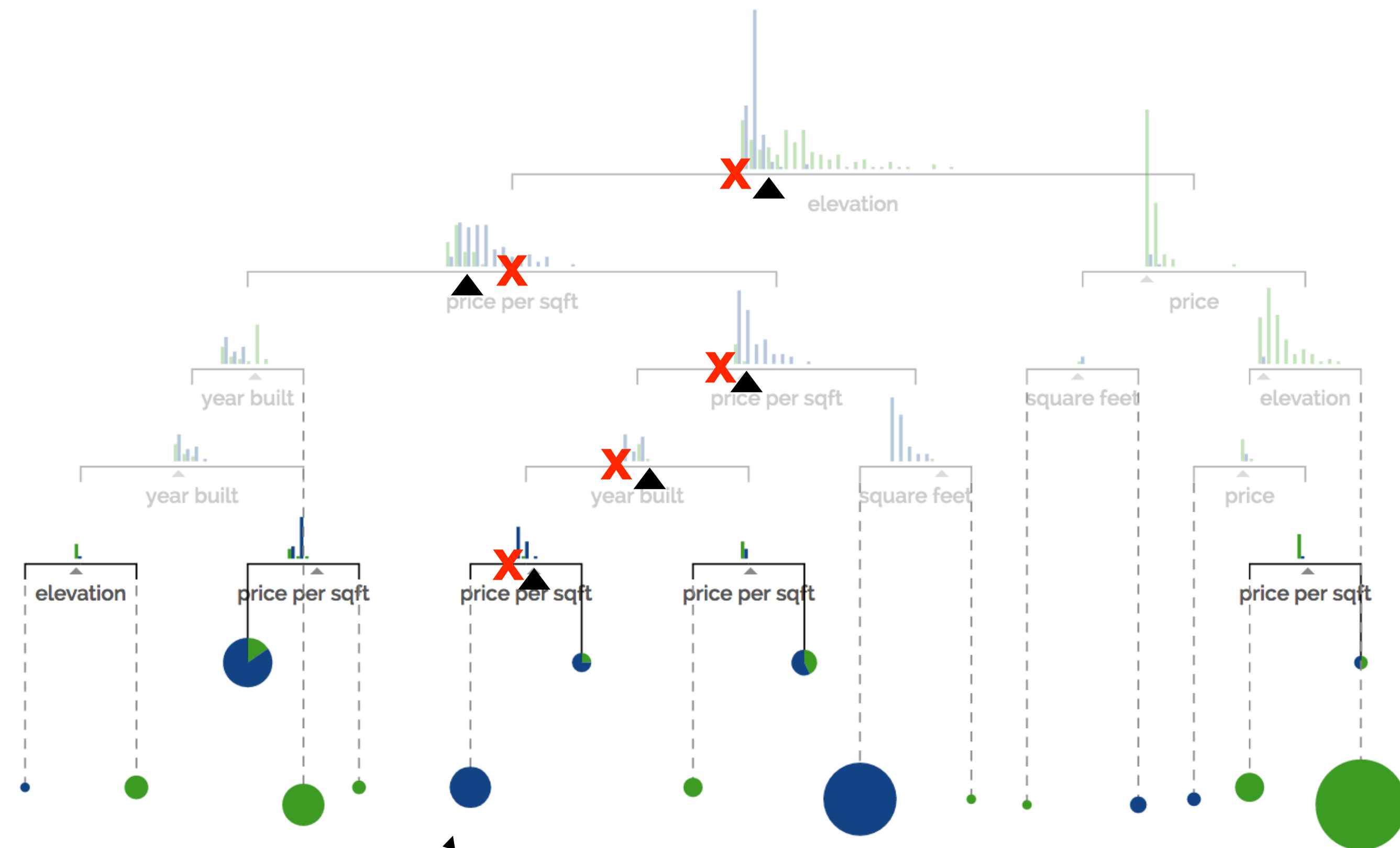
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Predict *New York*

Decision tree learning pseudocode

- function learn(dataset X, labels Y):
 - ▶ if stopping_criterion:
 - $n \leftarrow$ new leaf, $n.\text{prediction} = \text{majority}(Y)$
 - ▶ else:
 - splits $\leftarrow$ list of allowed split tests # e.g., feature j = value
 - for s in splits:
 - criterion[s] $\leftarrow$ evaluate(X, Y, s) # compute split criterion
 - best_s $\leftarrow$ argmax(criterion)
 - mask $\leftarrow$ apply best_s to get T/F for each row of X
 - $n \leftarrow$ new node
 - $n.\text{split} = \text{best_s}$
 - $n.\text{left} \leftarrow \text{learn}(X[\text{mask},:], Y[\text{mask}])$
 - $n.\text{right} \leftarrow \text{learn}(X[\sim\text{mask},:], Y[\sim\text{mask}])$
 - ▶ return n

Categorical, continuous features

- Often we have features with more than two values
 - ▶ $\text{color} \in \{\text{red, green, blue, cyan, magenta, yellow}\}$
 - ▶ $\text{length} \in [2\text{cm}, 7\text{cm}]$
- Typically we still use ***binary*** splits when learning a tree
 - ▶ e.g., $[\text{color} = \text{cyan}]$ or $[\text{length} \leq 4.2\text{cm}]$
 - ▶ greedy tree learning works better if we grow gradually
- Might need to test several possible splits per feature
 - ▶ but still finitely many (even for continuous features!)

Splitting criteria

- Common criteria
 - ▶ minimize training error
 - ▶ maximize mutual information → ID3
 - ▶ minimize Gini impurity criterion → CART

Splitting criteria

- Common criteria
 - ▶ minimize training error
 - ▶ maximize mutual information → ID3
 - ▶ minimize Gini impurity criterion → CART

Mutual information as a splitting criterion

- When we run a decision tree, each example sorts to some leaf l and has true label y
- A good decision tree has high mutual information between l and y (knowing l is helpful in predicting y)
- Splitting criterion: which split increases this mutual information the most, as measured on the training set?

Underfitting and Overfitting in Decision Tree Learning

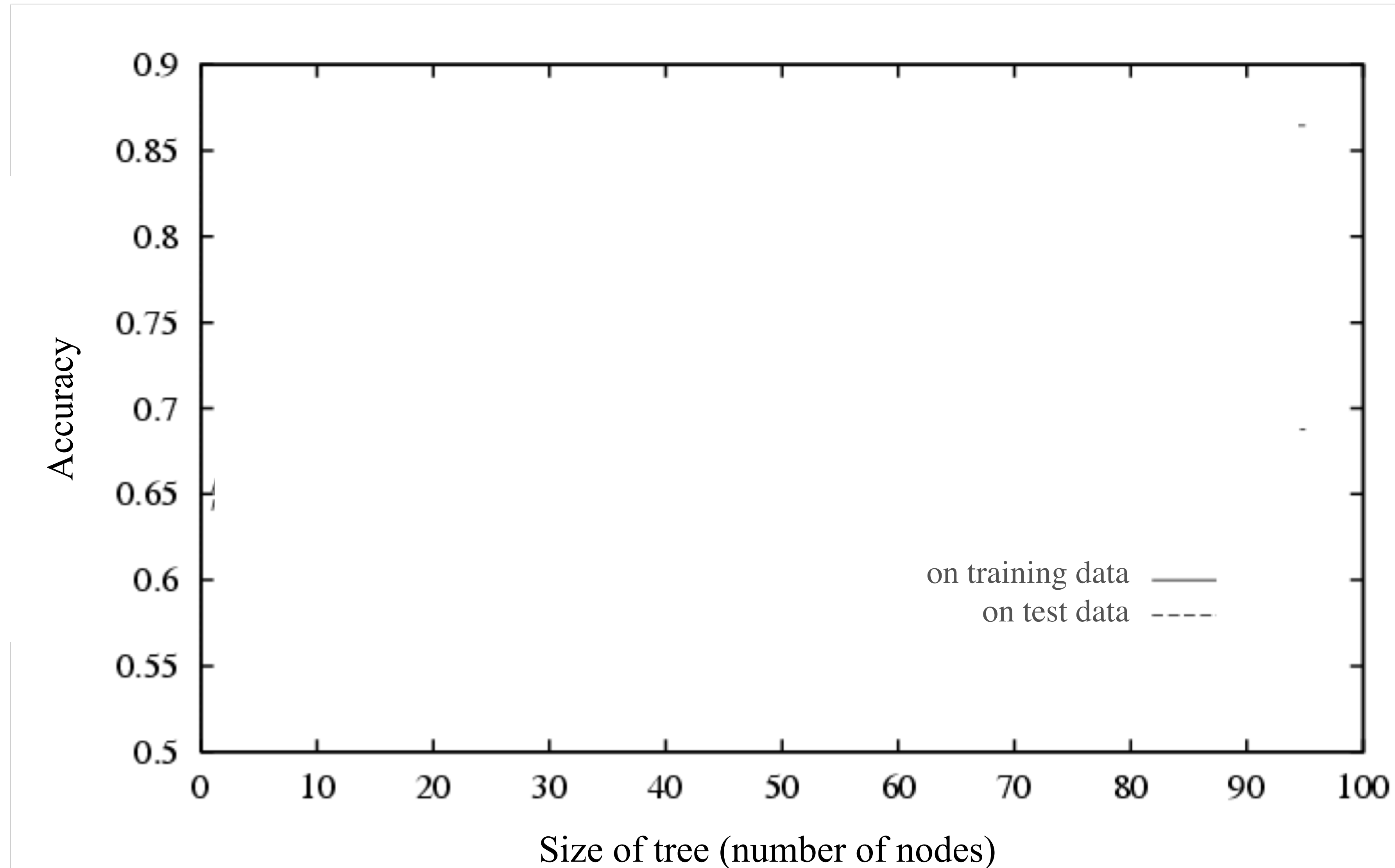


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

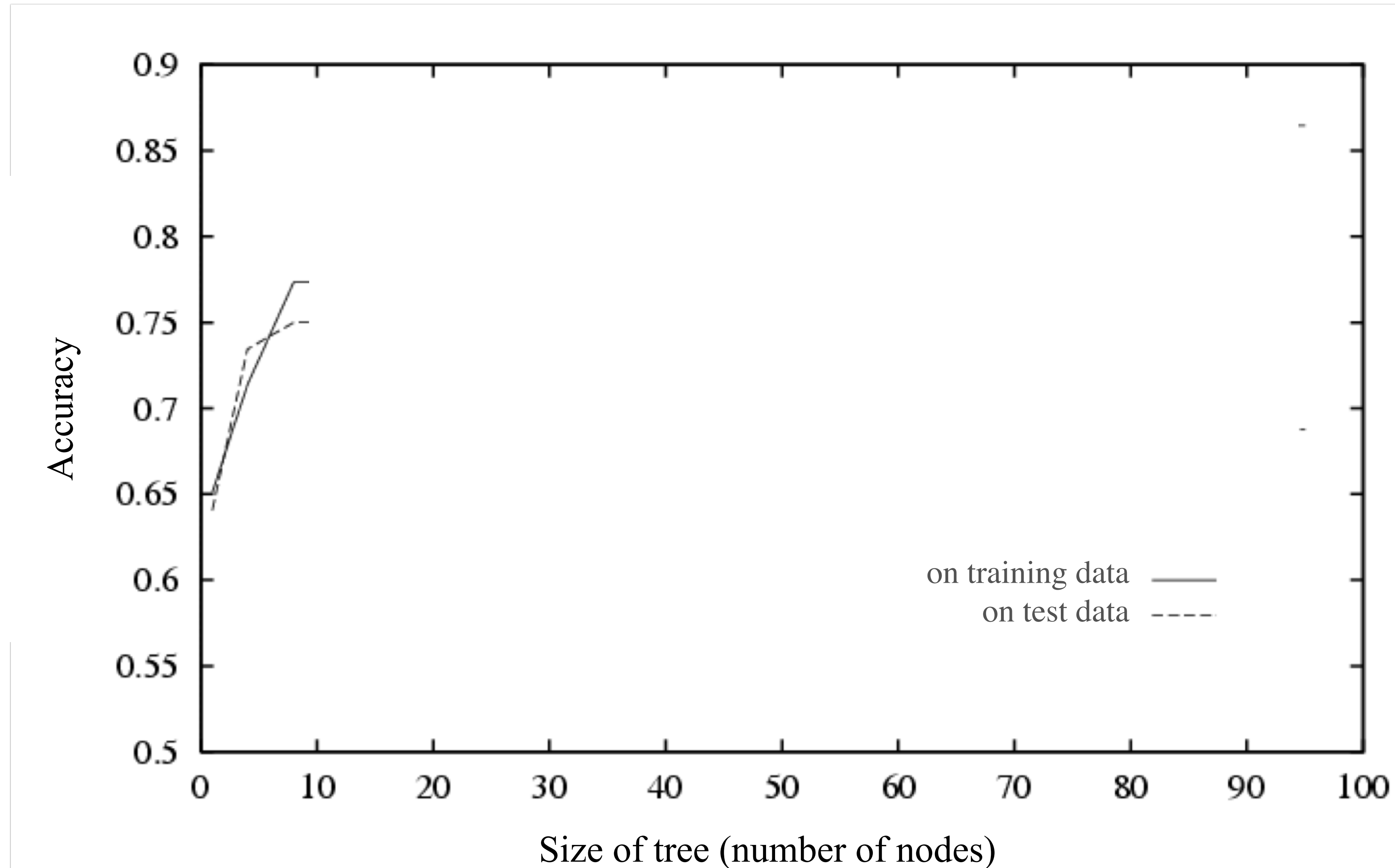


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

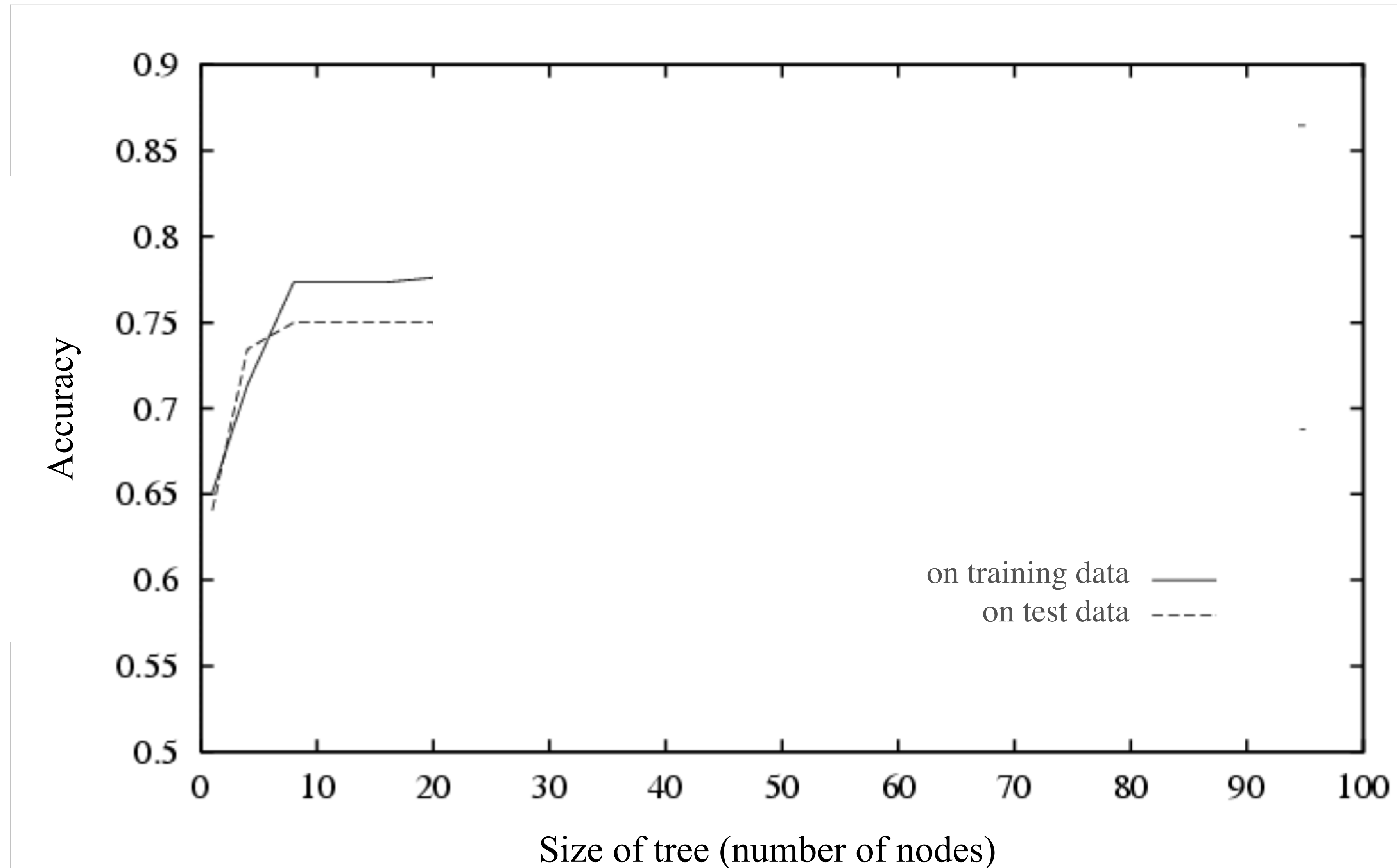


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

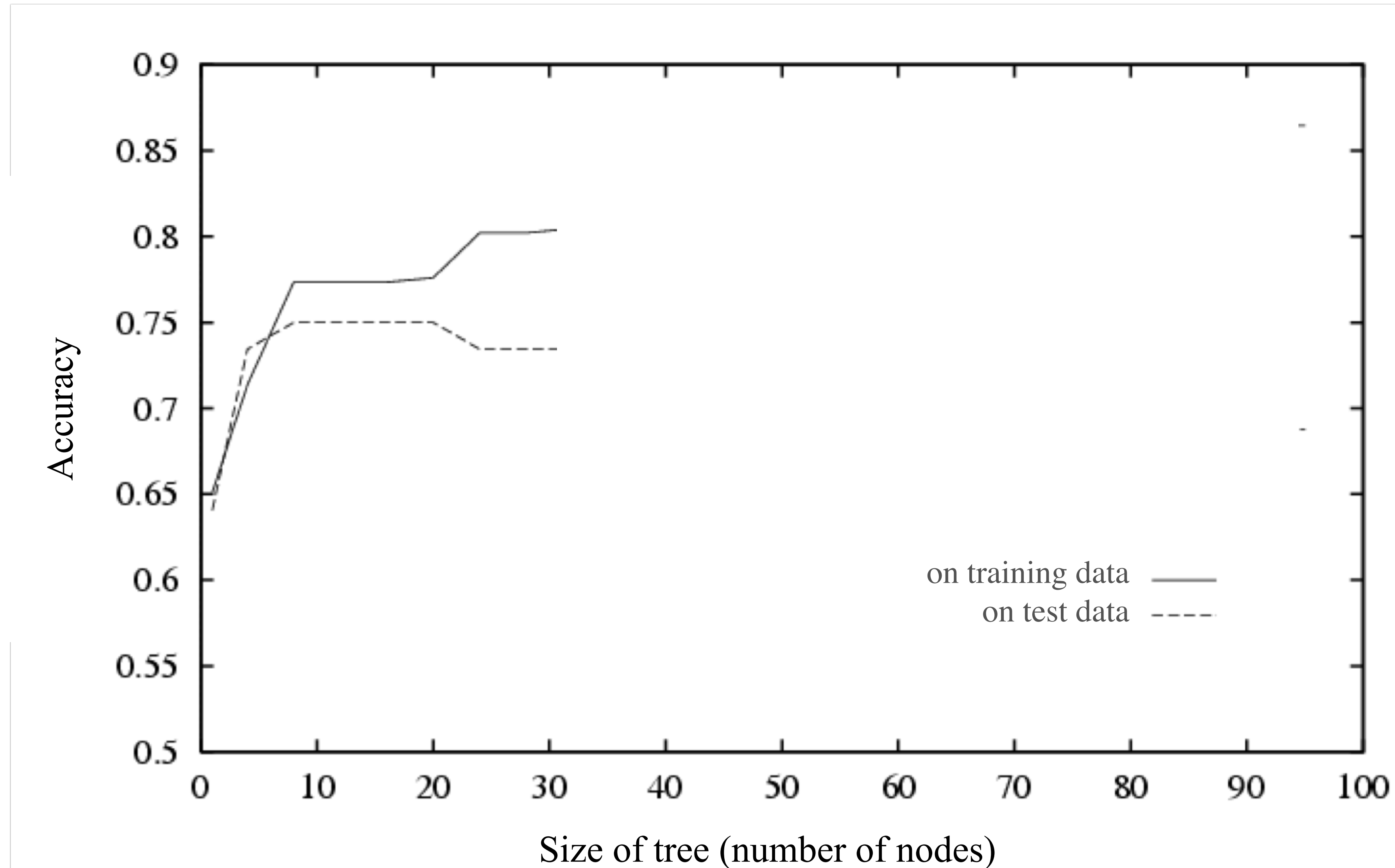


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

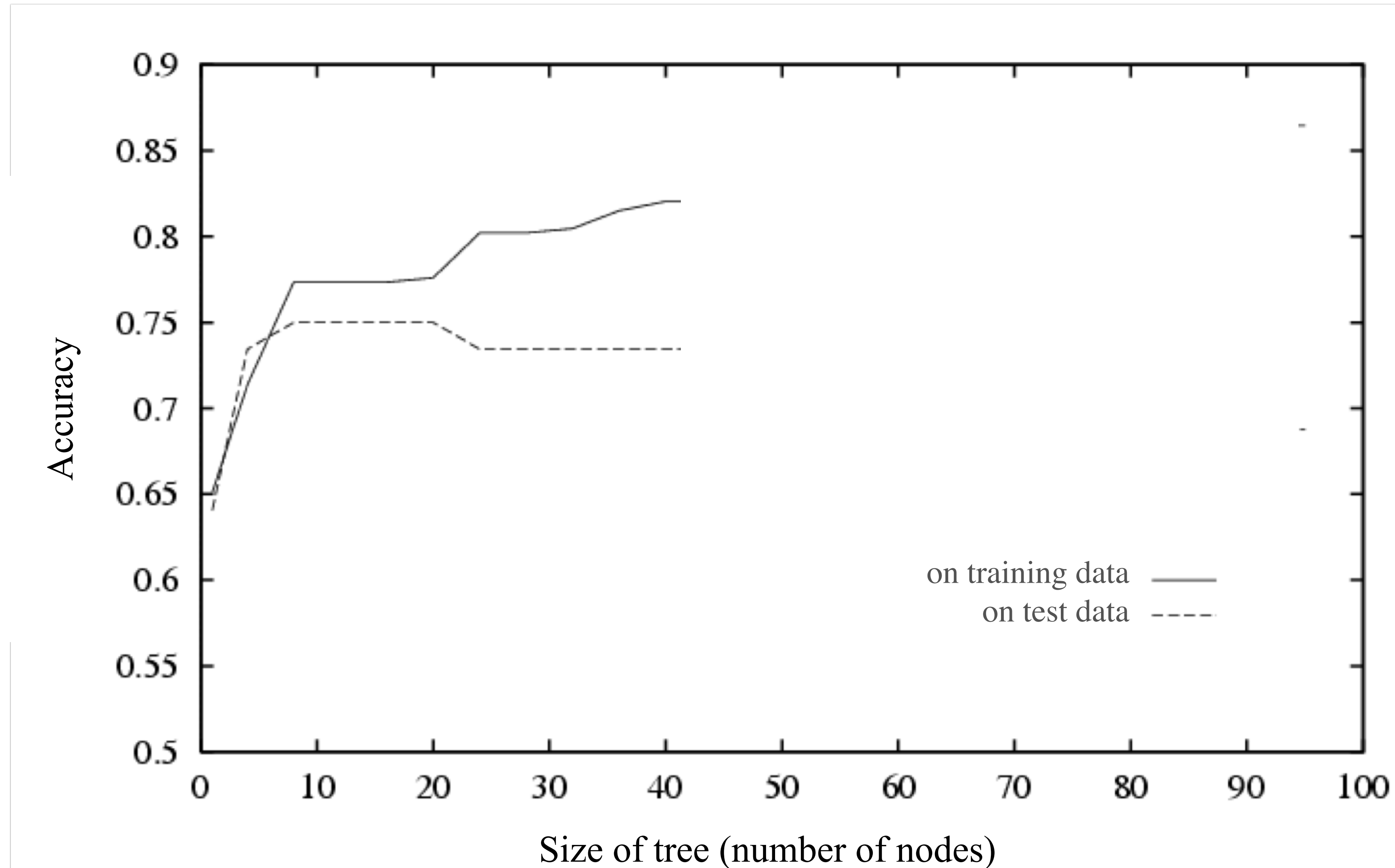


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

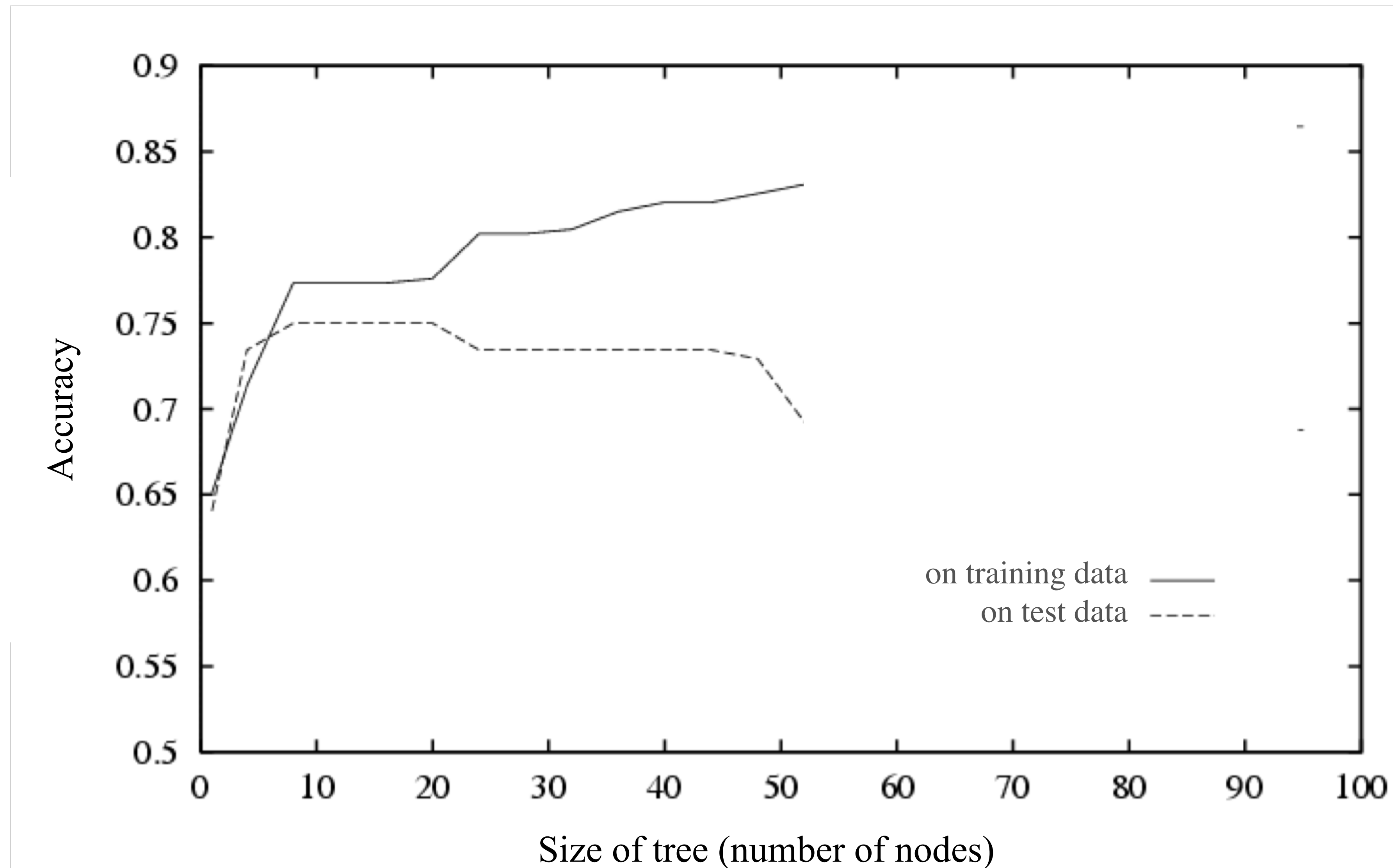


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

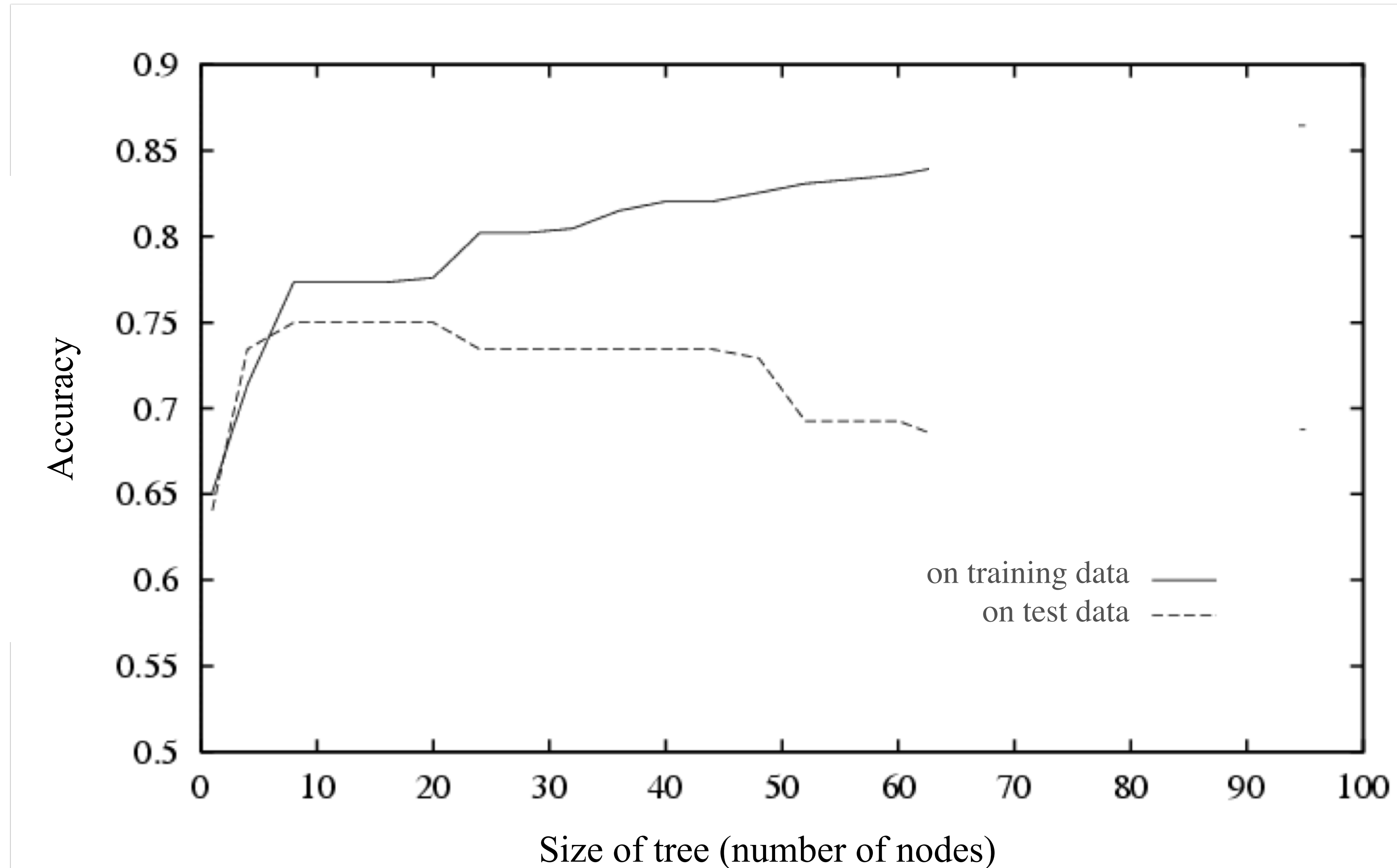


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

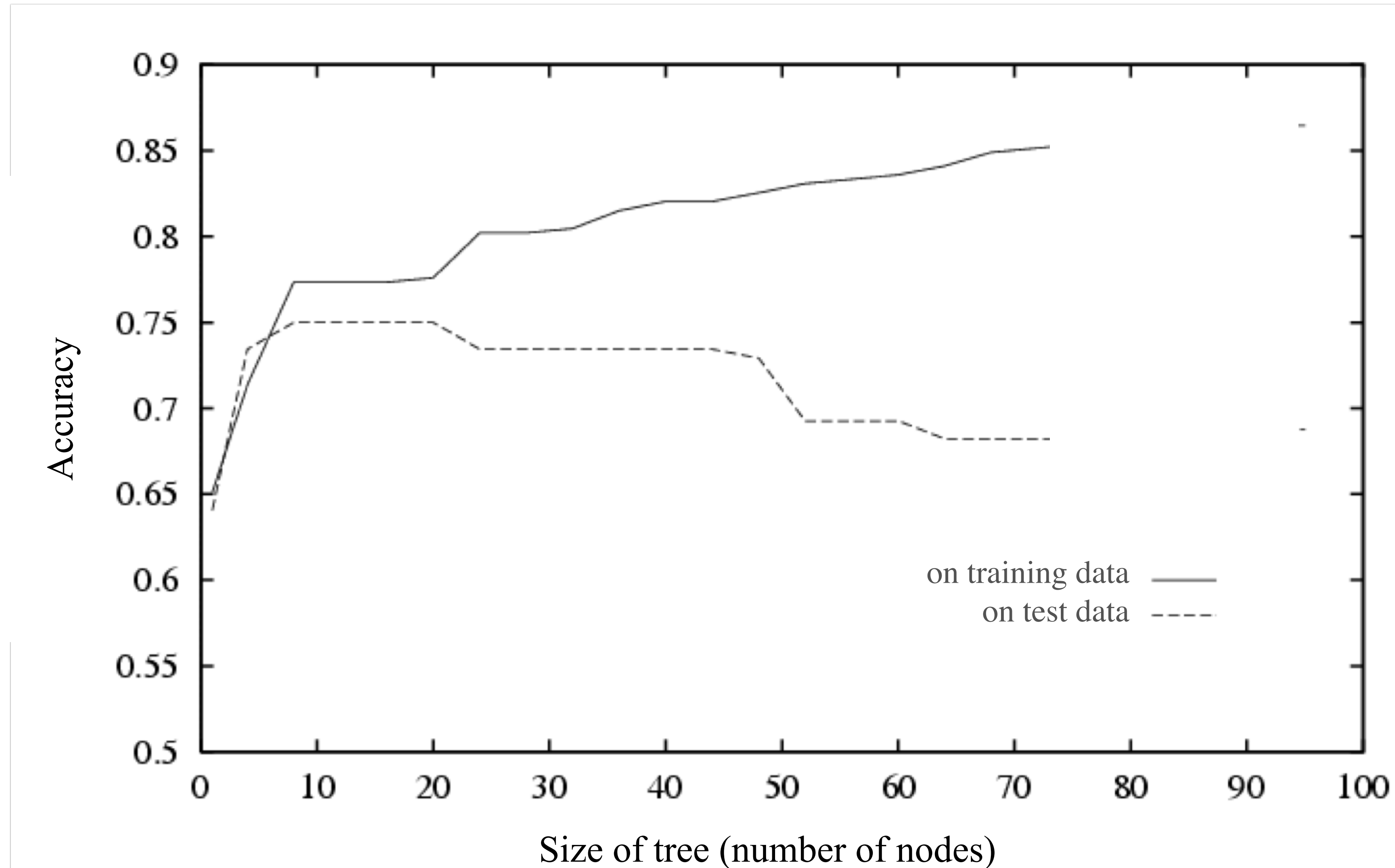


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

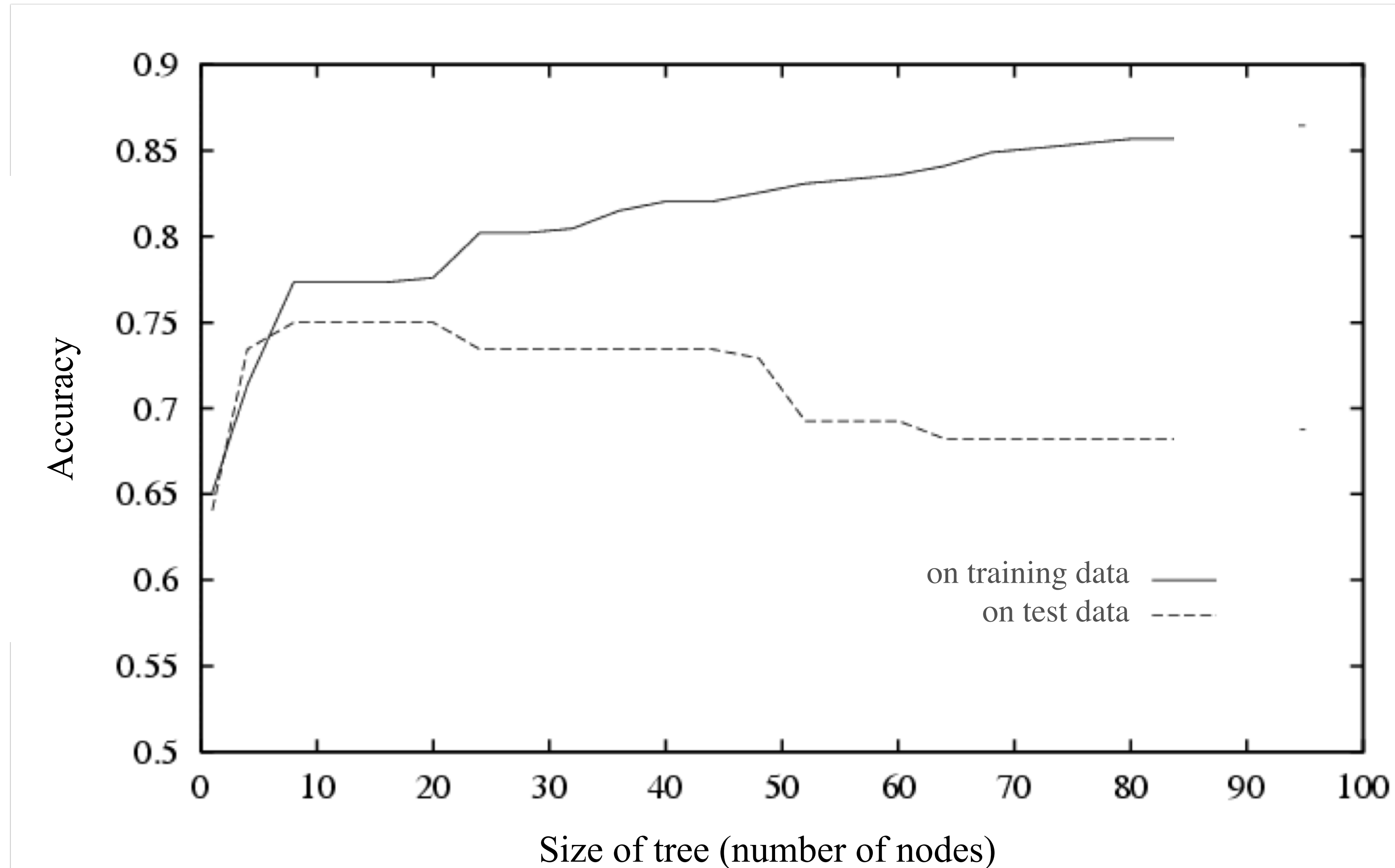


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

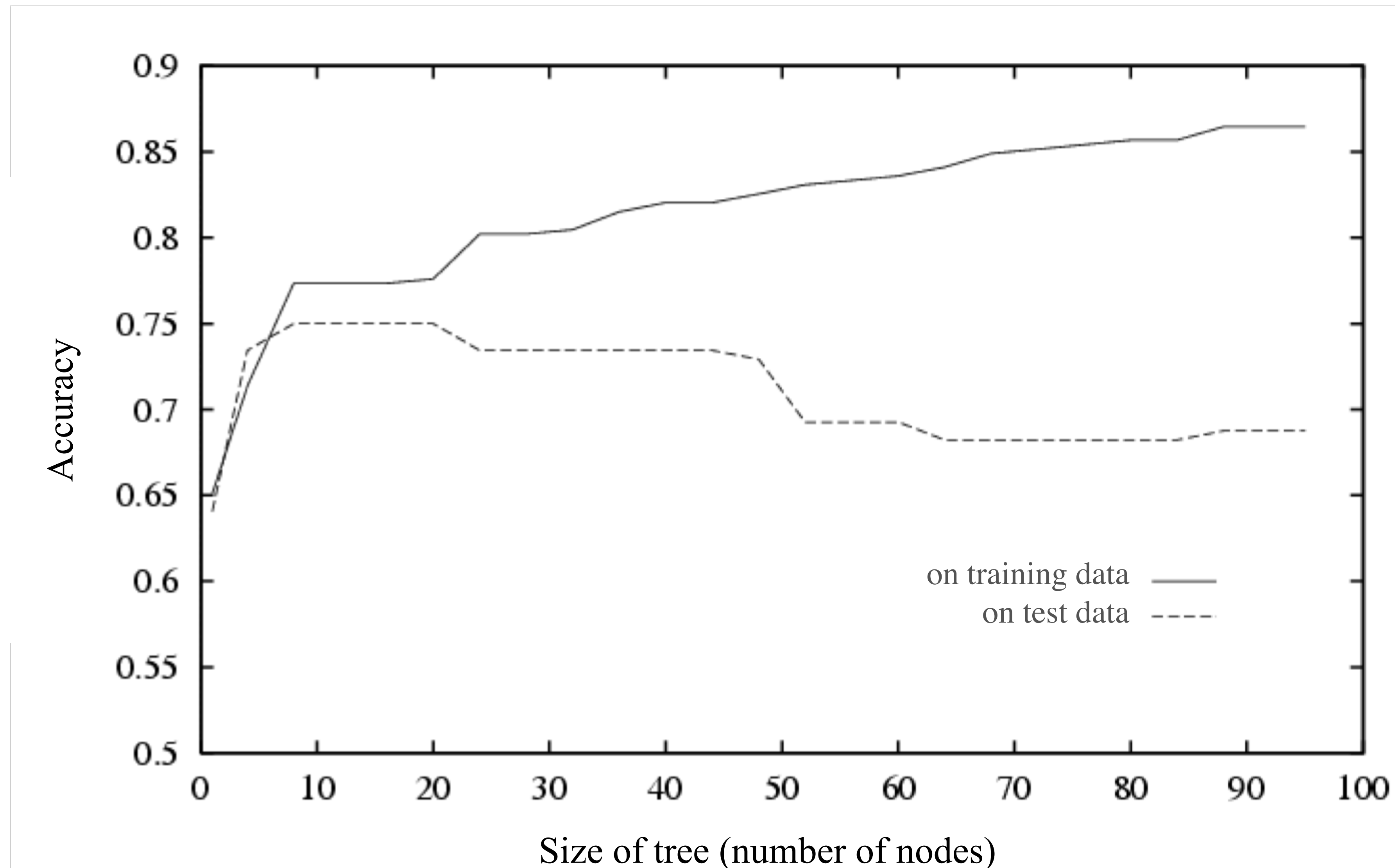


Figure from Tom Mitchell

How can we avoid overfitting in decision trees?

- Need a measure of simplicity/complexity
 - ▶ depth, number of nodes/leaves, size (# examples) of leaf
- Some proposals for minimizing these measures:
 - ▶ do not grow tree beyond some maximum depth or minimum leaf size
 - ▶ do not split if criterion (e.g. increase in MI) is too low
 - ▶ stop growing when the change in criterion is not statistically significant
 - ▶ grow the entire tree, then prune

Prune

- Prune: undo some splits, making tree simpler (reduce depth and number of leaves, increase examples/leaf)
 - ▶ replace pruned subtree with a leaf, predict *majority*
- Idea: measure the predicted test error, prune a node and its subtree if doing so improves prediction
- How do we measure predicted test error?
 - ▶ **hide** some data from the learner — take what would have been our training data and split it
 - ▶ use one part as the actual training set (learner sees this)
 - ▶ use other part to stand in for test set (learner doesn't see this) — called **validation set**
- Original test set stays the same — can still use it to evaluate final pruned tree
 - ▶ why can't we use test set to prune?

Train/ validation/ test split

validation is sometimes
called ***hold-out*** set

- Start with available data $\mathcal{D}$
- Split into three parts:
 - ▶ $\mathcal{D}_{\text{train}}$: learning the tree
 - ▶ $\mathcal{D}_{\text{val}}$: pruning
 - ▶ $\mathcal{D}_{\text{test}}$: final evaluation
- Tension:
 - ▶ bigger $\mathcal{D}_{\text{train}}$ for better initial learning
 - ▶ bigger $\mathcal{D}_{\text{val}}$ for better pruning
 - ▶ bigger $\mathcal{D}_{\text{test}}$ for accurate evaluation

features			labels
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes
No	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Abnormal	Yes

Train/ validation/ test split

validation is sometimes
called ***hold-out*** set

- Start with available data $\mathcal{D}$
- Split into three parts:
 - ▶ $\mathcal{D}_{\text{train}}$: learning the tree
 - ▶ $\mathcal{D}_{\text{val}}$: pruning
 - ▶ $\mathcal{D}_{\text{test}}$: final evaluation
- Tension:
 - ▶ bigger $\mathcal{D}_{\text{train}}$ for better initial learning
 - ▶ bigger $\mathcal{D}_{\text{val}}$ for better pruning
 - ▶ bigger $\mathcal{D}_{\text{test}}$ for accurate evaluation

features			labels
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes
No	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Abnormal	Yes

Hidden dependence

- Common gotcha: examples not completely independent
 - ▶ surveys given by Bob vs. surveys given by Alice
 - ▶ measurements taken from nearby plots of land
 - ▶ slow changes over time
- Causes two problems:
 - ▶ information leakage: training set contains spoilers about test set (so performance estimates become optimistic)
 - ▶ generalization failure: ML depends on similarity between training and test (*iid* assumption); if something changes, what we learned might not be as relevant

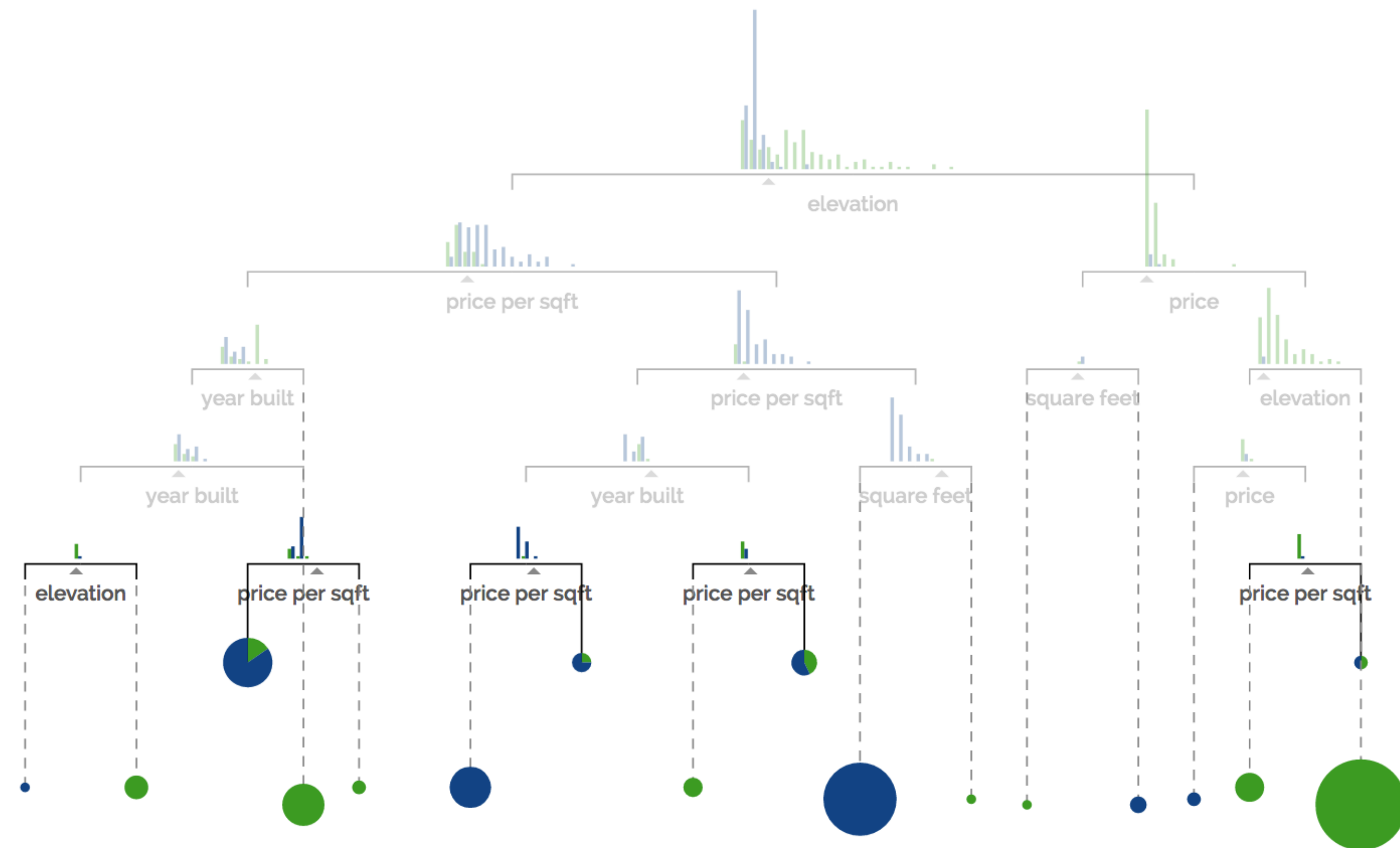
Possible fixes

- Block randomization:
 - ▶ sometimes we have information about dependence (who gave the survey, which plot of land, timestamp)
 - ▶ randomize **blocks** of related examples together to train, validation, or test
 - ▶ e.g., a block of time all goes to test, or all of Alice's surveys go to validation — reduces leakage
- Shuffle before partitioning
 - ▶ can increase leakage, but better match to iid

Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

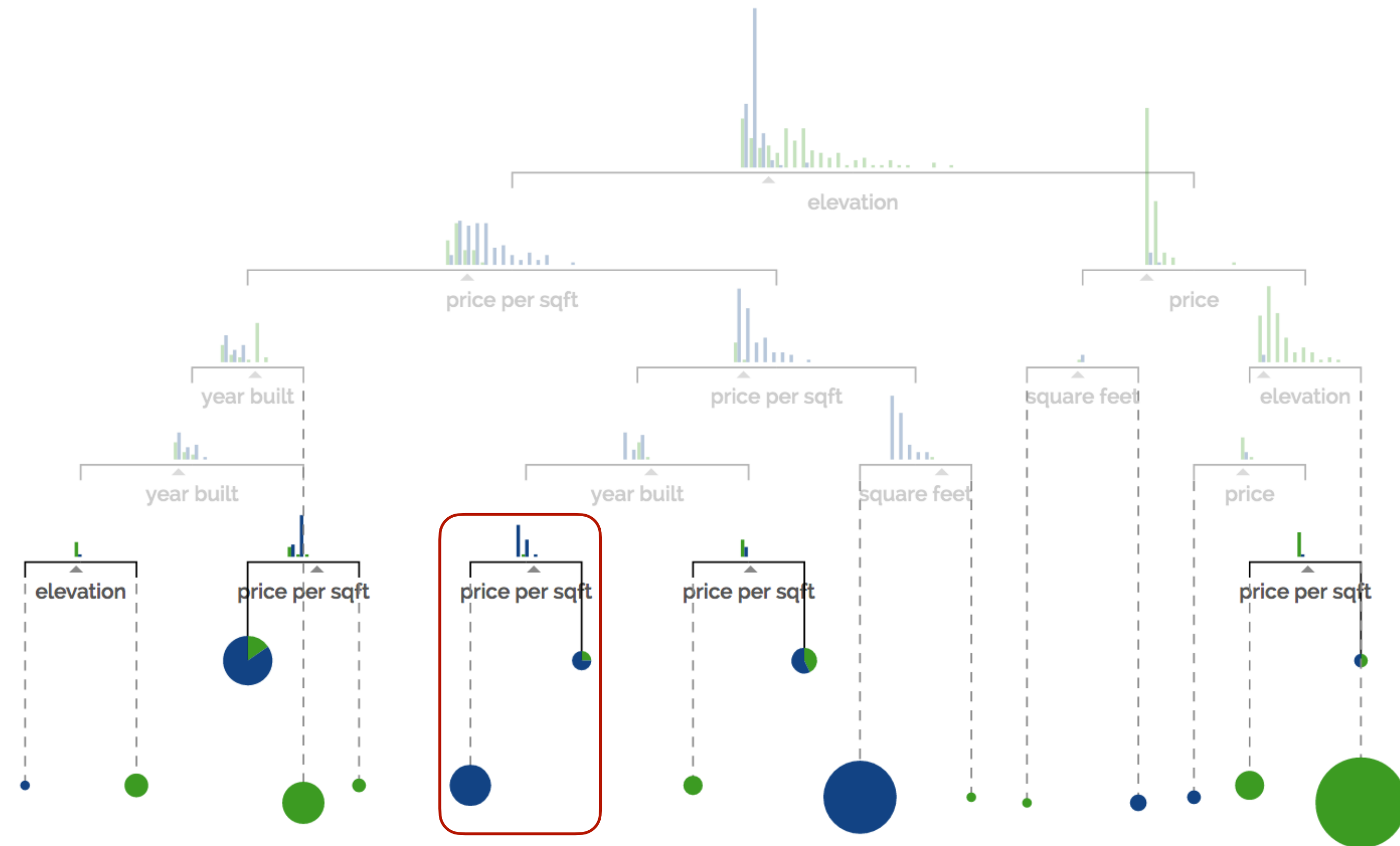
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

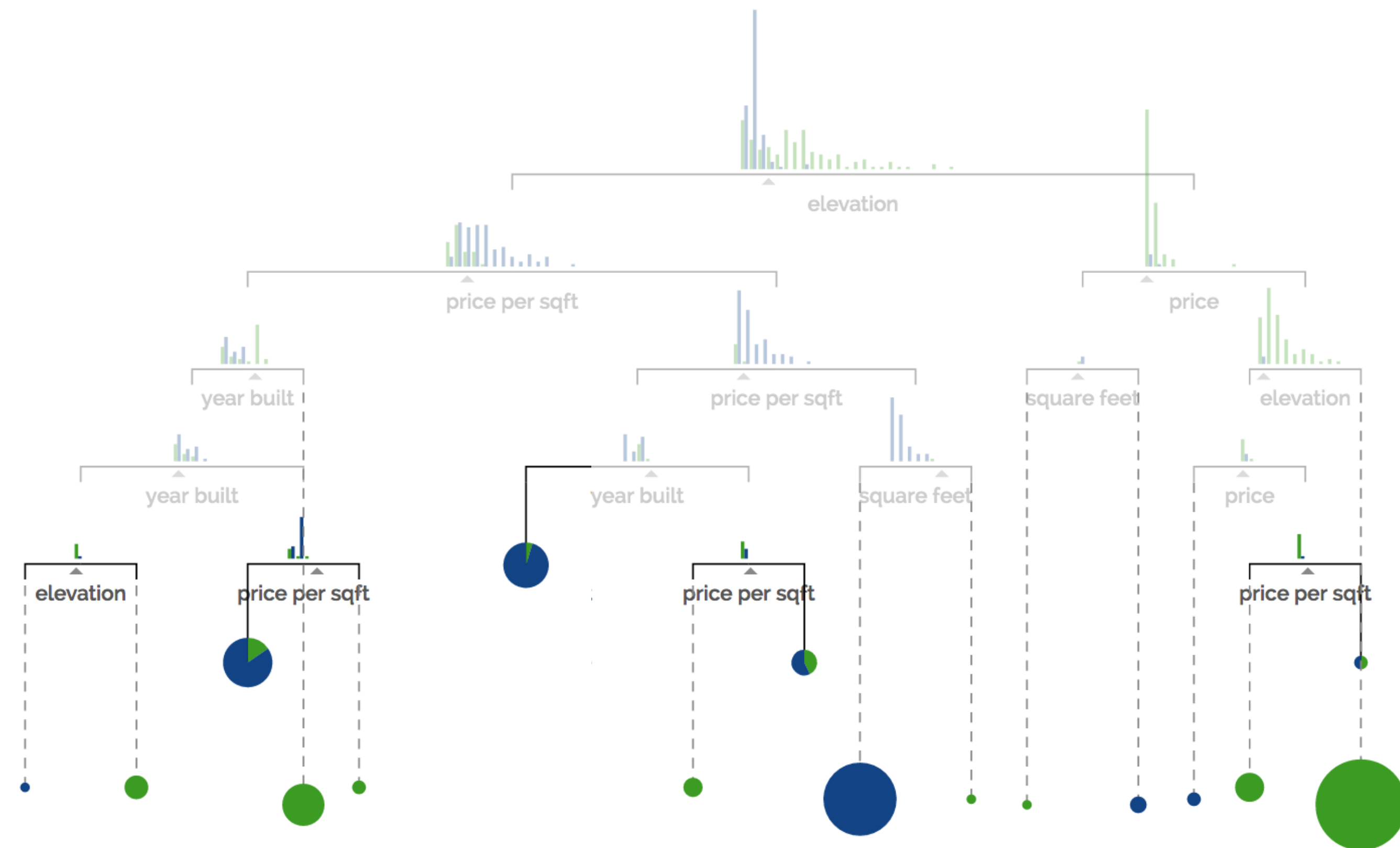
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in **San Francisco** or **New York** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

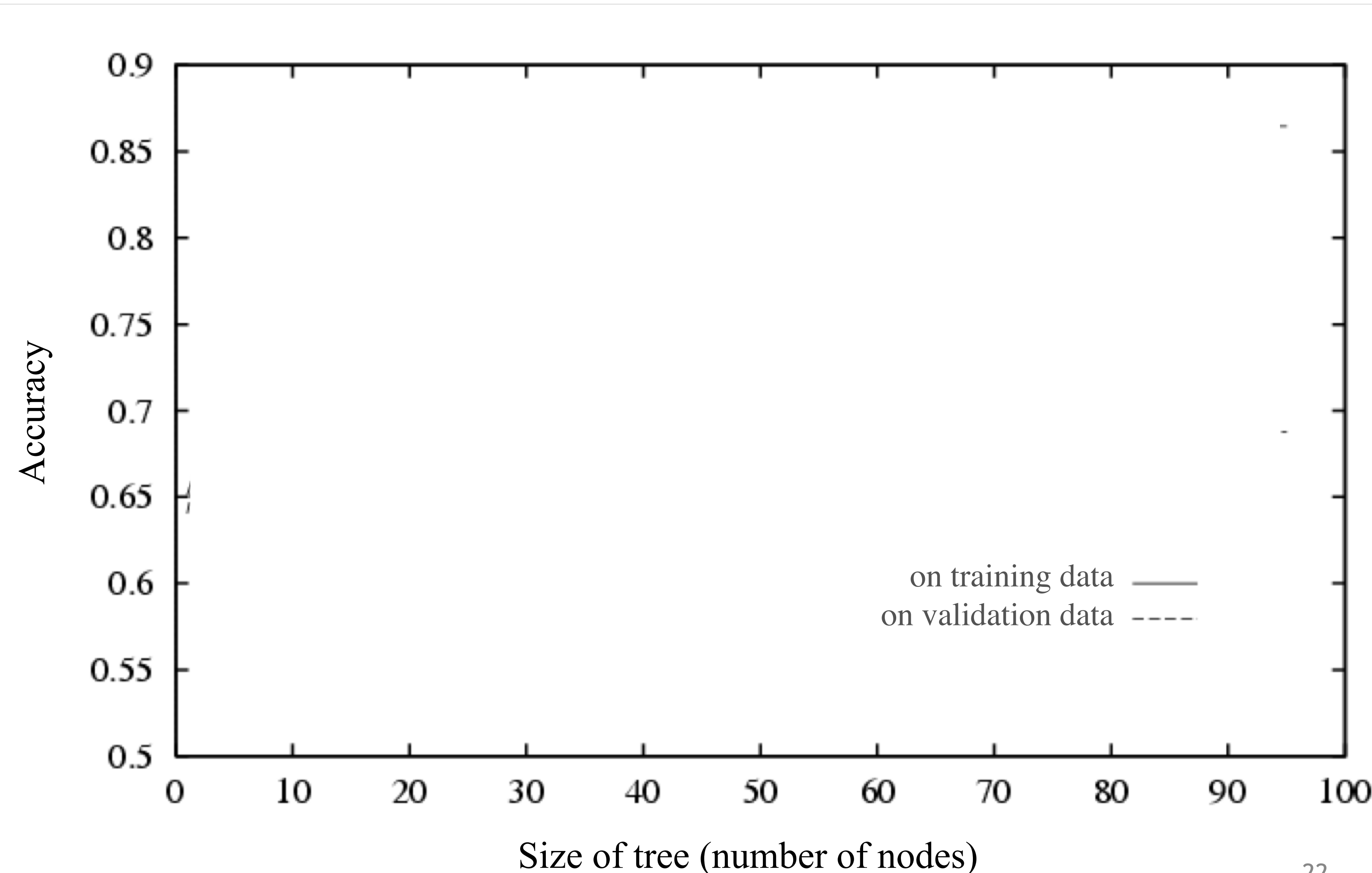


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

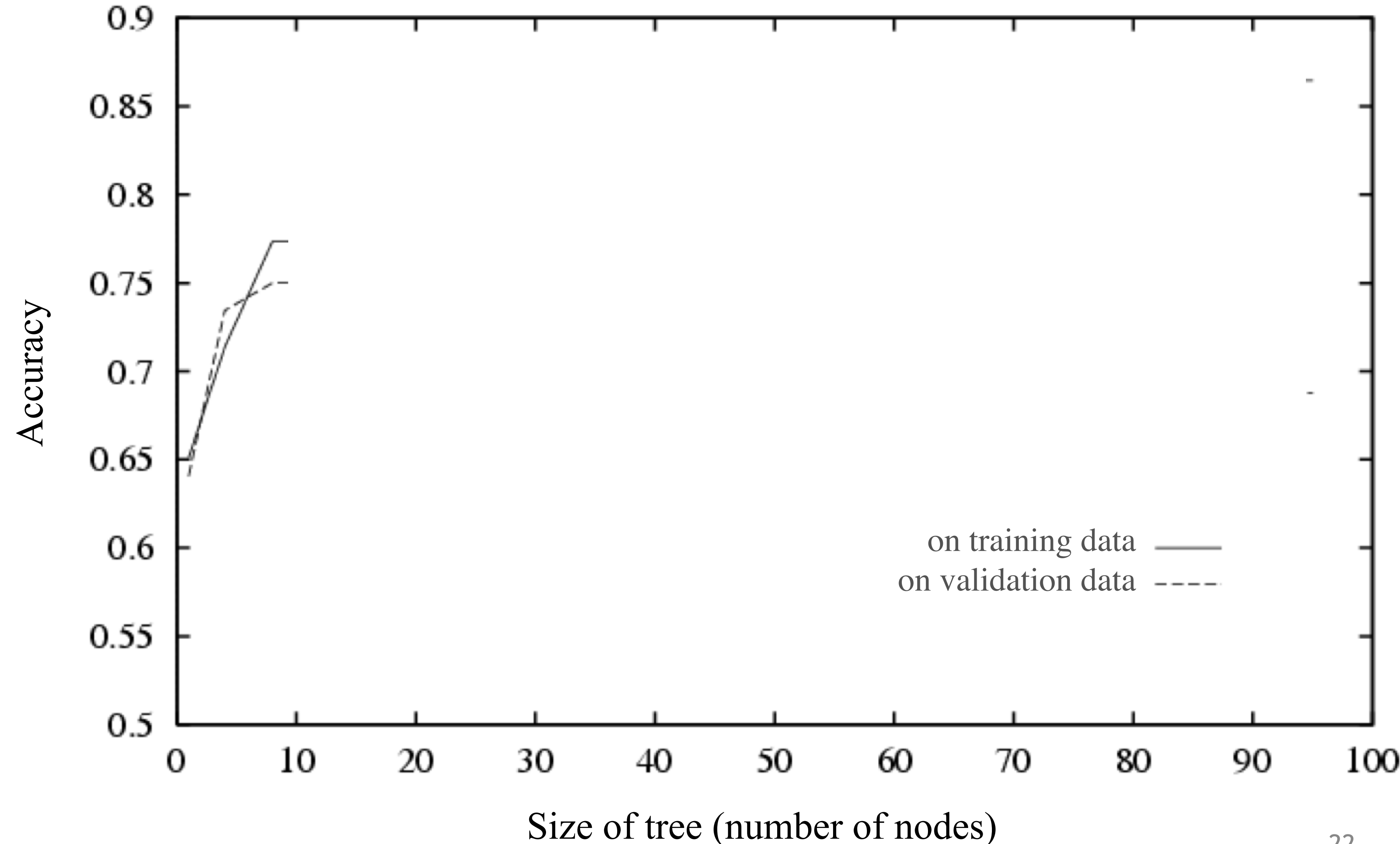


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

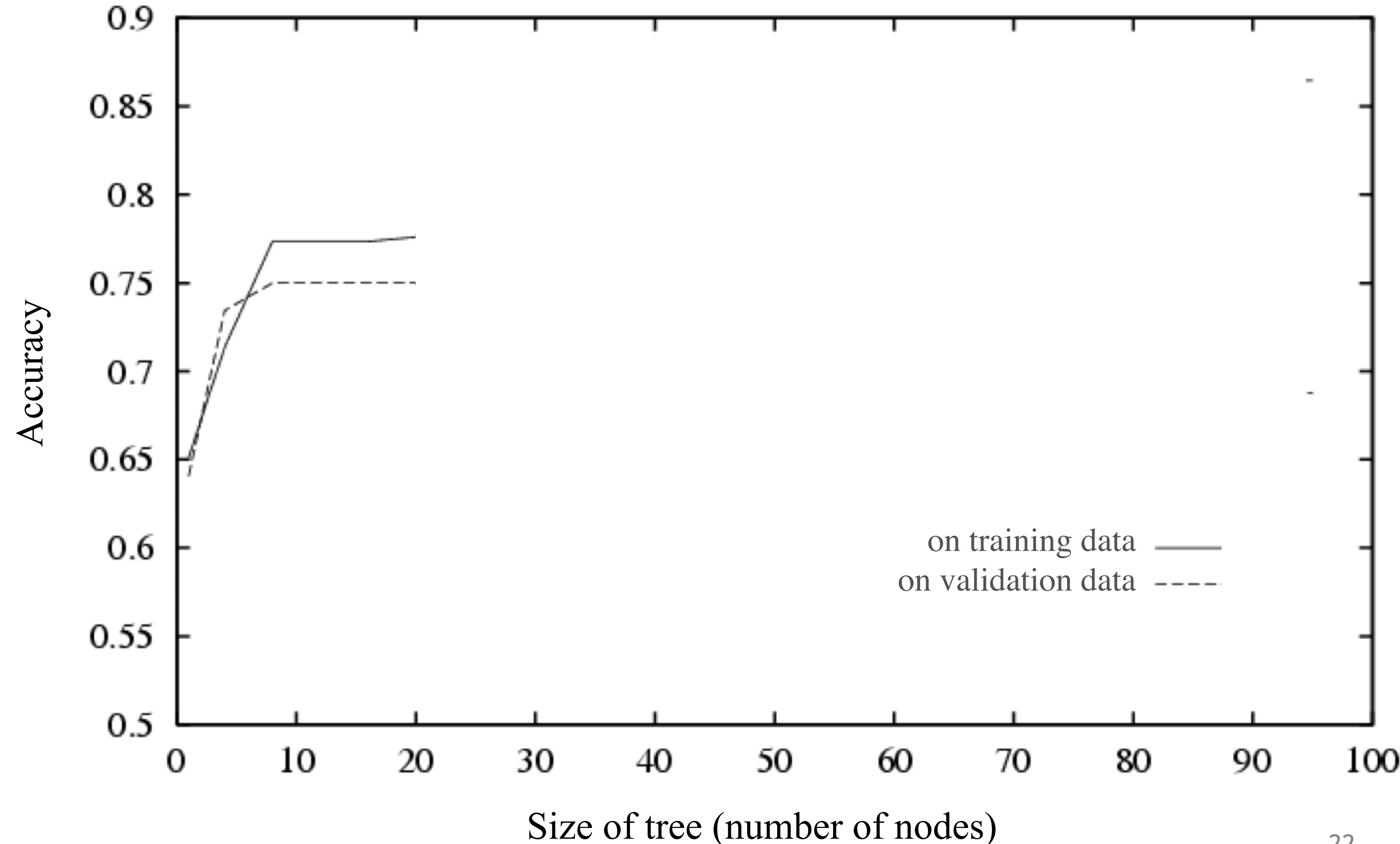


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

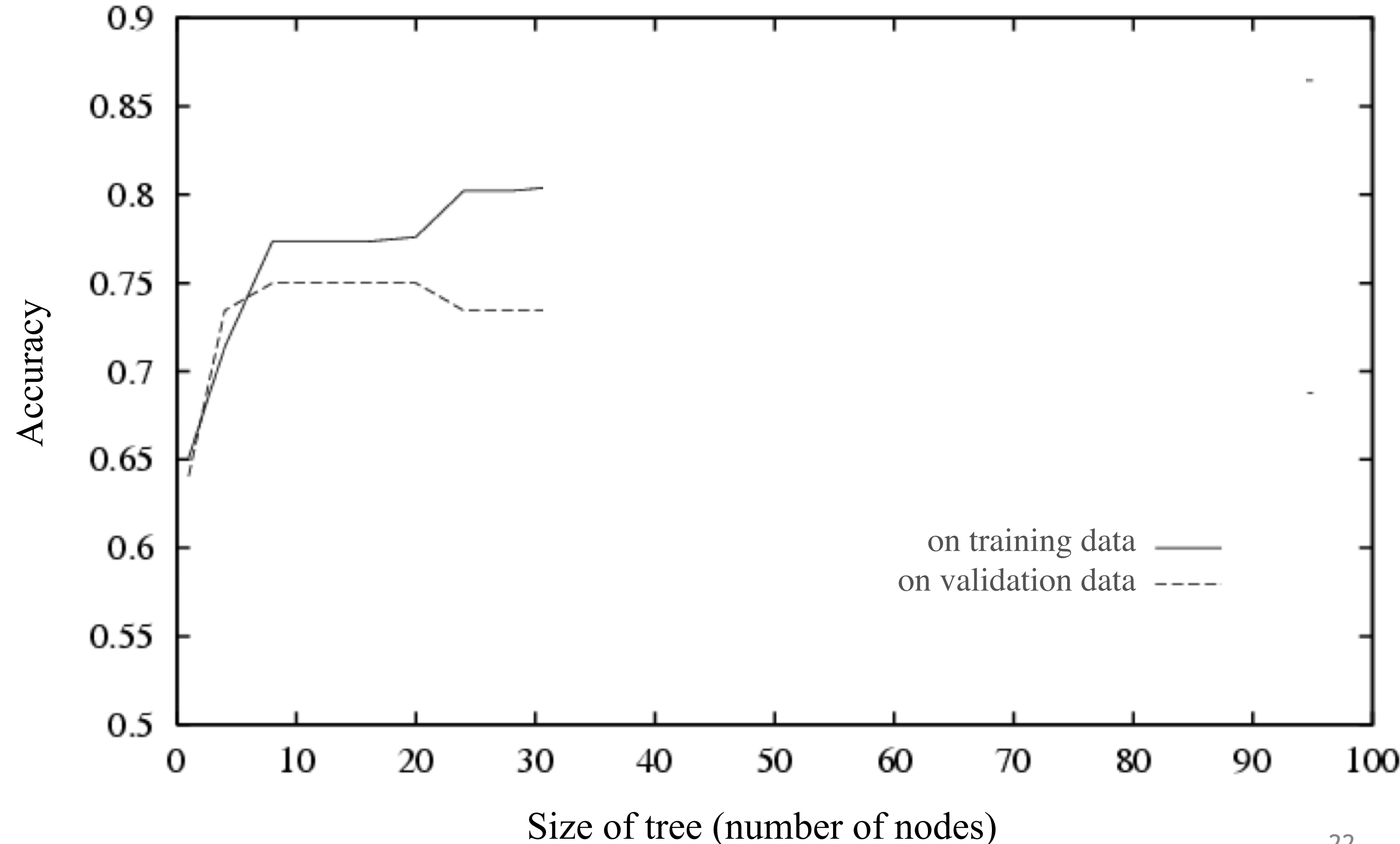


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

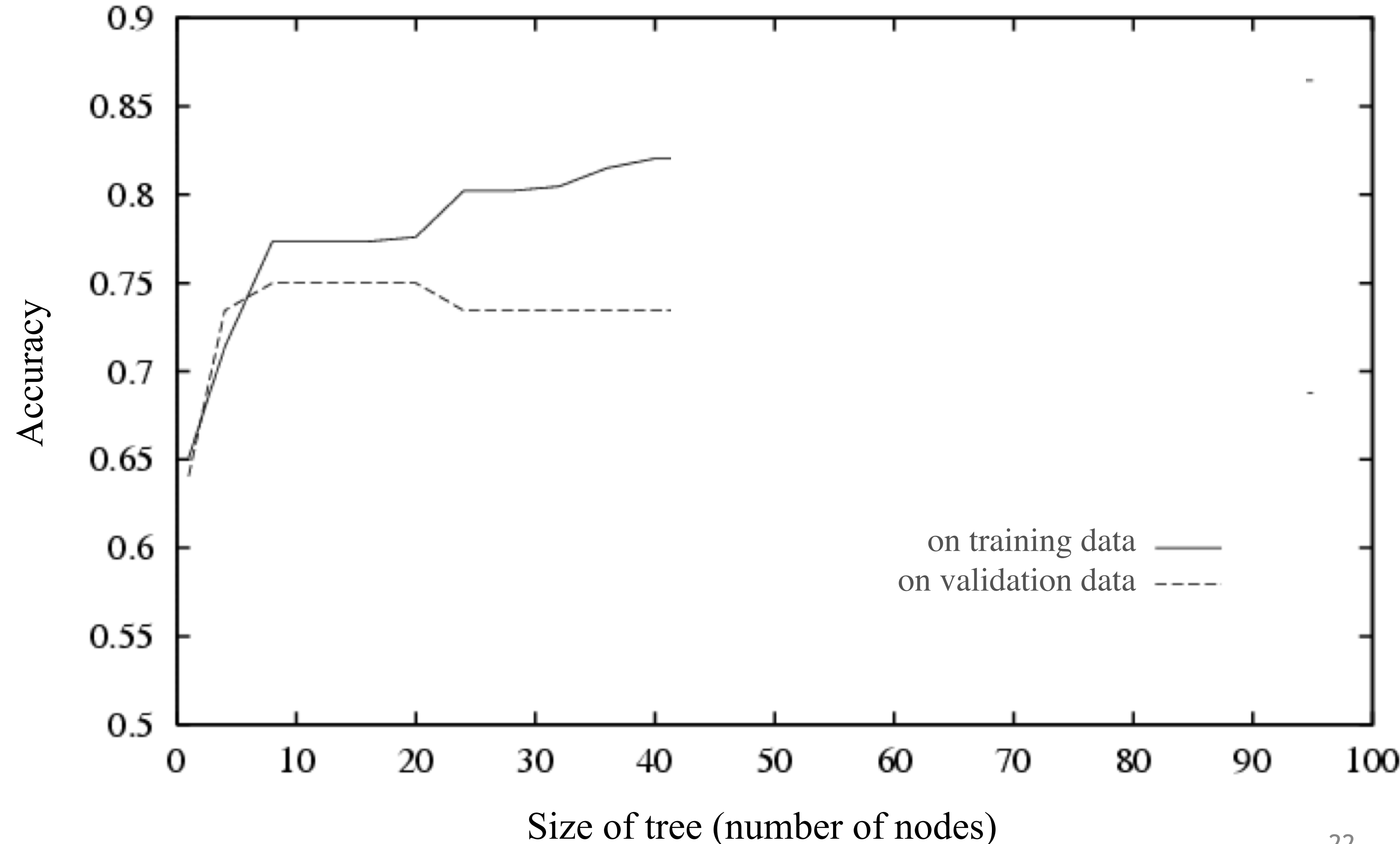


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

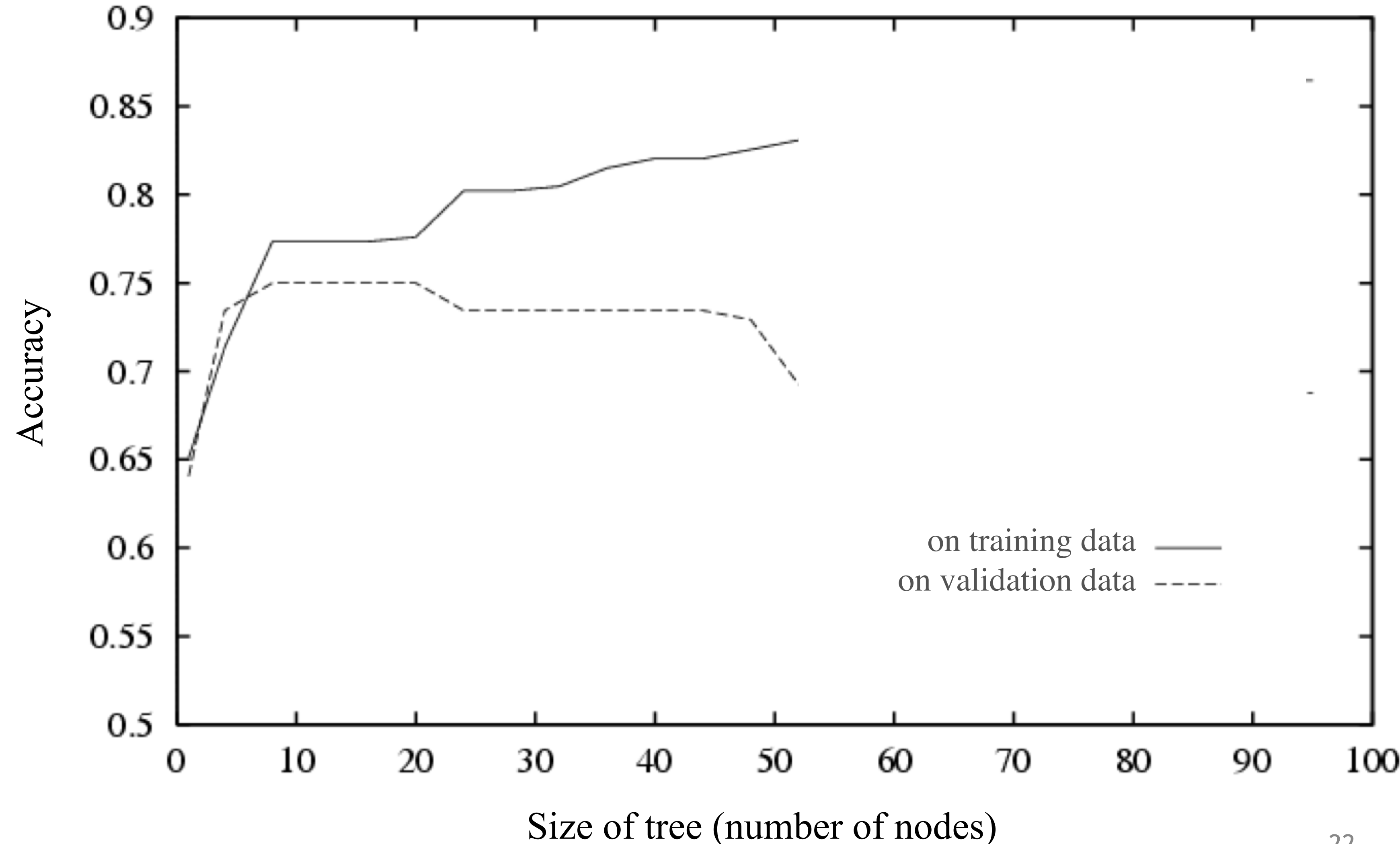


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

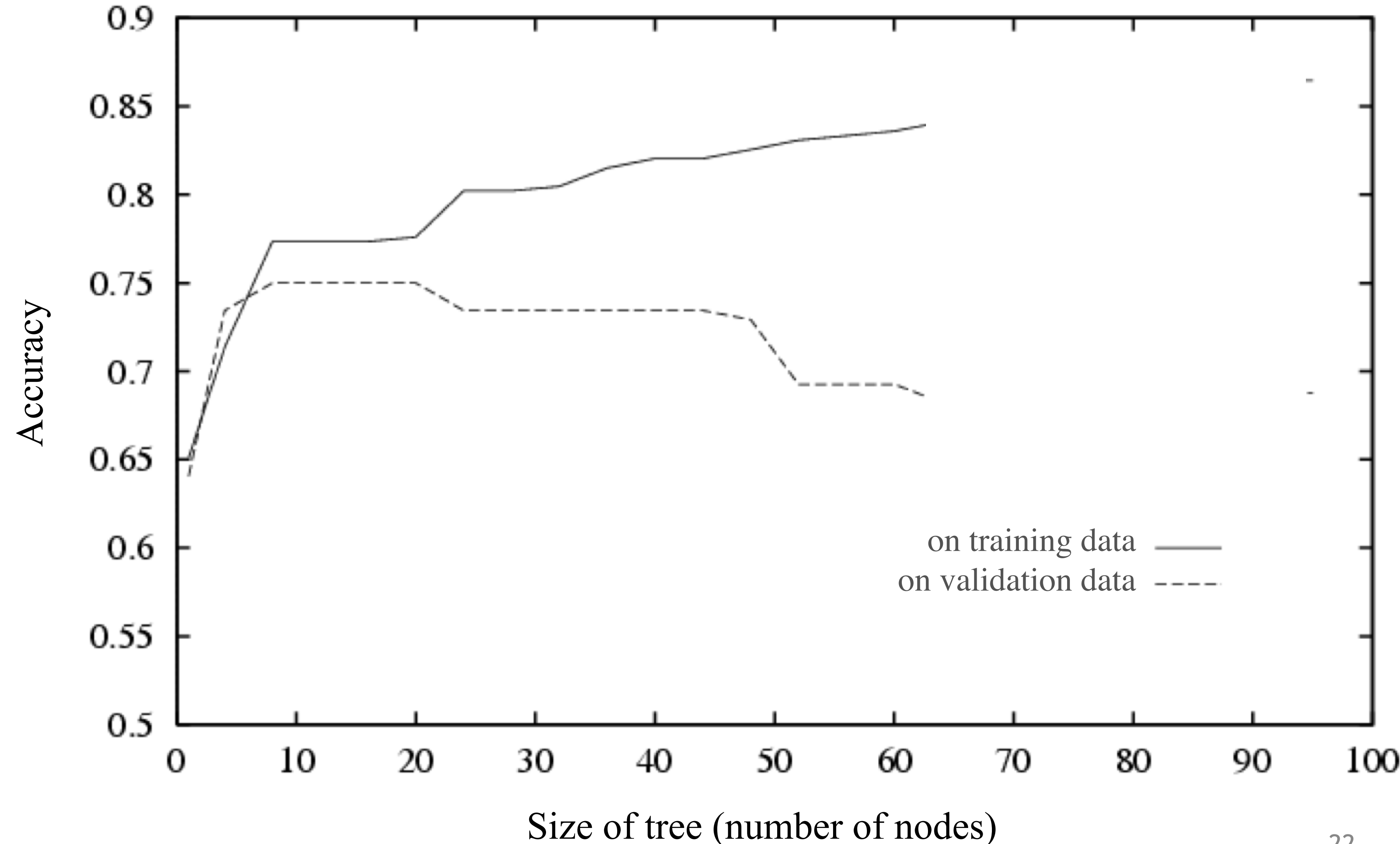


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

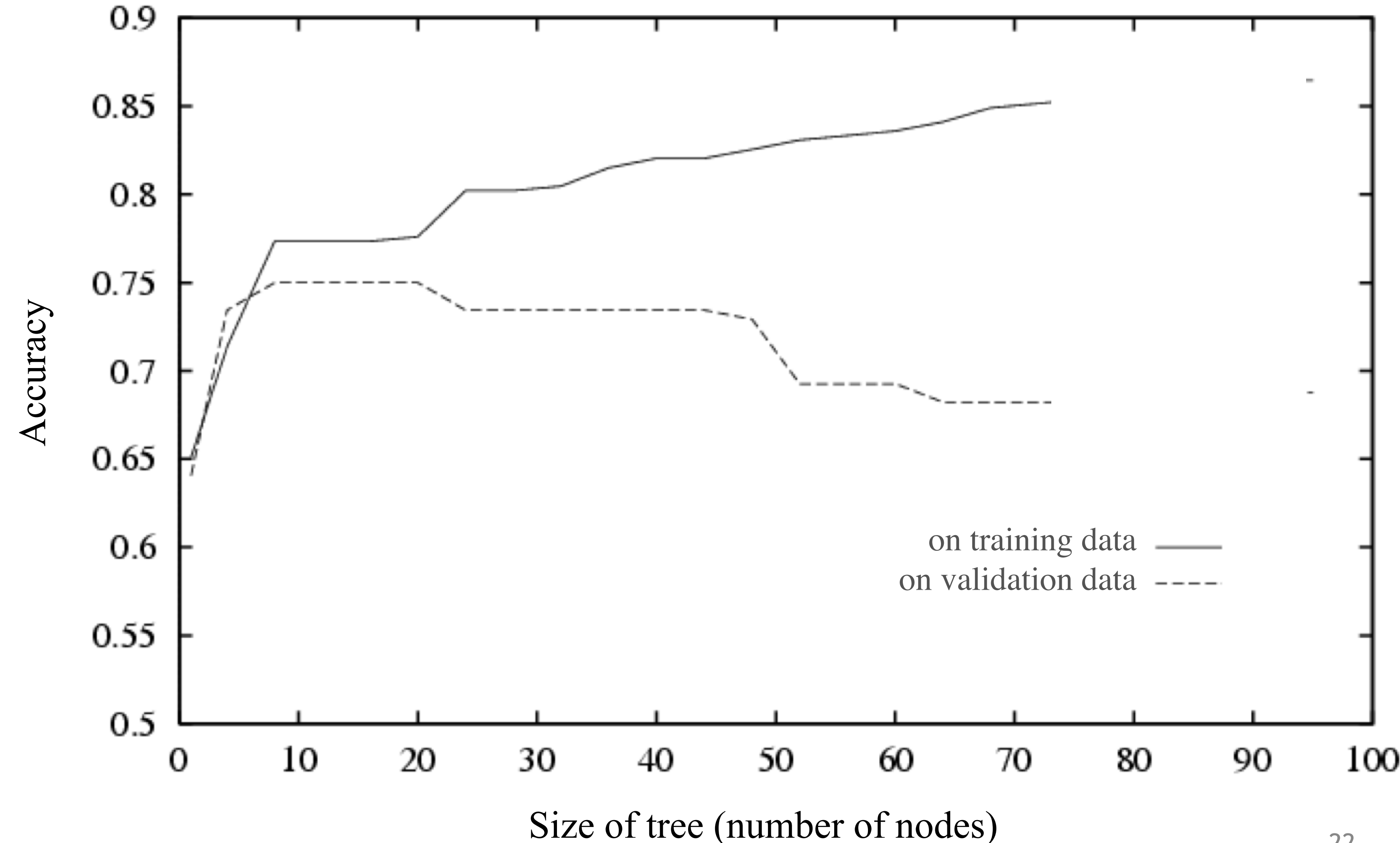


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

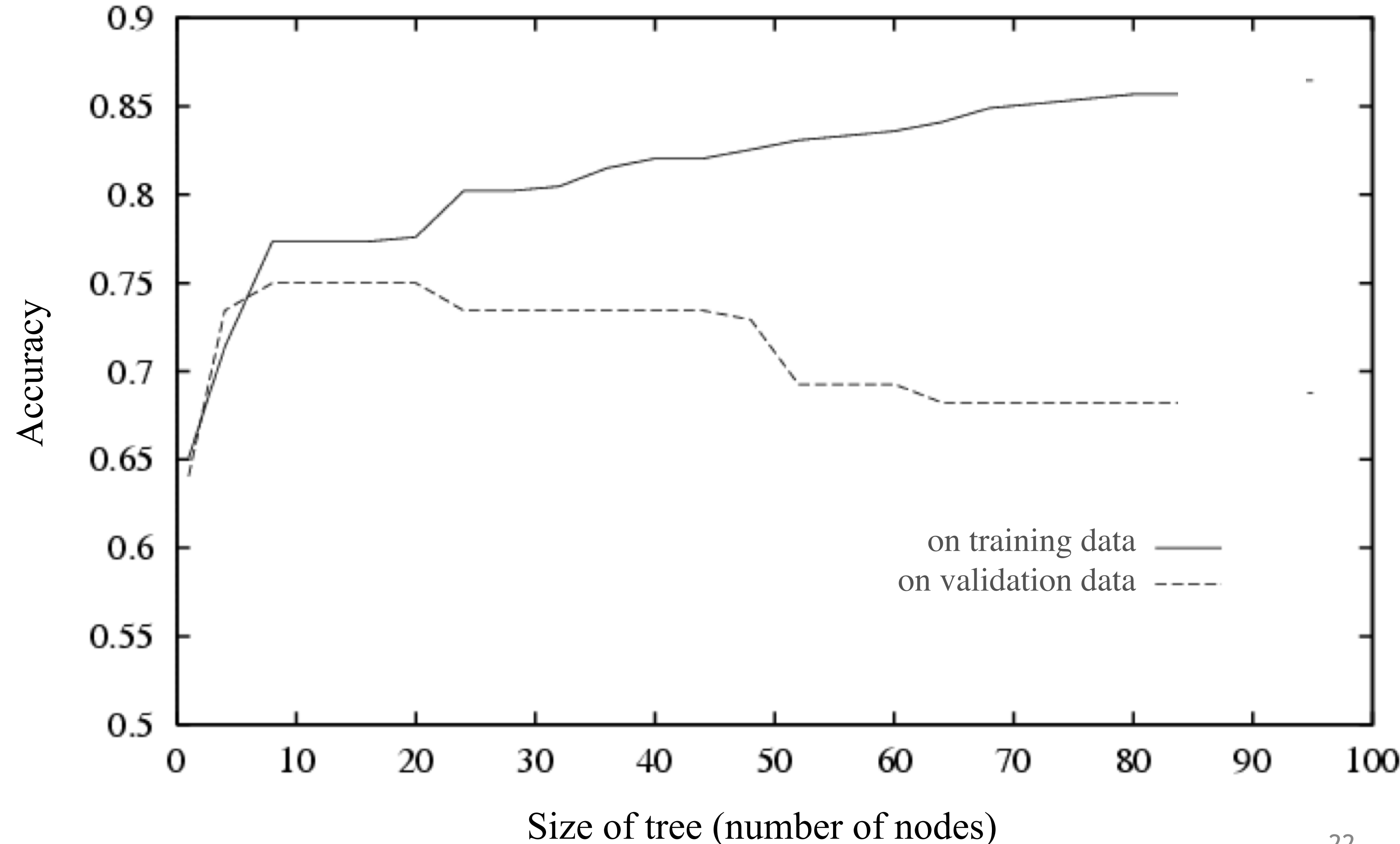


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

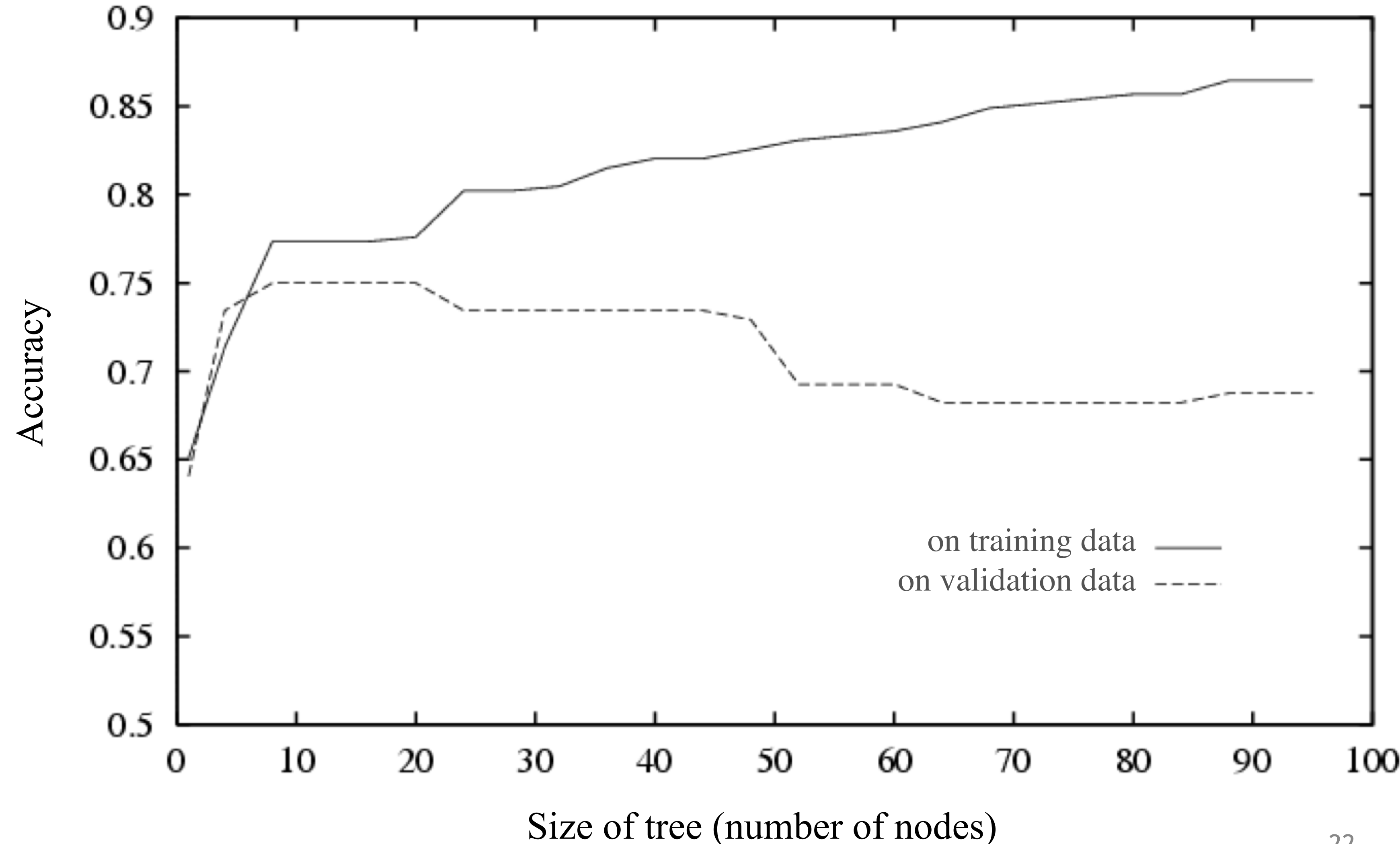


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

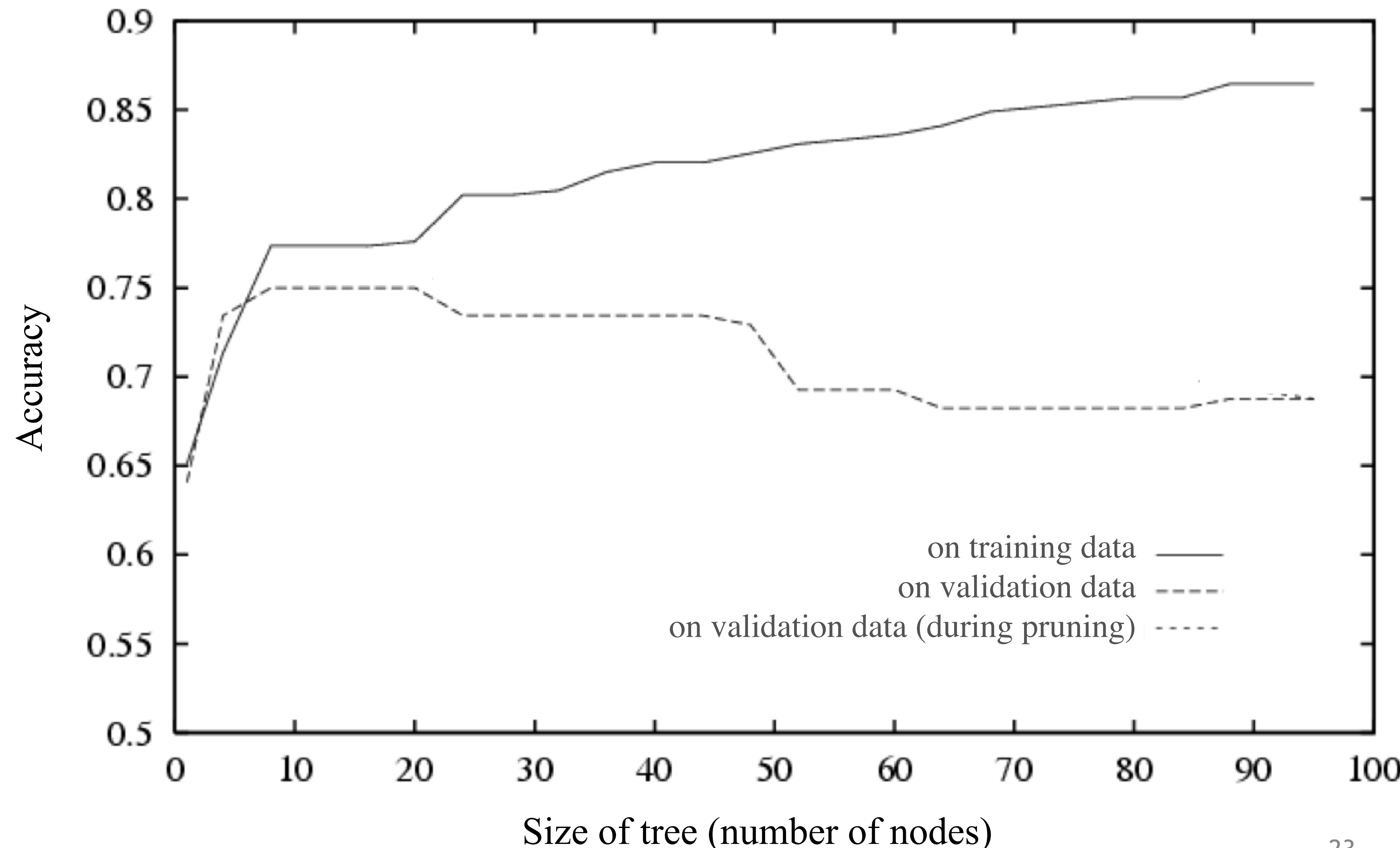


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

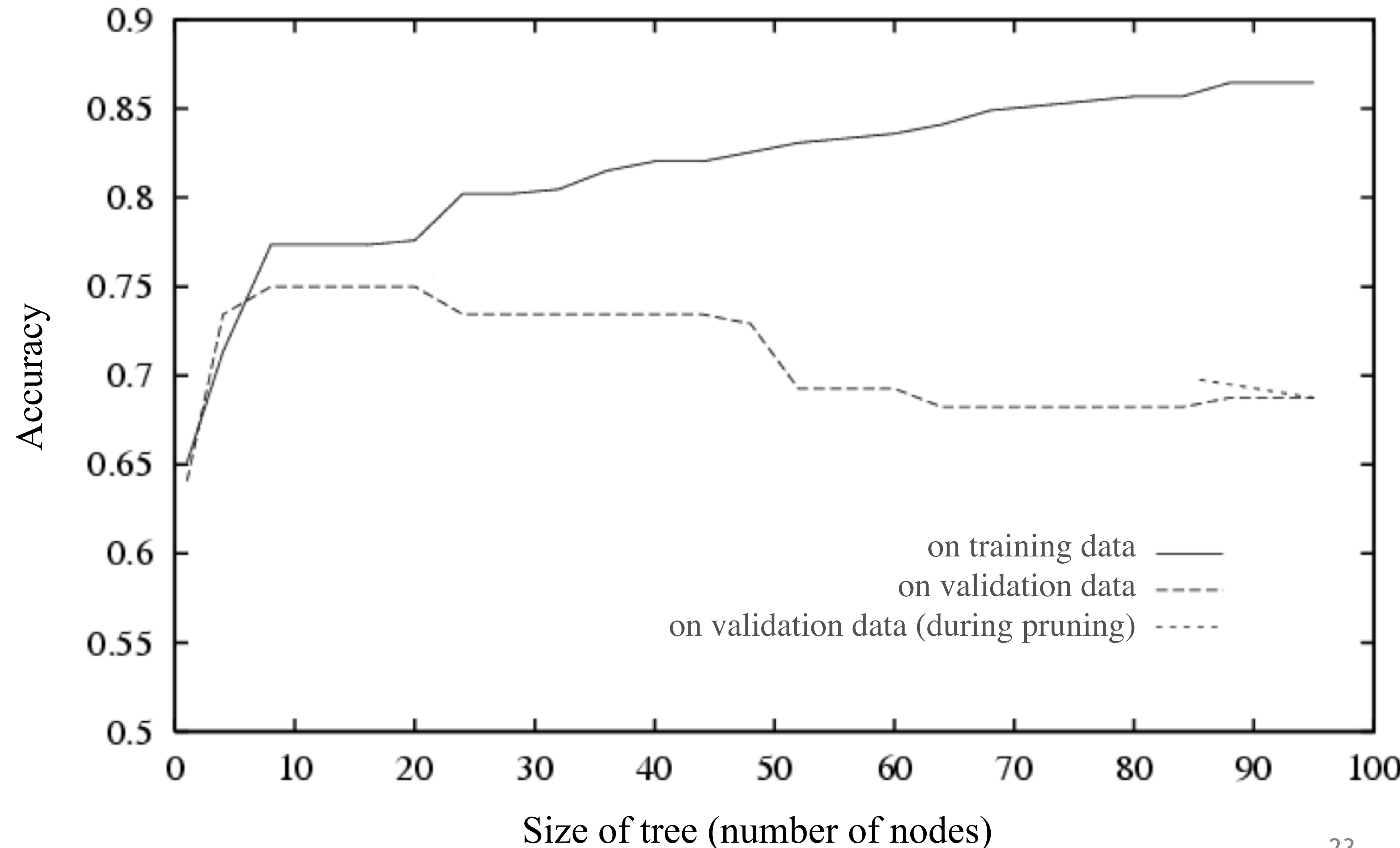


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

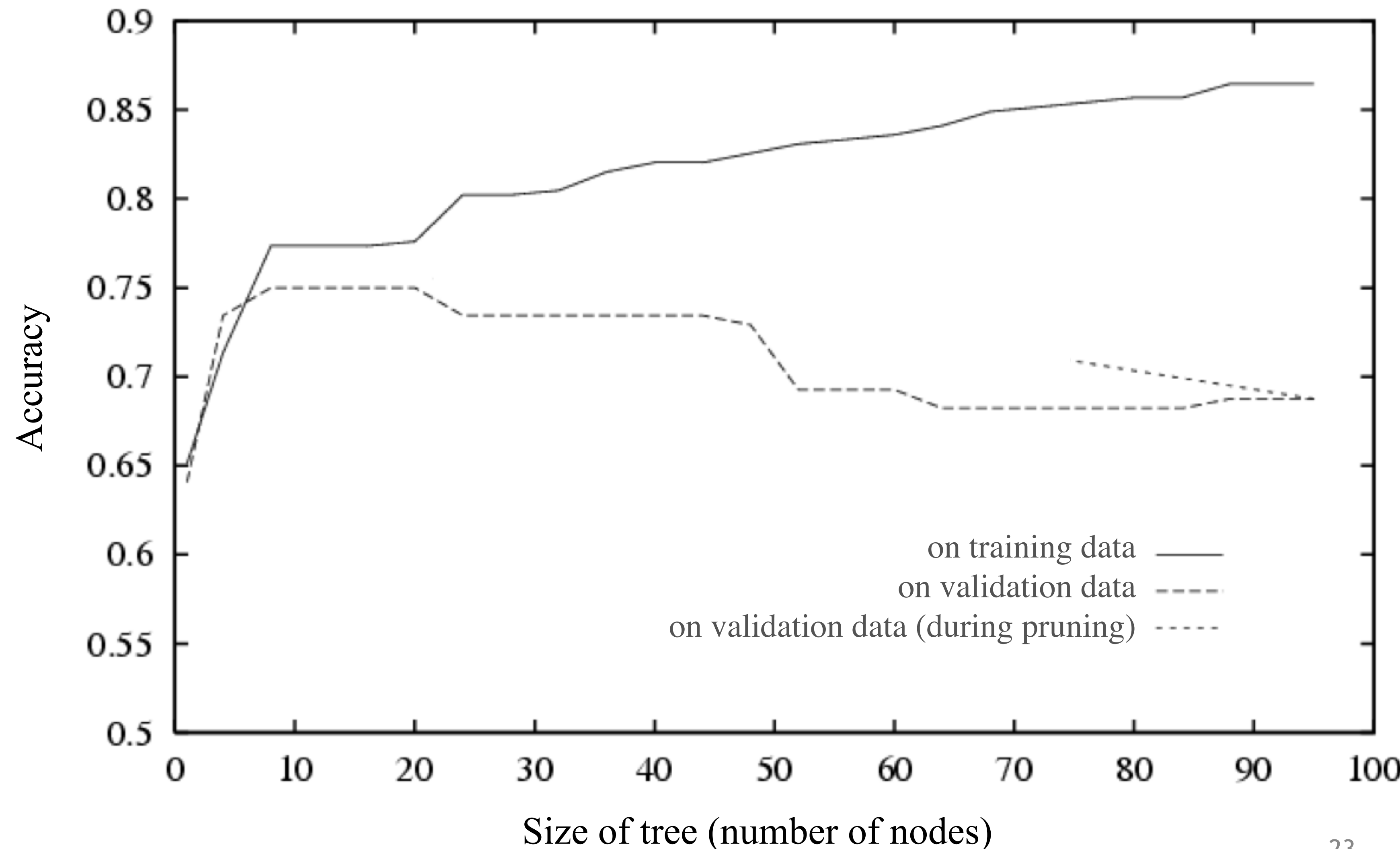


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

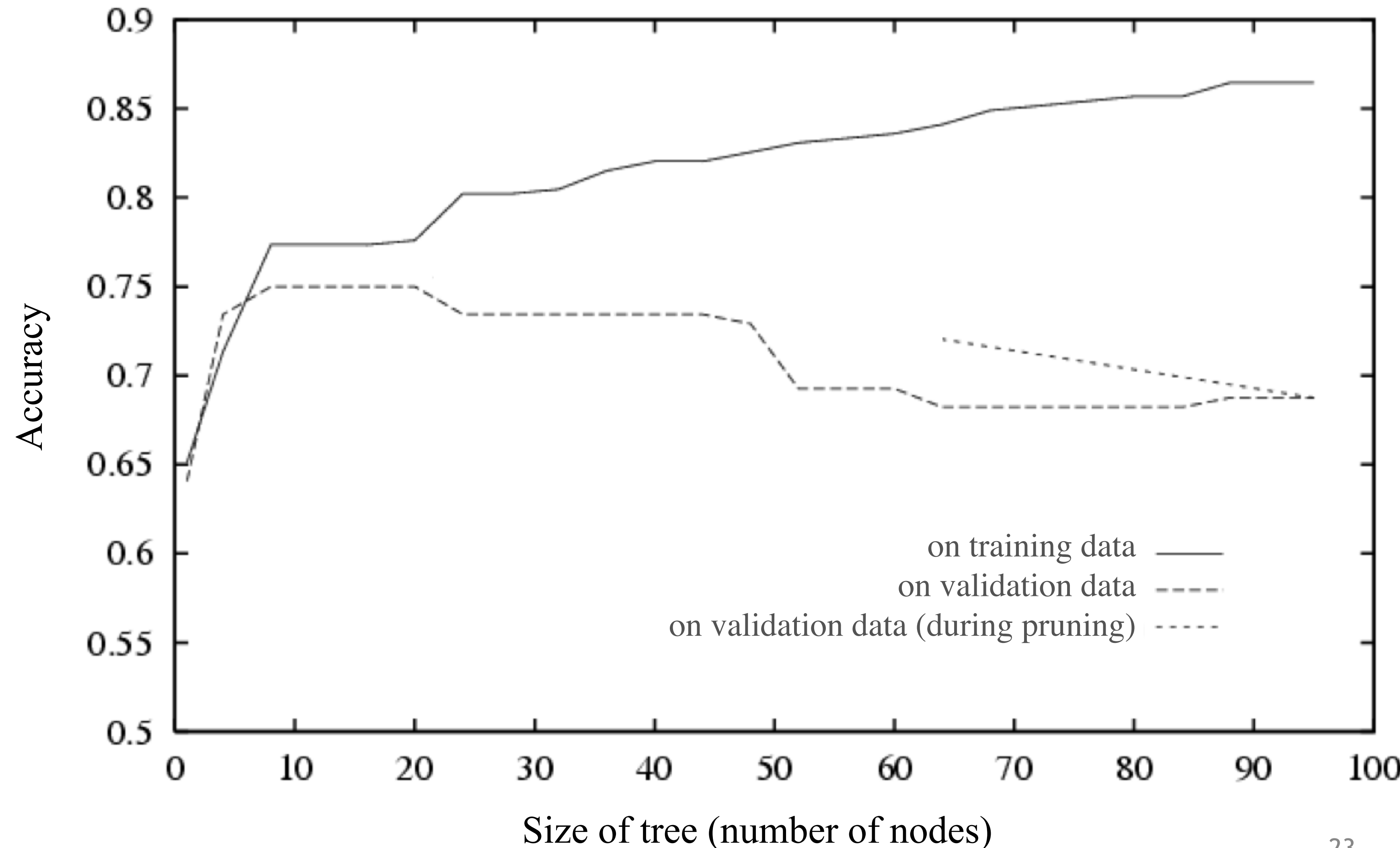


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

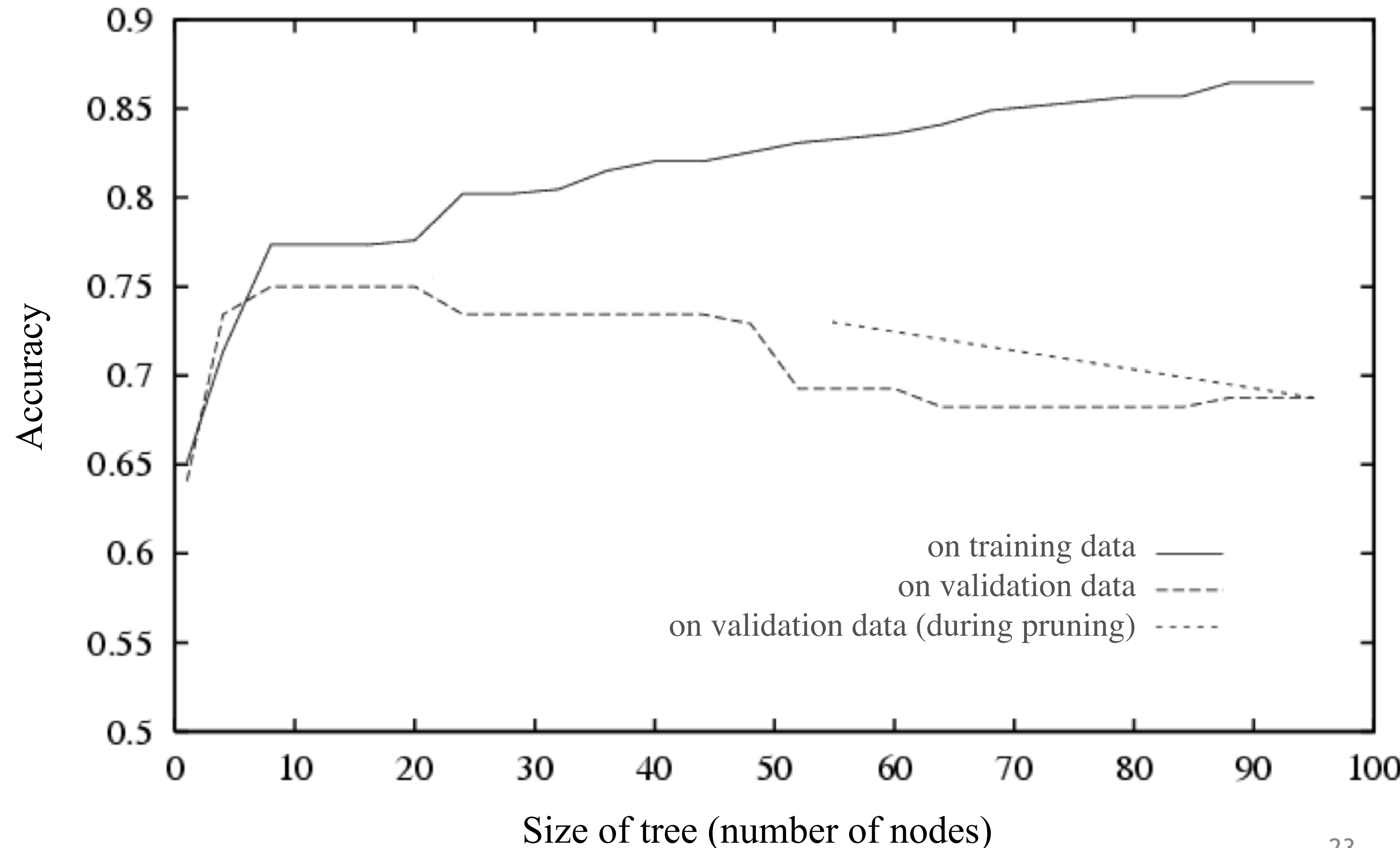


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

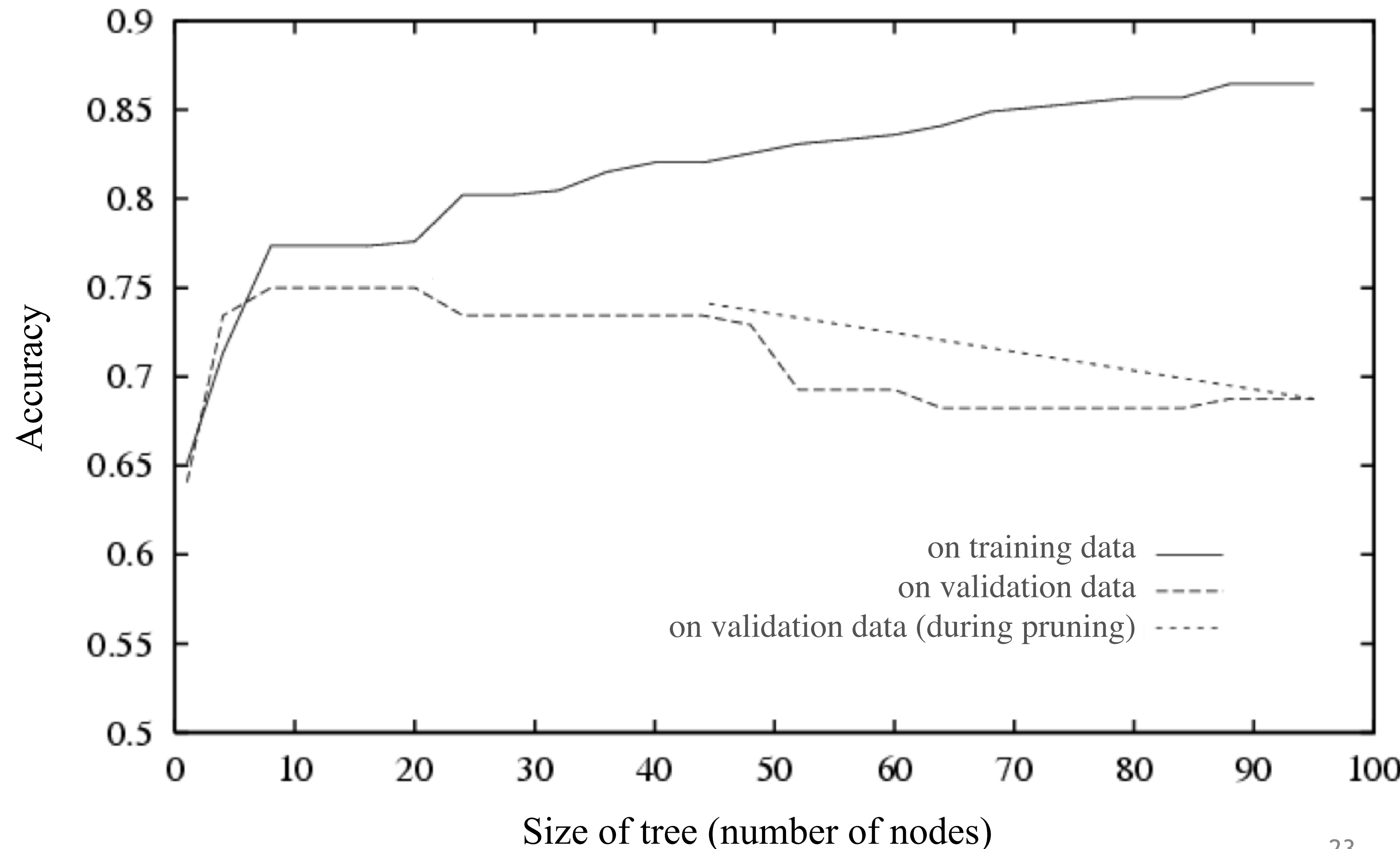


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

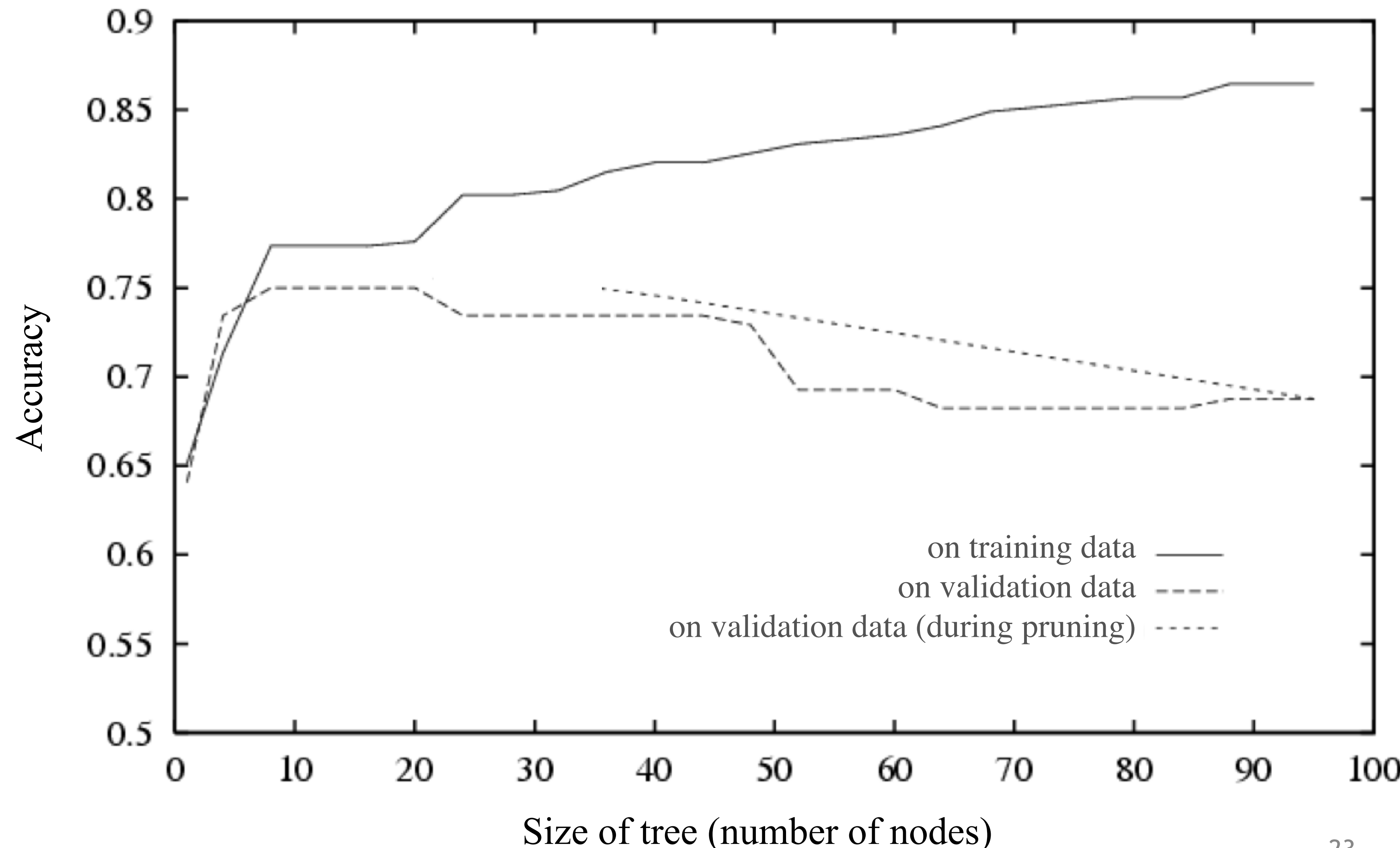


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

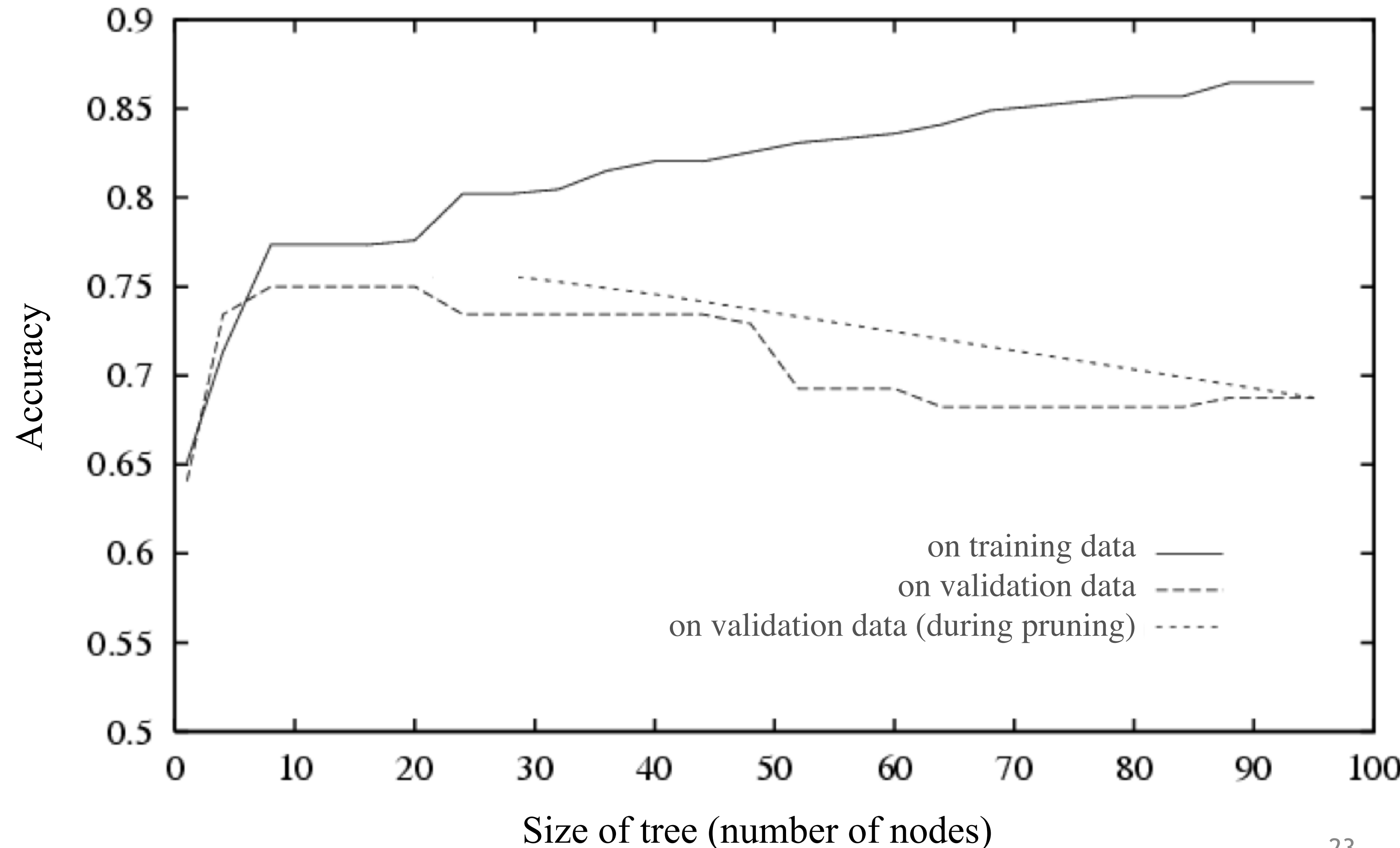


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
 - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
 - (Greedy) remove the split that most decreases the validation error rate
 - Stop if no split improves validation error, otherwise repeat

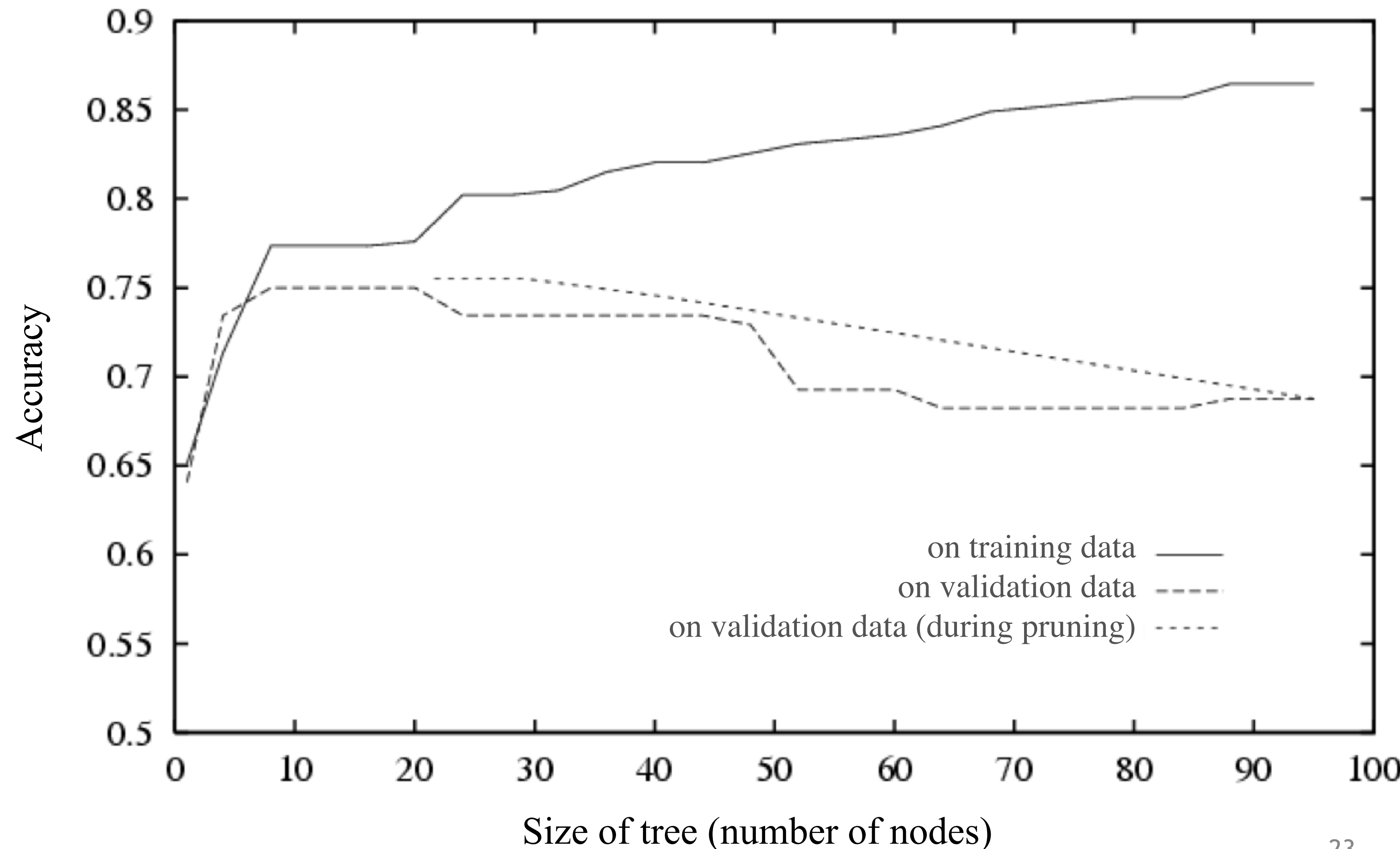
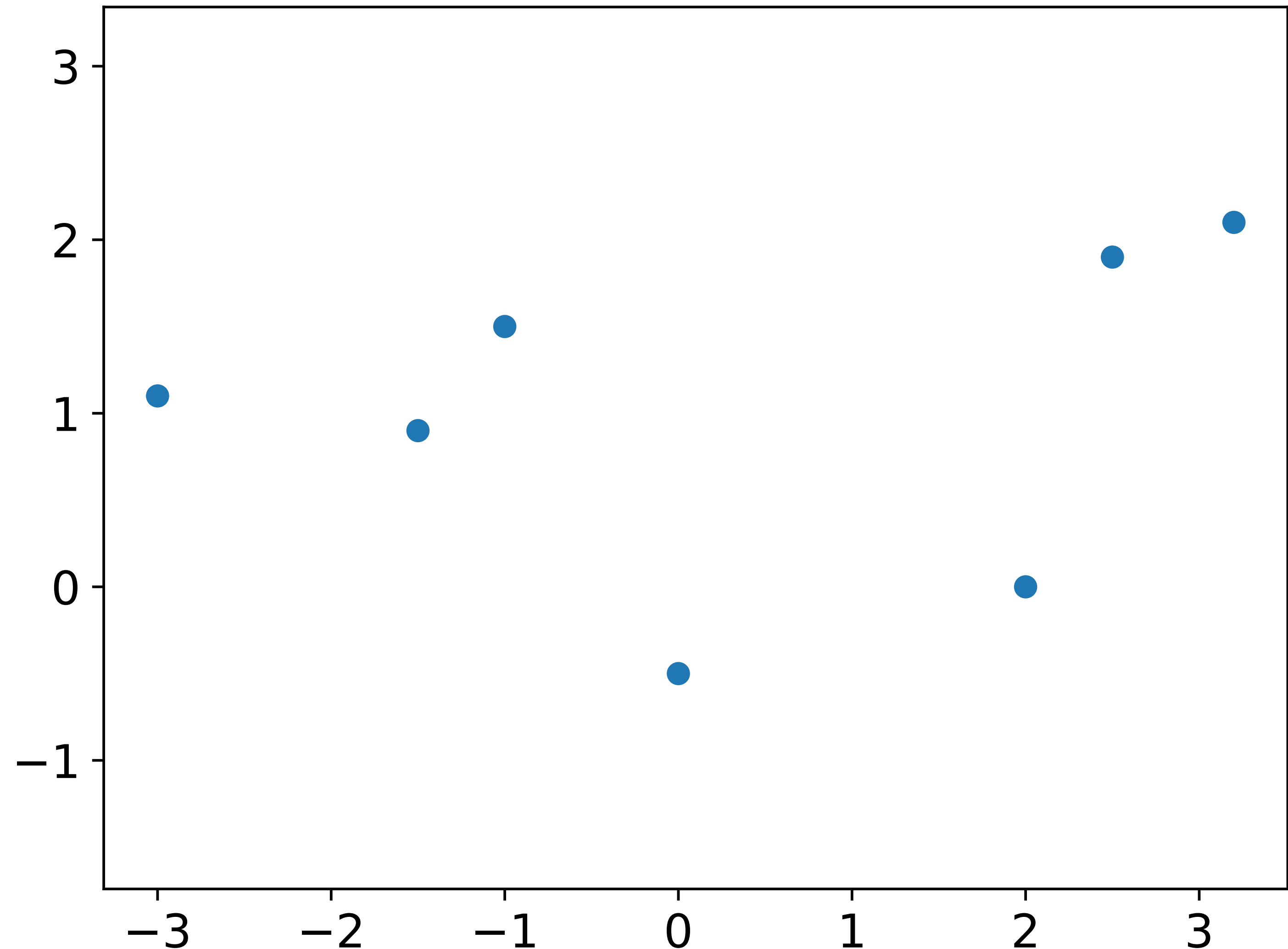


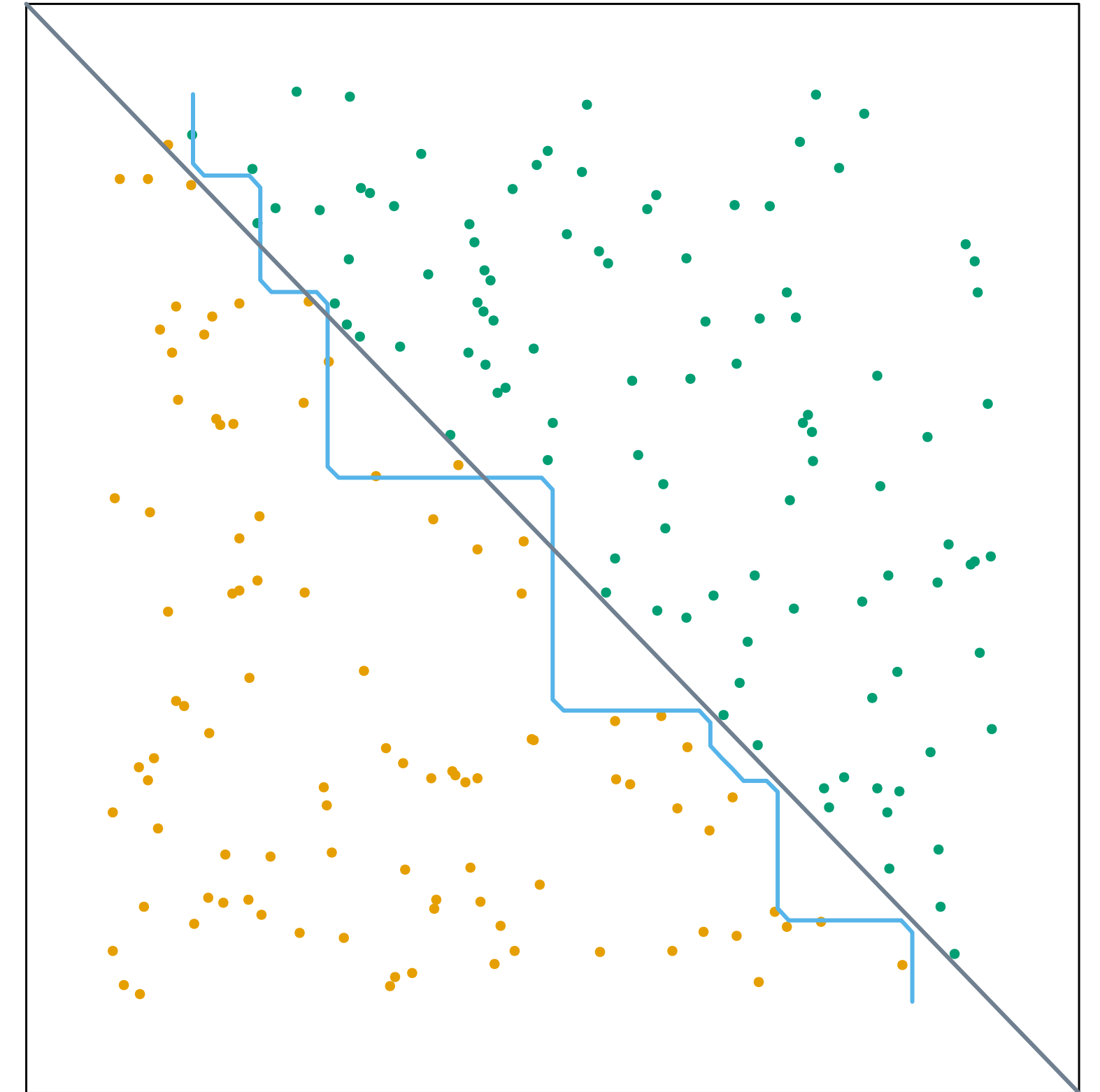
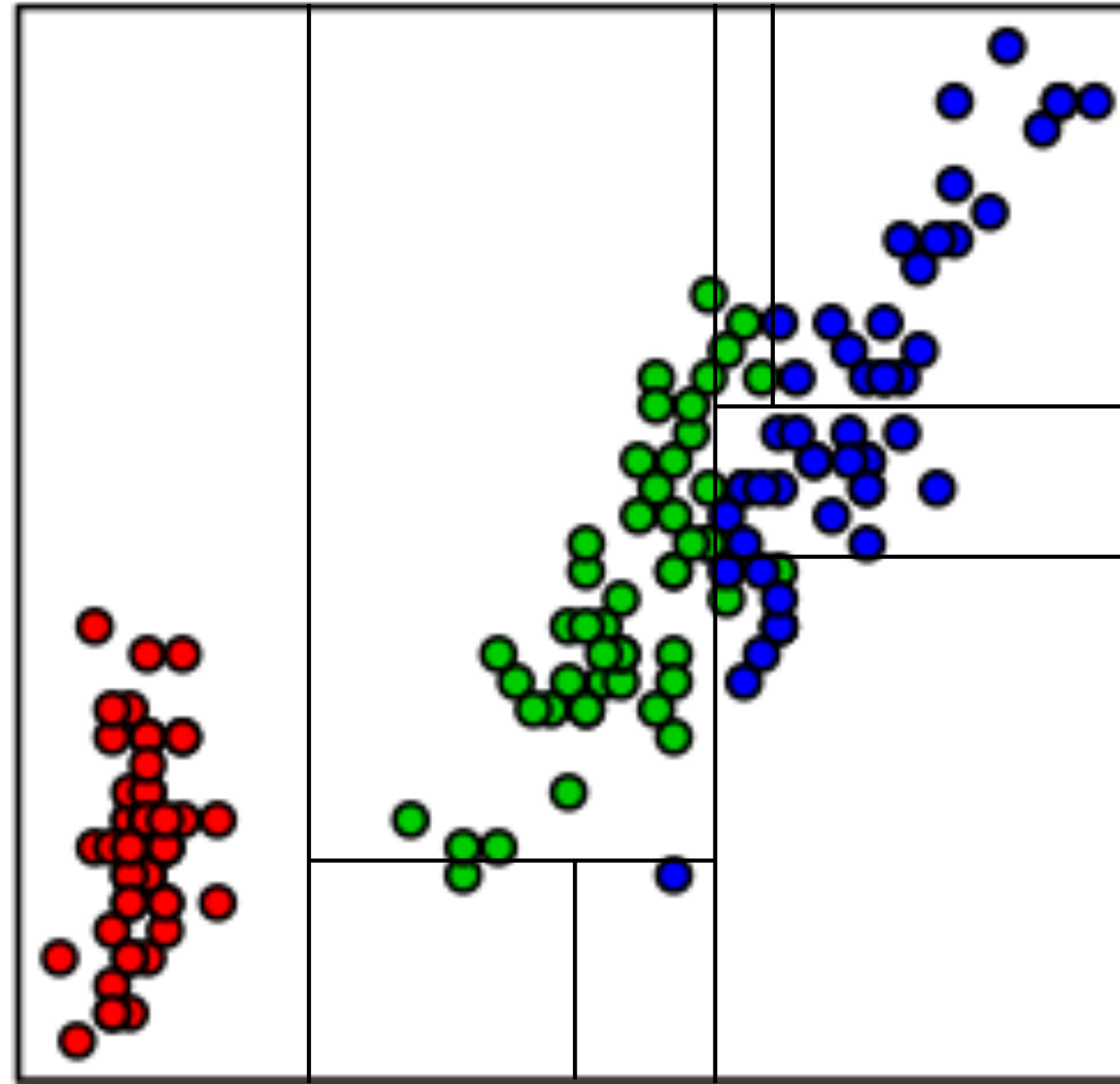
Figure from Tom Mitchell

How to handle real outputs: regression tree



- In each leaf: predict mean
- Splitting criterion: prediction error $\sum_{\text{examples } i} (y_i - \hat{y}_i)^2$

Inductive bias of decision trees



- Divide space into nested regions
- Piecewise constant in regions
- Axis-parallel splits
- Decisions expressed as simple rules: sepal width ≤ 5.2 & sepal length $\in [2.5, 4.0]$

Learning objectives— decision trees

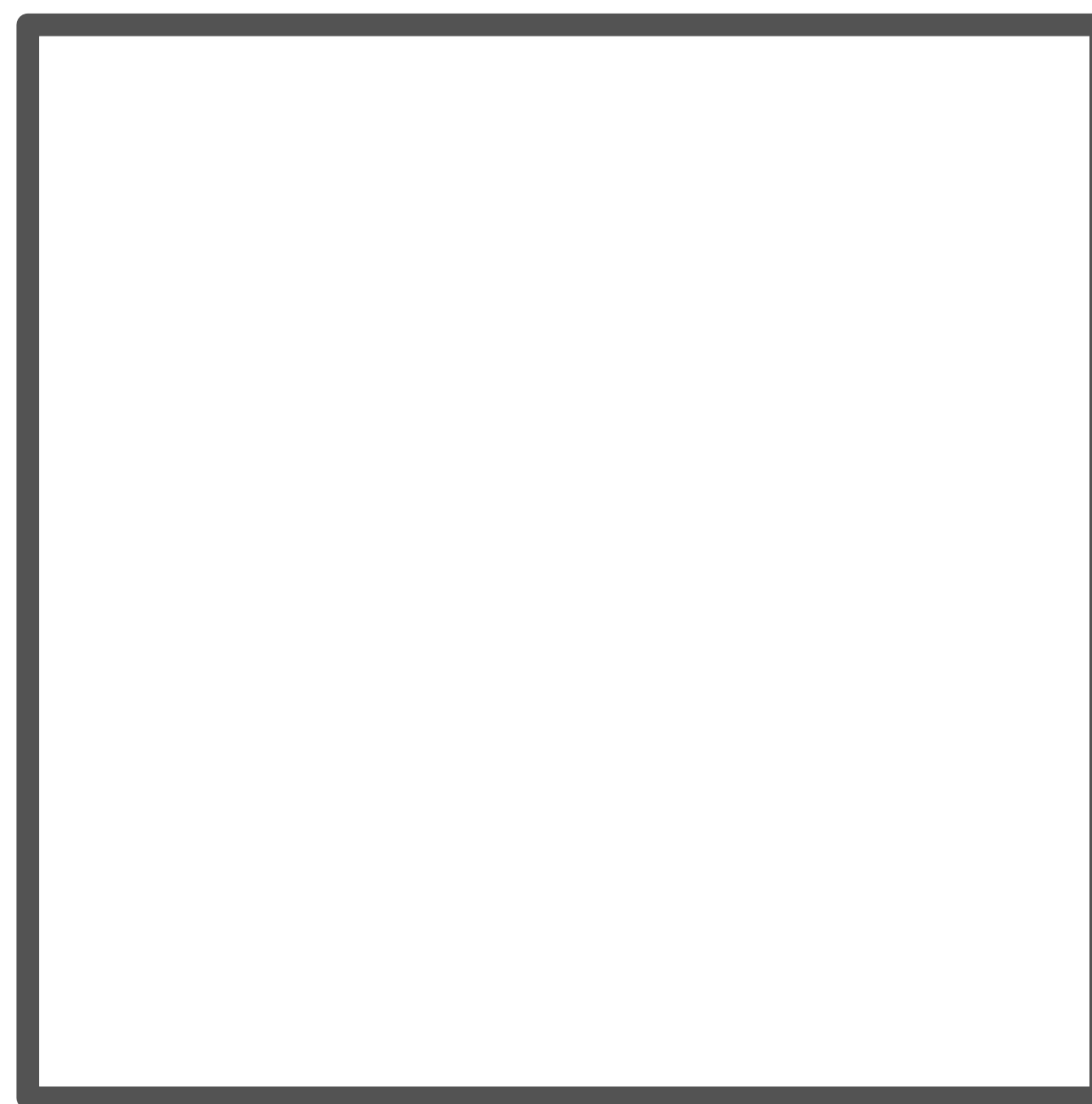
- What is a decision tree? How do we use it to predict?
- What is the inductive bias of a decision tree?
- How can we grow a decision tree?
- How do we use mutual information as a splitting criterion?
- How can we prune a decision tree? Why would we?
- What is a validation set, how does it differ from a test set, and why do we need one?

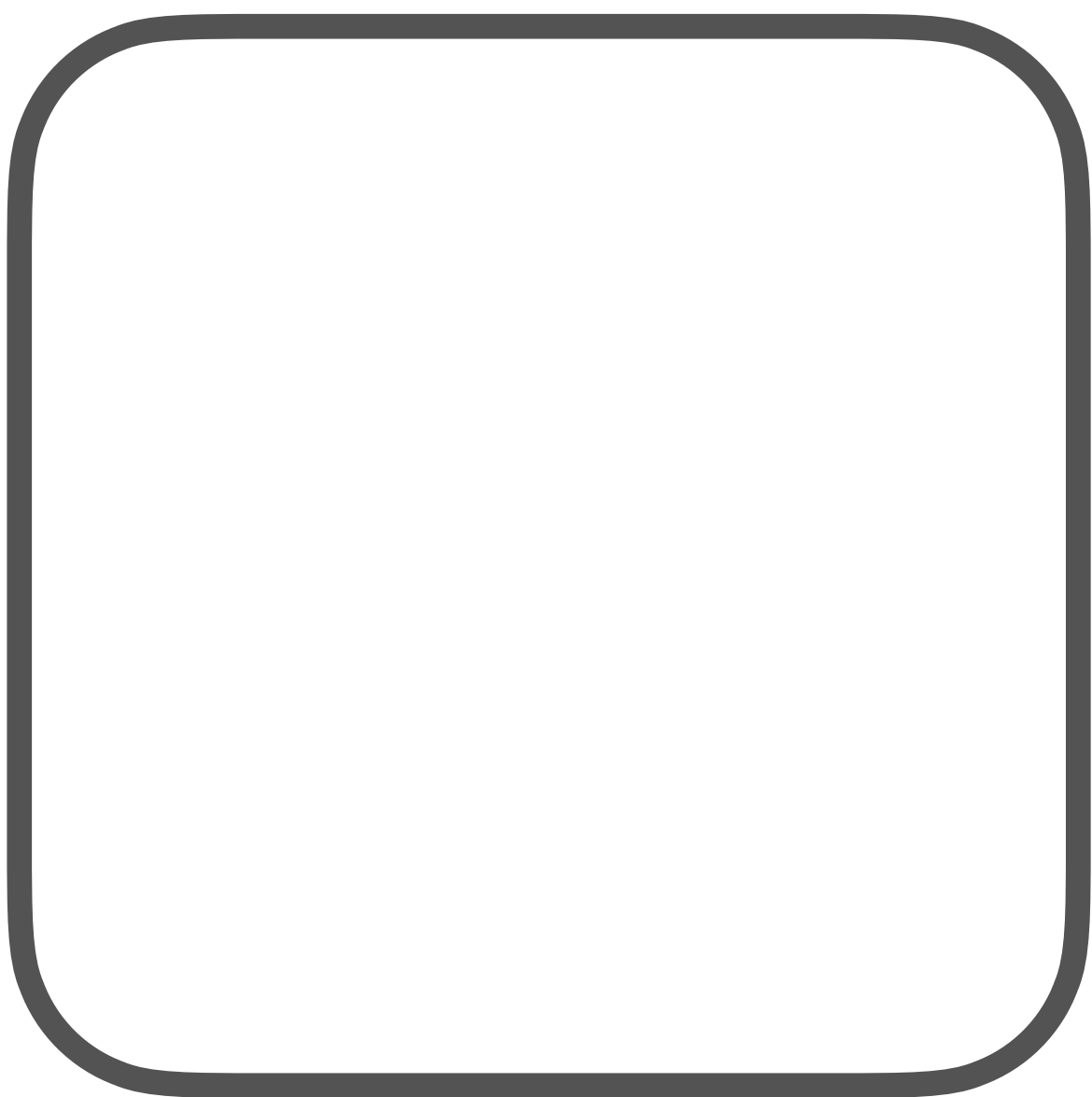
Try it at home!

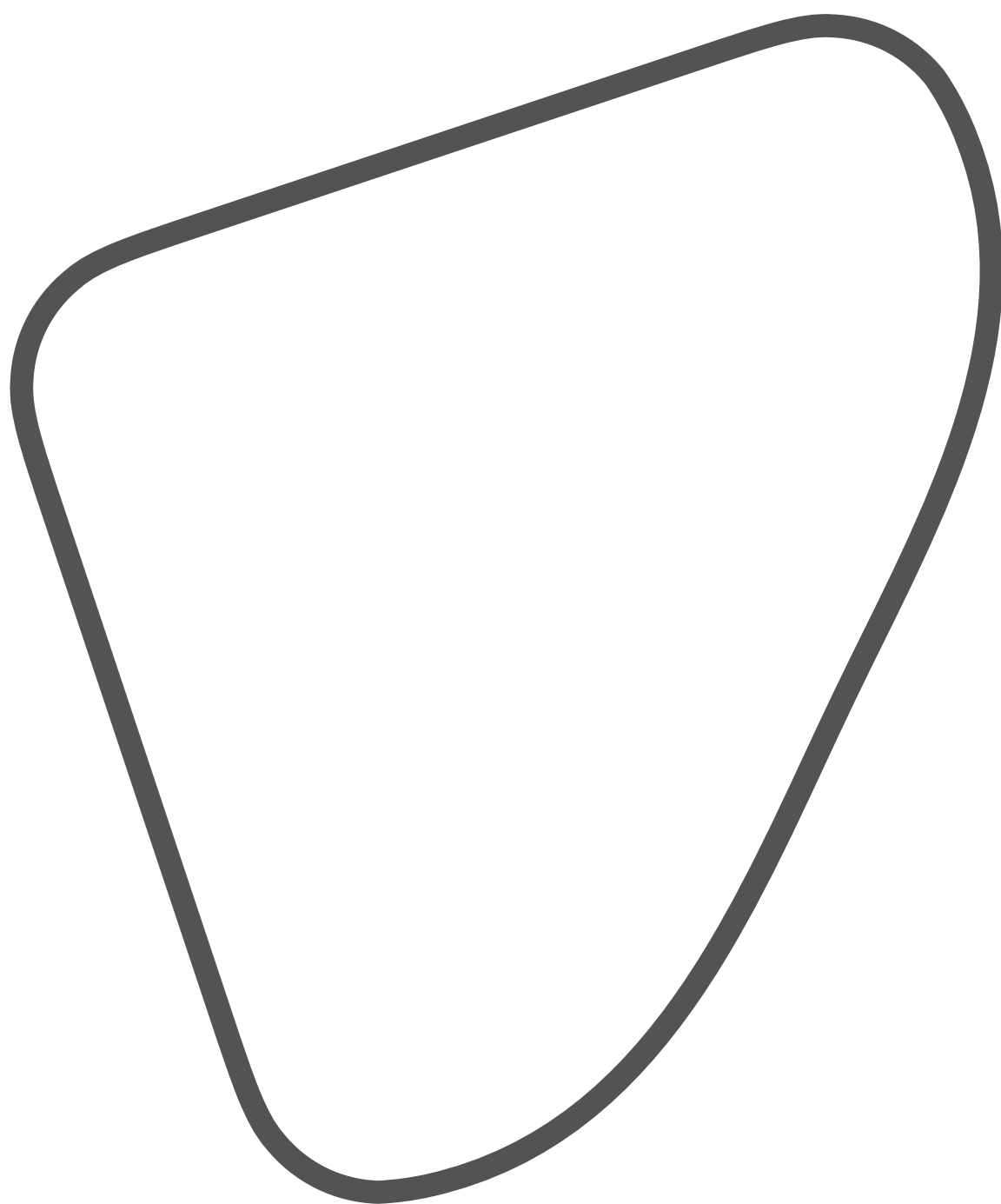


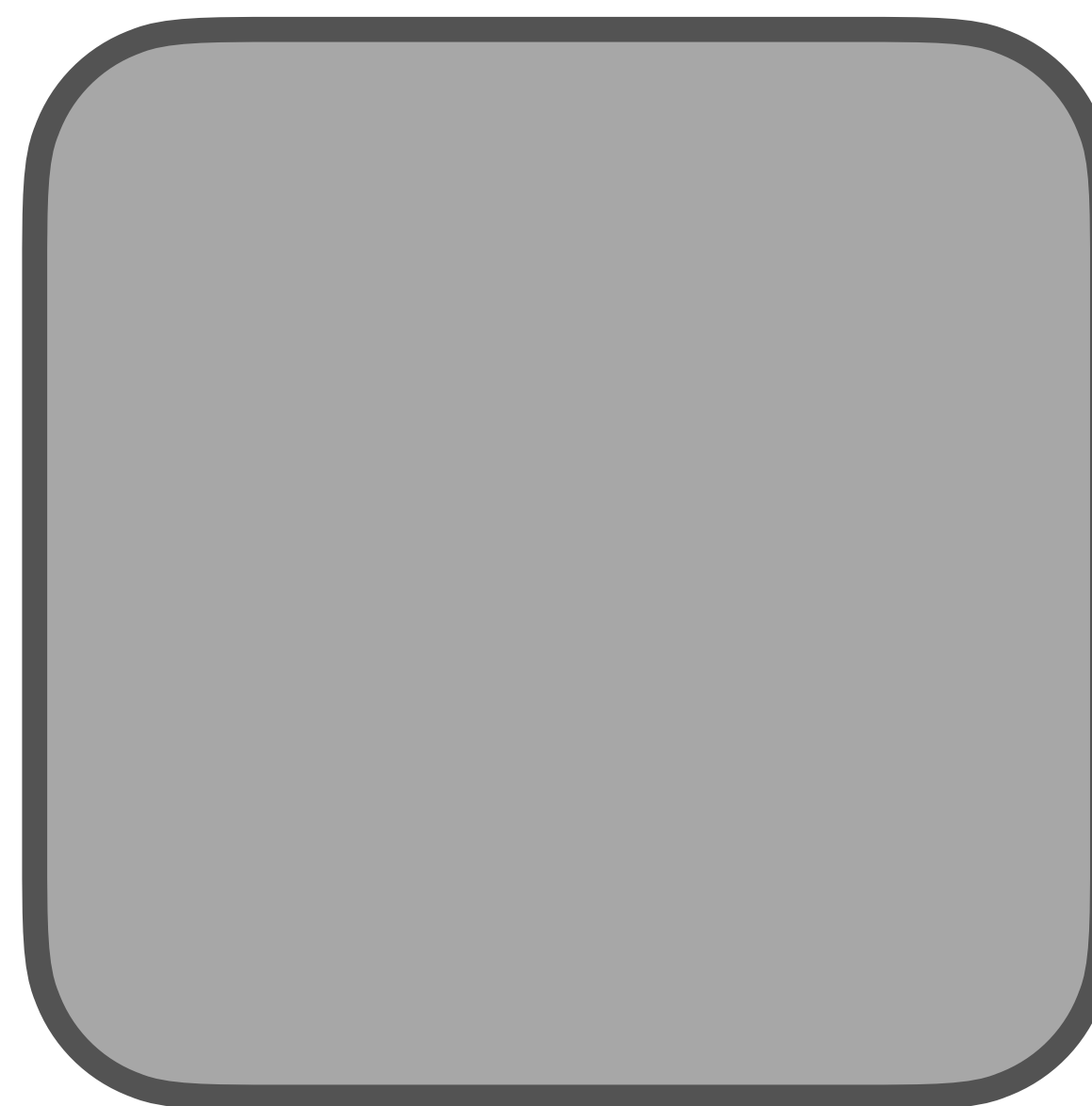
Result

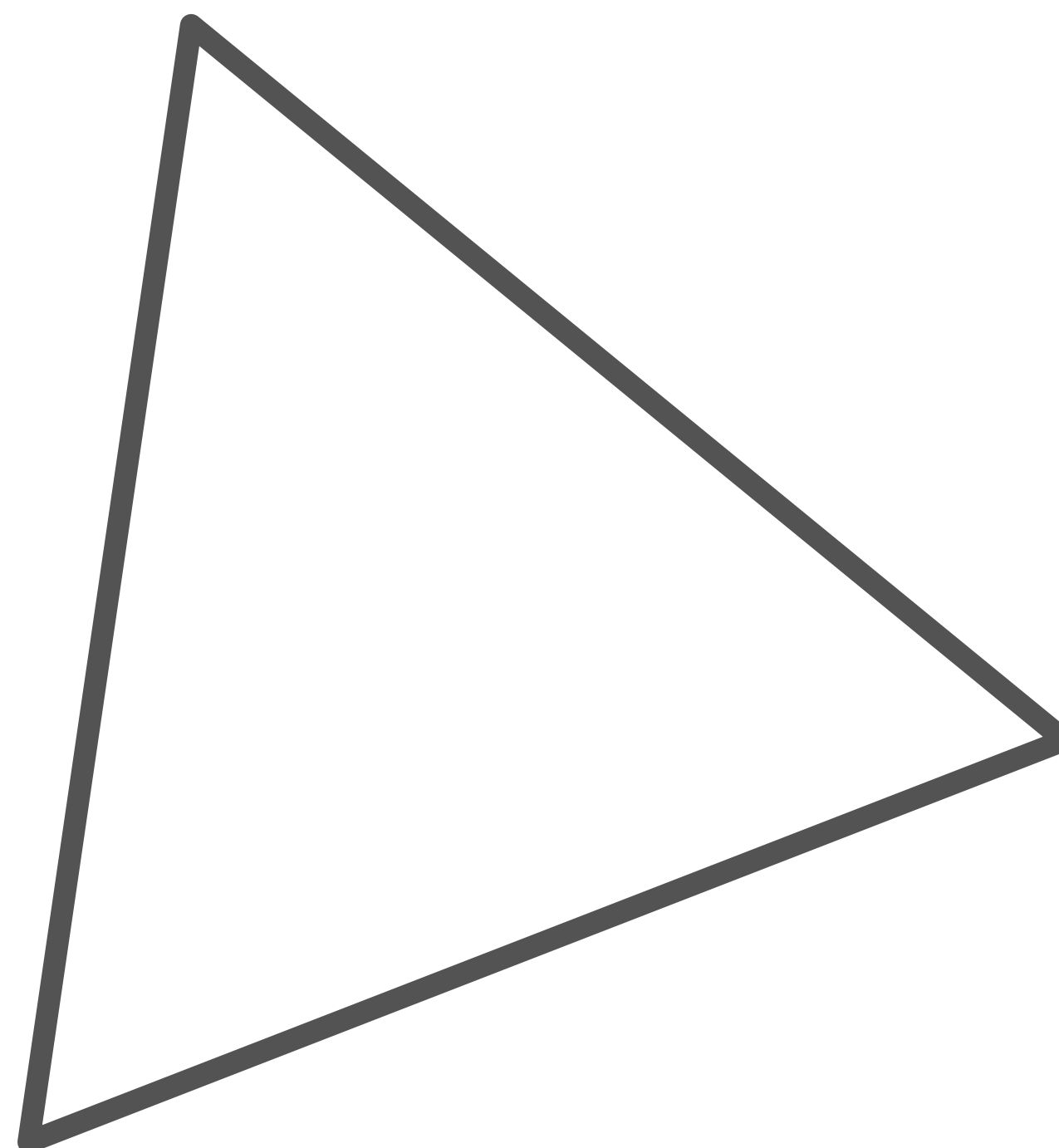
- Features:
- Learned tree:

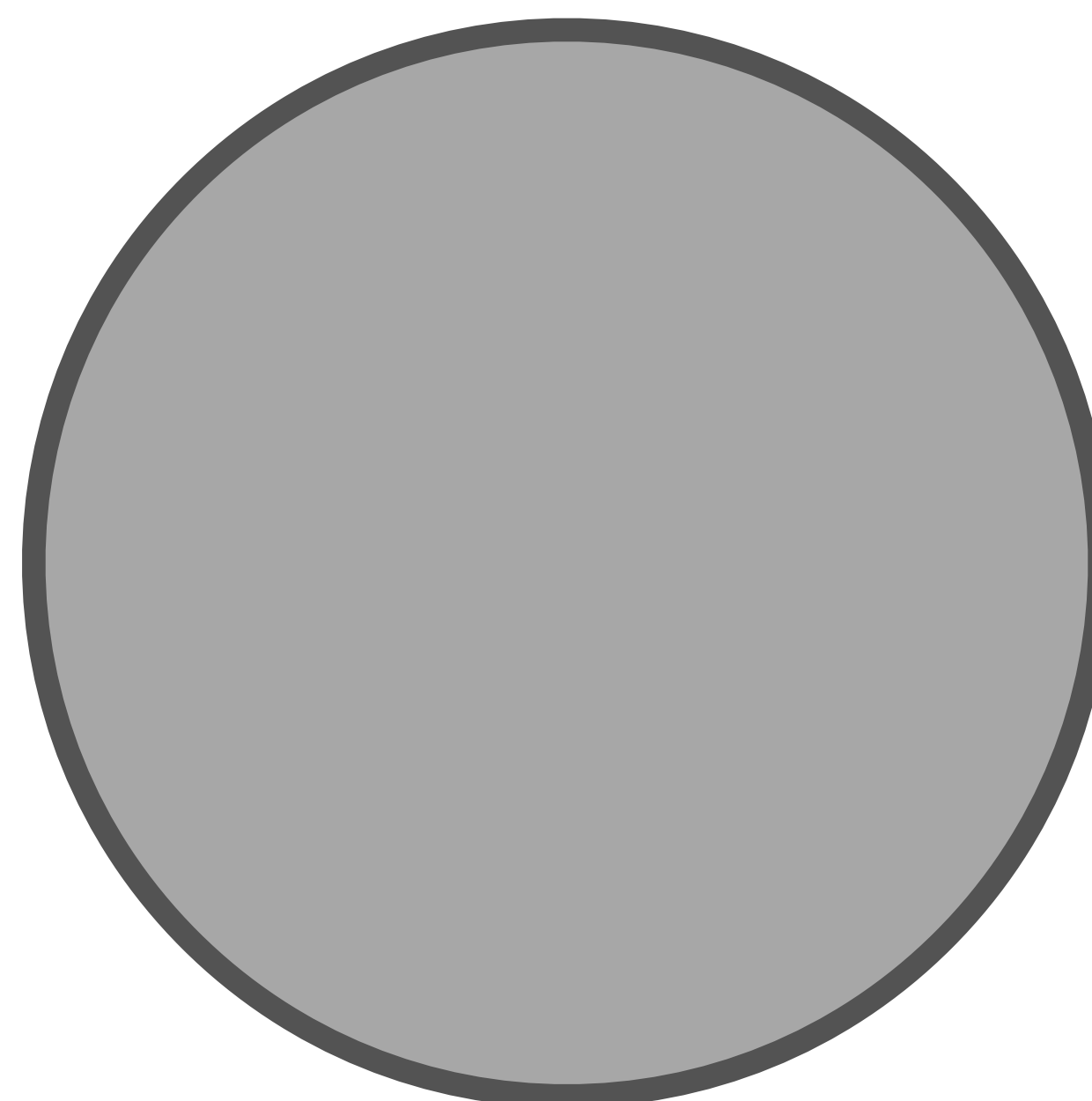


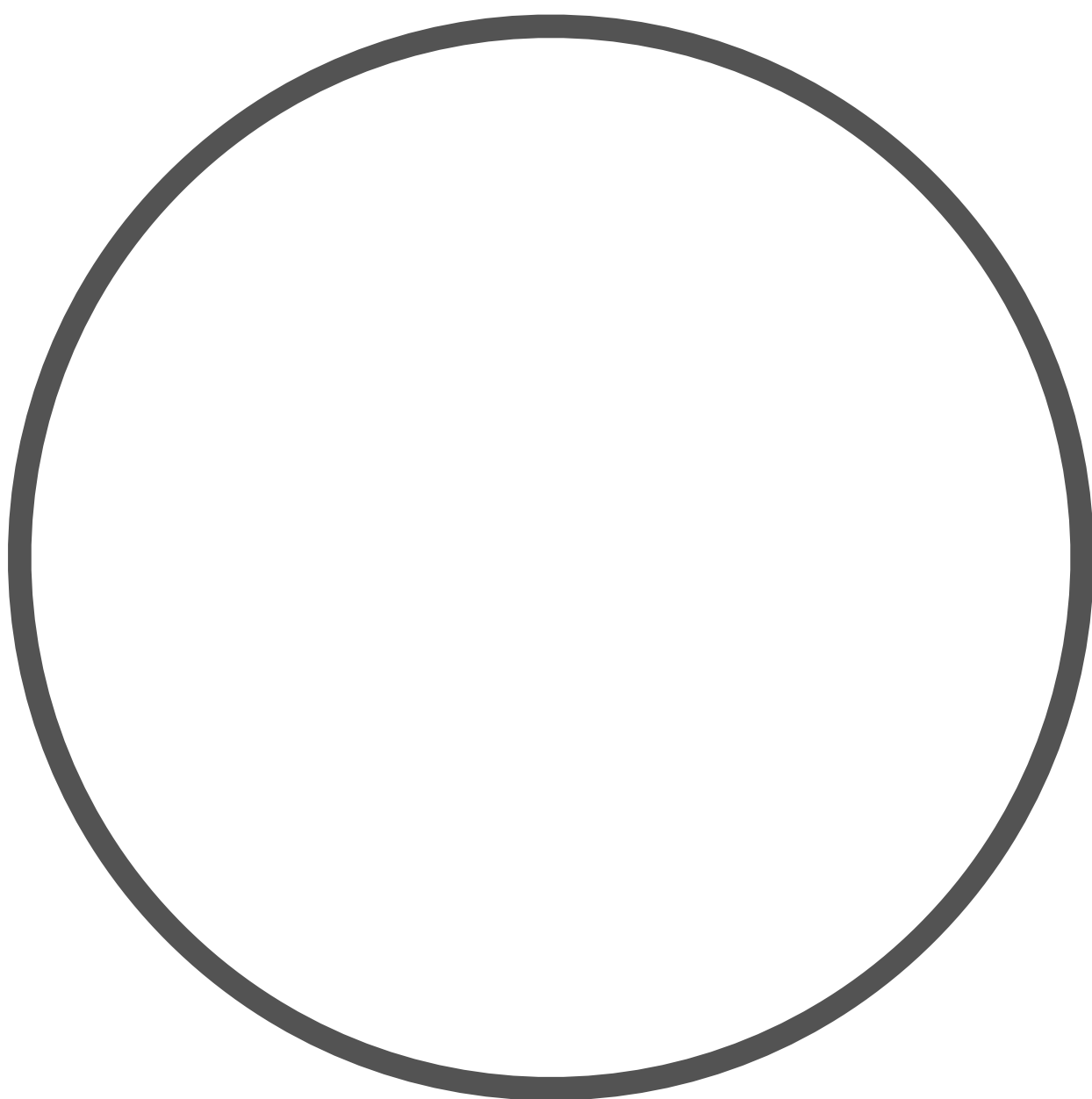












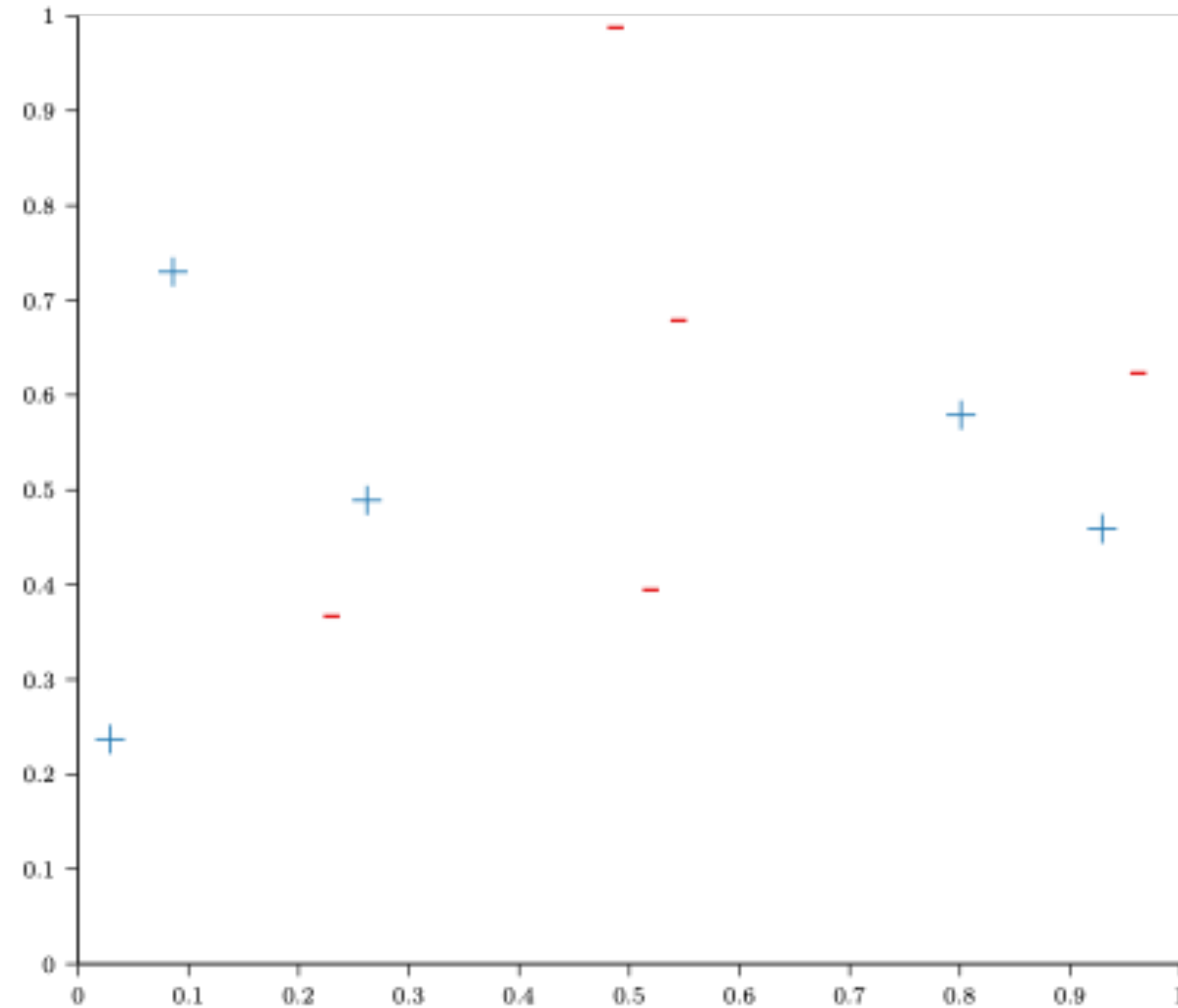
Neighbors

- Predictions for nearby points tend to be similar
- Motivates ***nearest-neighbor*** methods: predict based on nearby training points
 - ▶ often not SOTA accuracy
 - ▶ but highly interpretable
- NN methods form the basis for lots of AI systems
 - ▶ e.g., search or recommendation systems
 - ▶ e.g., related to attention in transformers: store some keys, retrieve best match(es) for a given query
 - ▶ useful if we don't just need to predict a label, but return an item (a webpage, a product to buy)

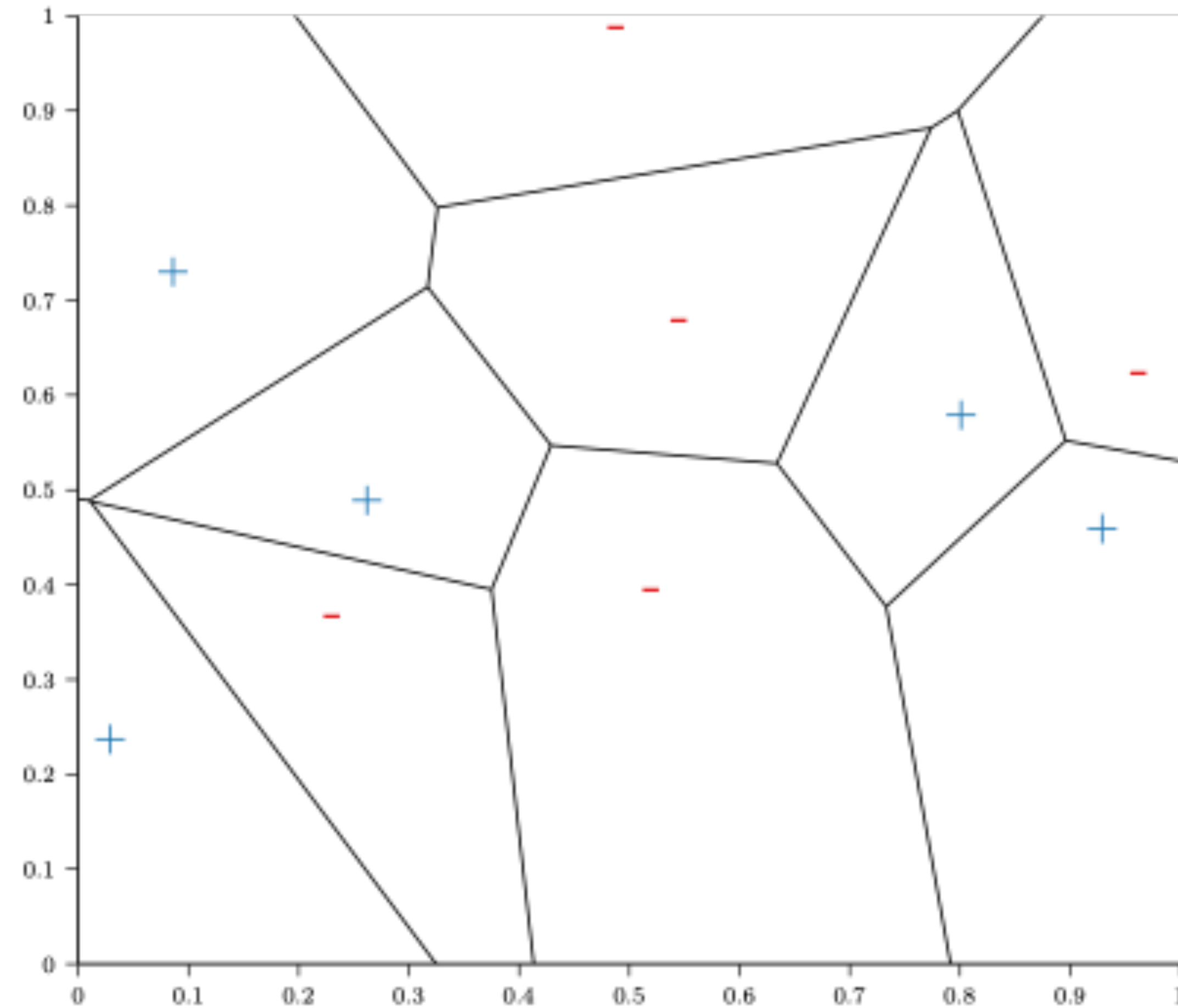
k-nearest-neighbors

- Train: just store the dataset!
 - ▶ no parameters to fit: k -NN is a ***nonparametric*** method
- Test: look up the k nearest training points
 - ▶ for classification, predict the most common label among neighbors (like *majority* classifier but only on neighbors)
 - ▶ for regression, predict the average label
- 1-NN is just called ***nearest neighbor***

Nearest Neighbor: Example

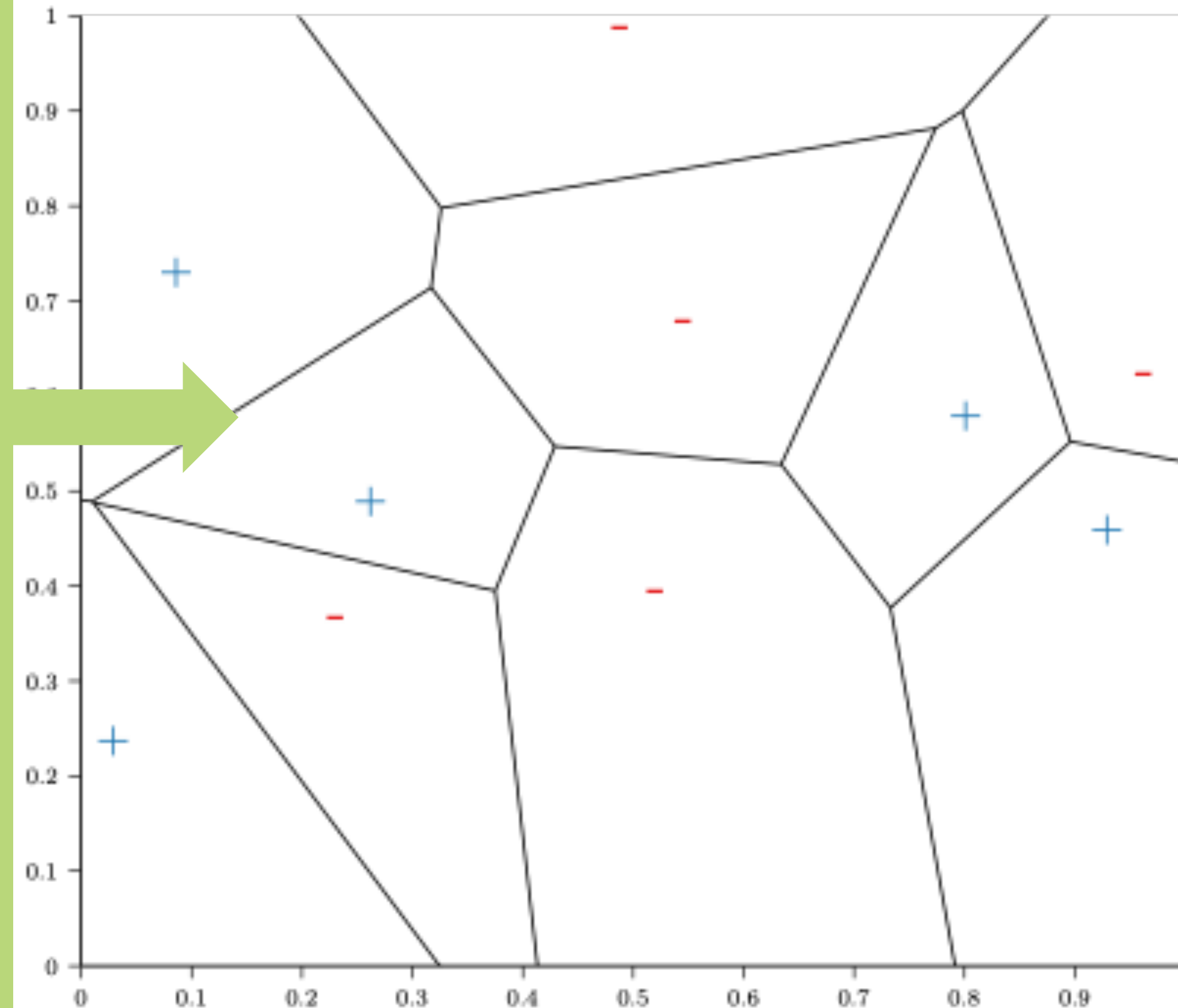


Nearest Neighbor: Example

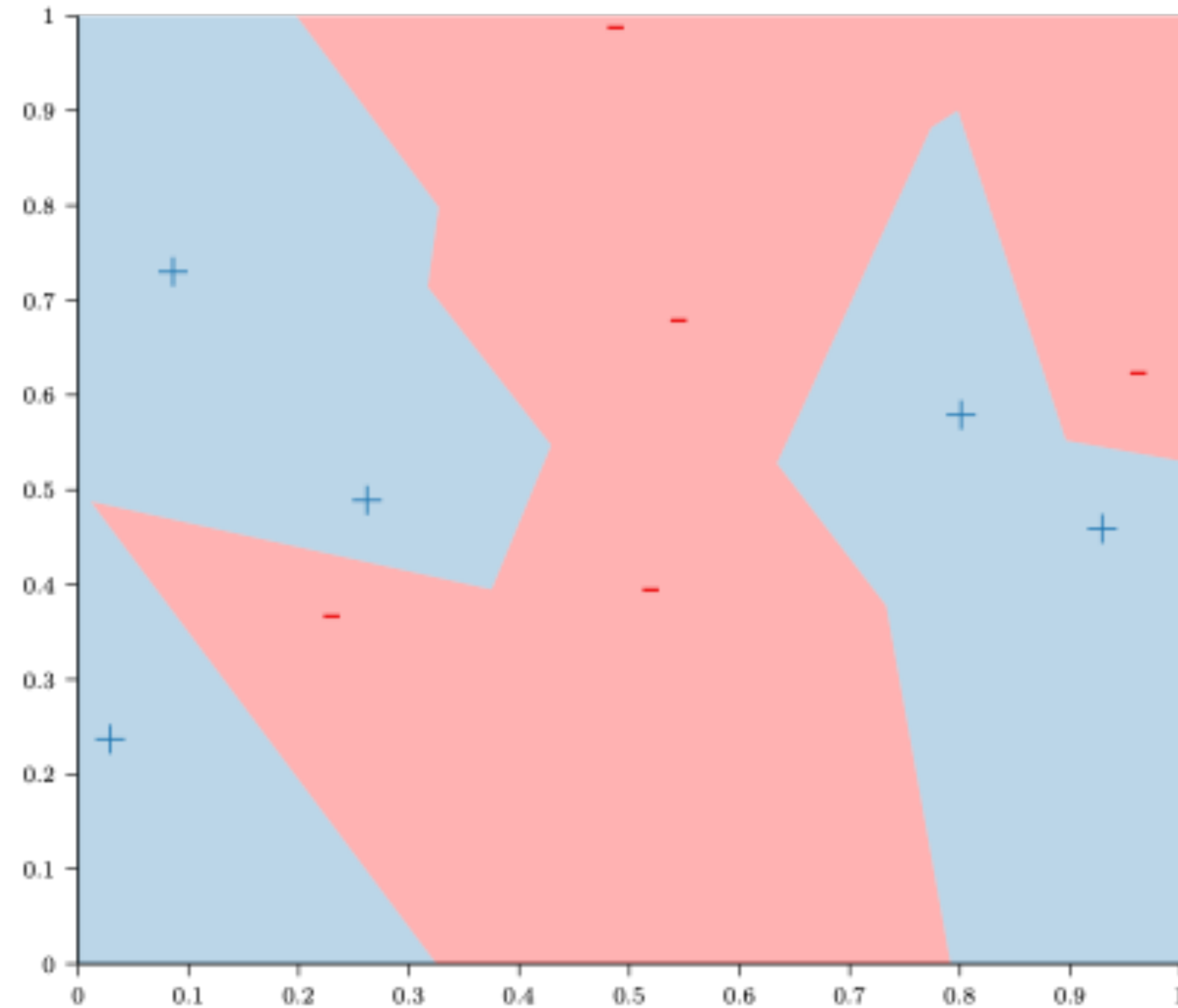


Nearest Neighbor: Example

- This is a **Voronoi diagram**
- Each **cell** contains one of our training examples
- **All points within a cell** are **closer** to that example than to any other example
- **Points on the Voronoi line segments** are **equidistant** to one or more training examples



Nearest Neighbor: Example



Distance metrics

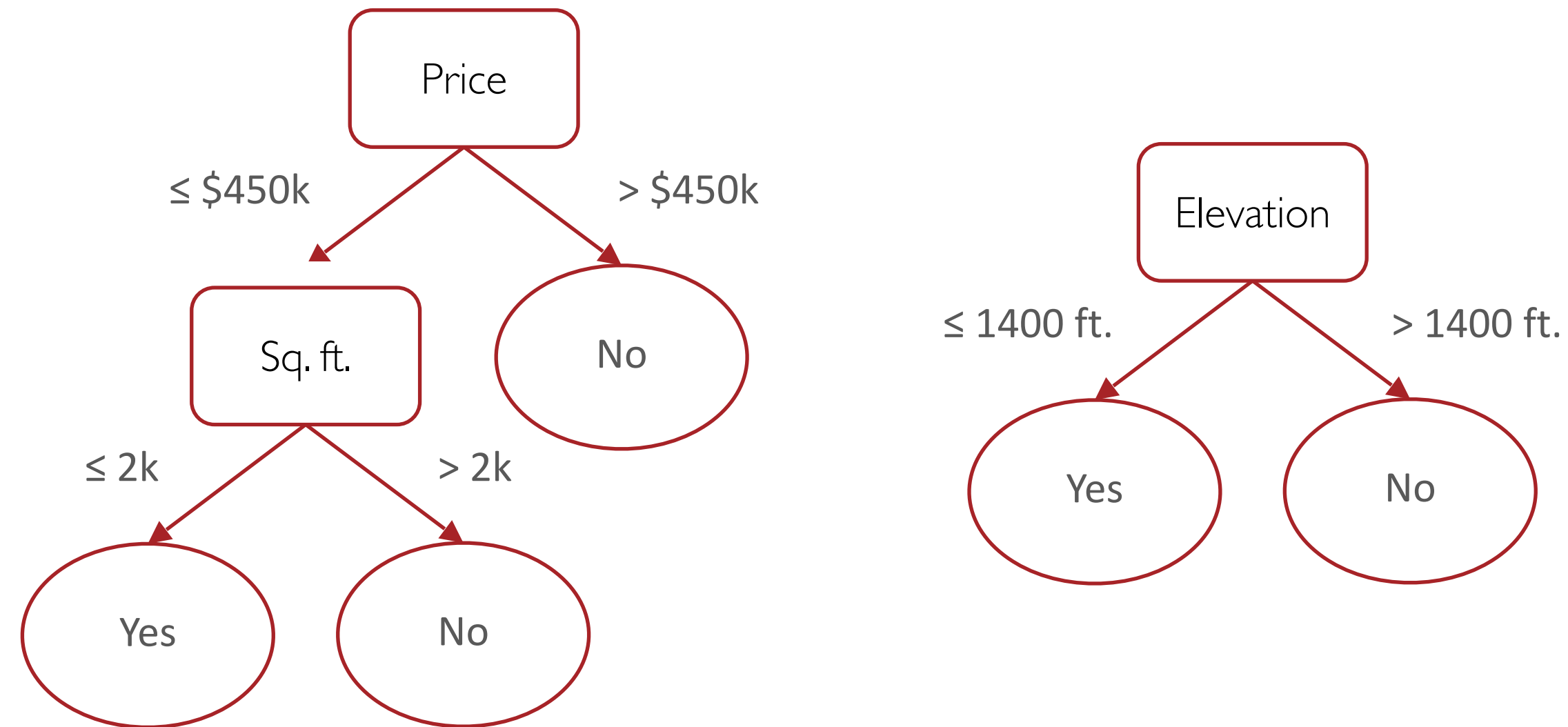
- k -NN has two hyperparameters: number of neighbors k and how we choose to measure distances
- Most common choice is Euclidean or L_2 distance metric:

- ▶ $d(x^{(t)}, x) = \sqrt{\sum_{j=1}^d (x_j^{(t)} - x_j)^2}$

- Many other possibilities — next most common is probably Manhattan or L_1 :

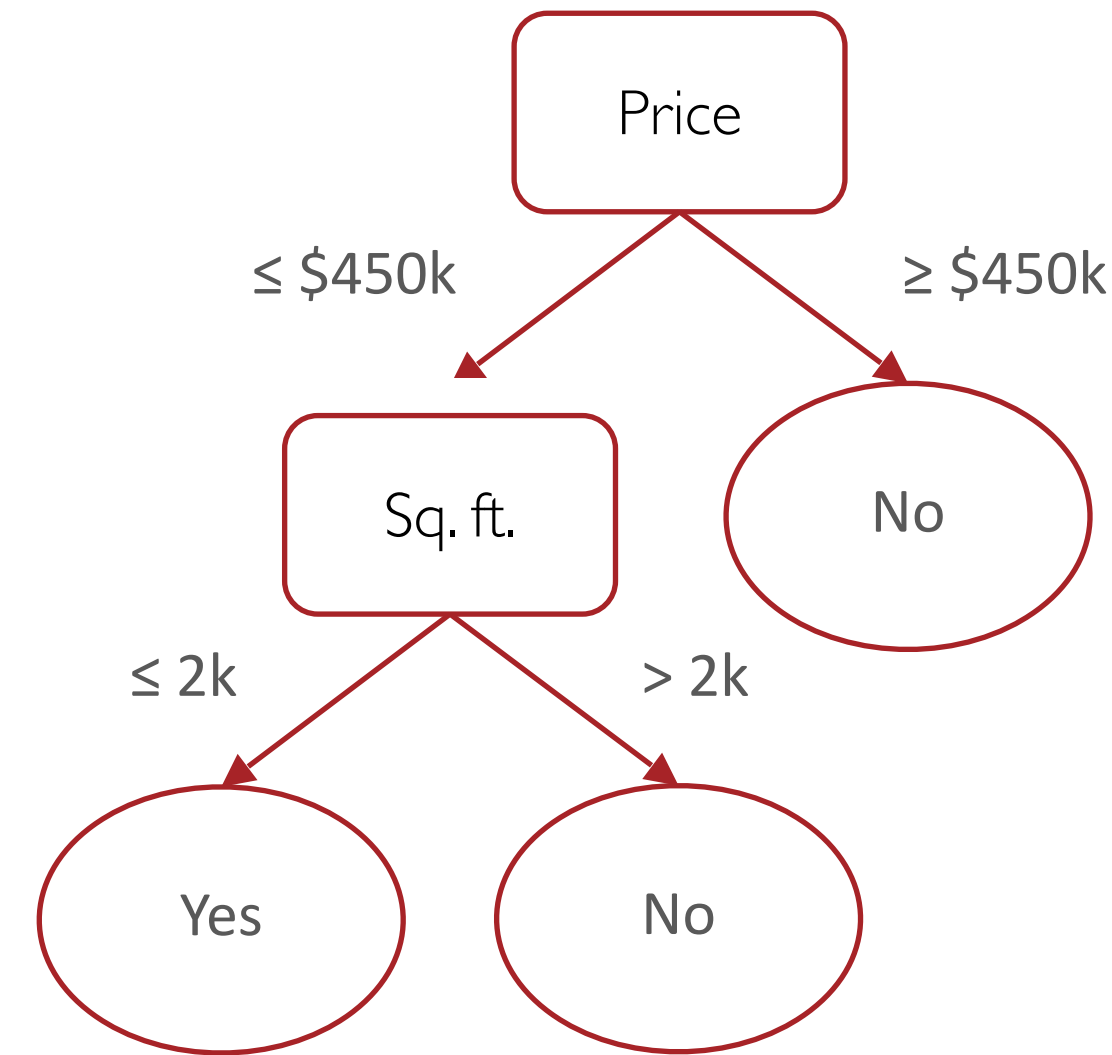
- ▶ $d(x^{(t)}, x) = \sum_{j=1}^d |x_j^{(t)} - x_j|$

Model selection



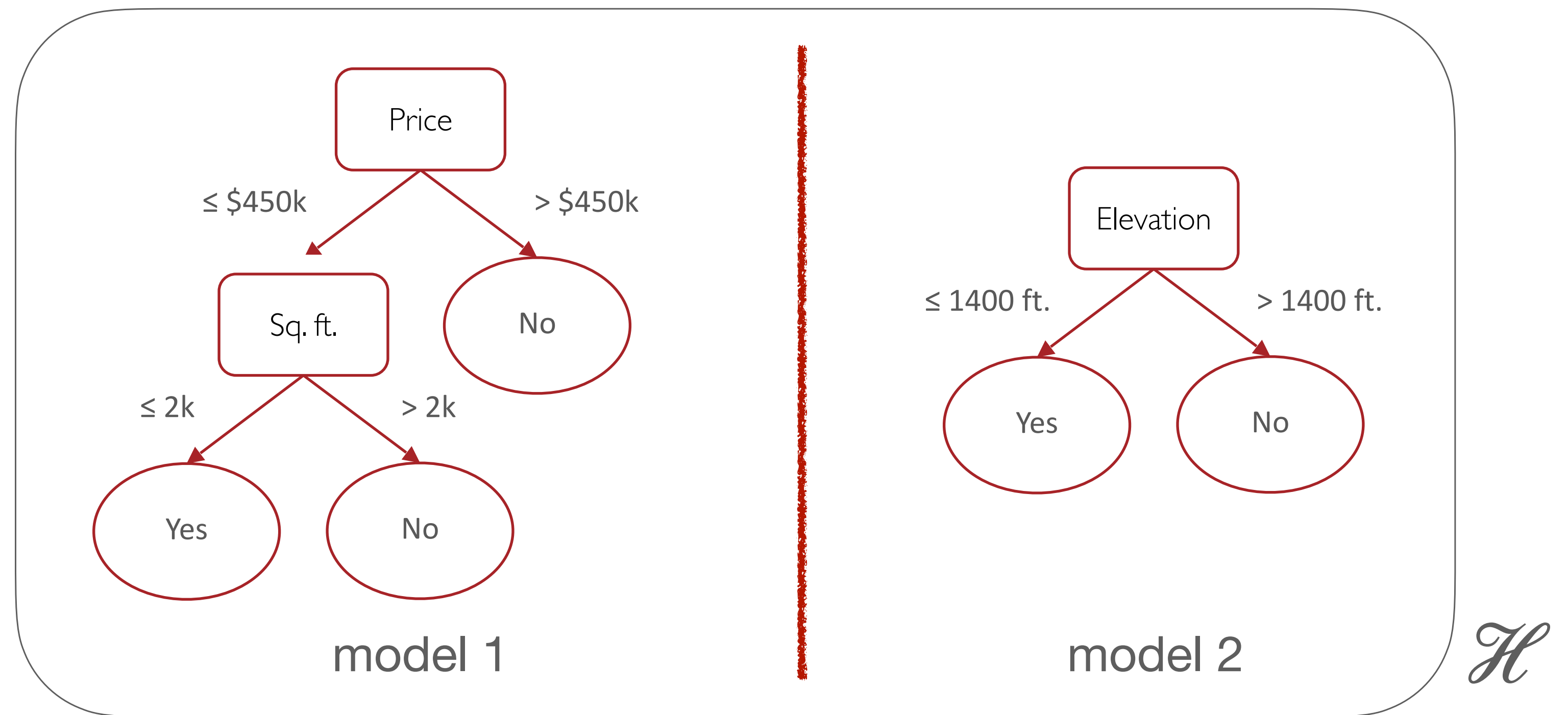
- A ***hypothesis space*** $\mathcal{H}$ is a set of hypotheses (classifiers, regression functions, ...)
 - ▶ e.g., all decision trees of depth ≤ 3 with splits on attributes { square feet, price, elevation, rooms }
- The learning algorithm searches through $\mathcal{H}$ to try to find a good hypothesis

Model selection



- If the hypotheses all share a similar form (e.g., decision trees of same structure), hypothesis space is often called a ***model***
 - ▶ different hypotheses (trained models) distinguished by (often real-valued) ***parameters***; it is learner's job to fit parameters

Model selection



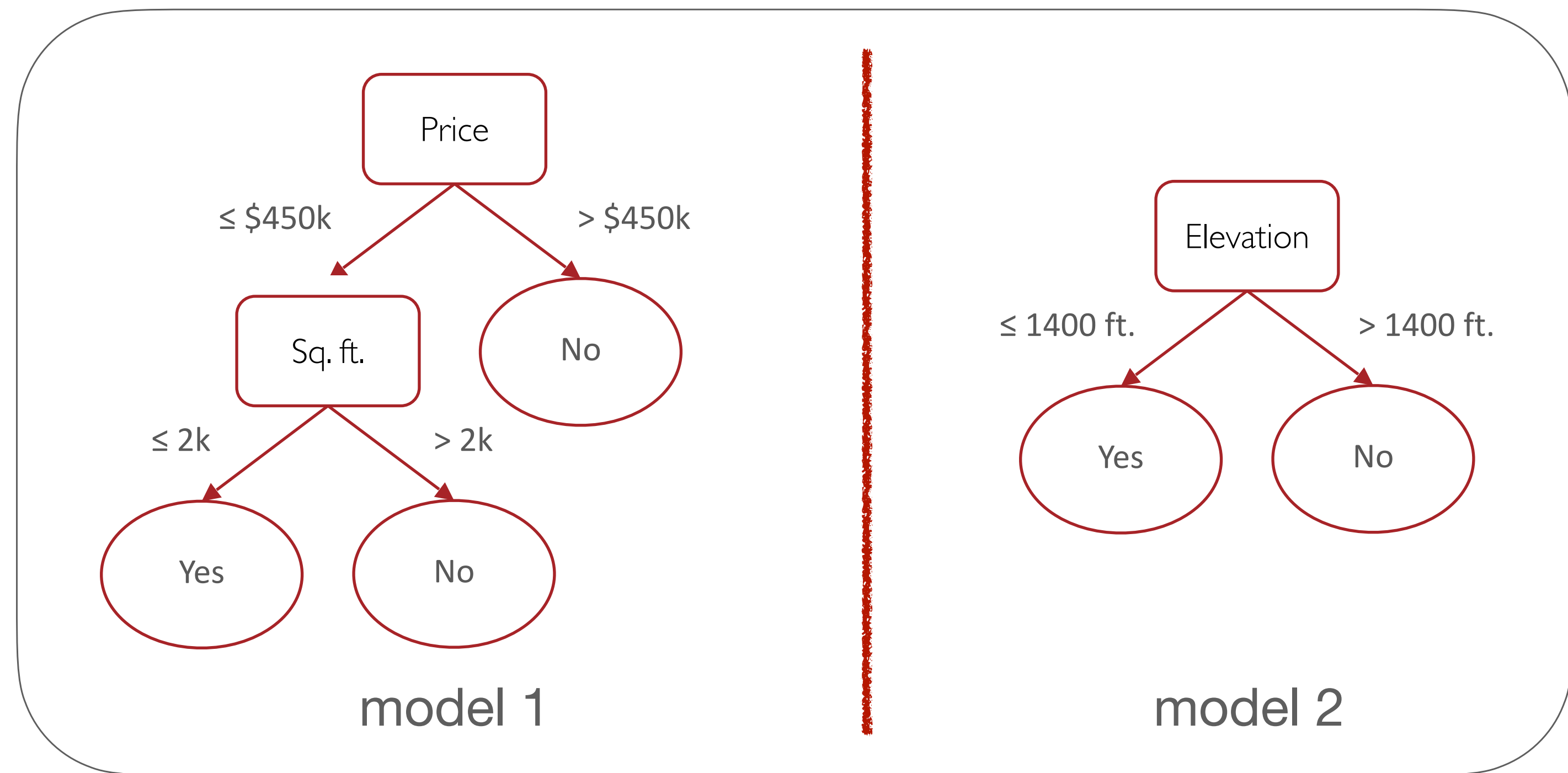
- More generally, may want to search over several models
 - ▶ hypothesis space is the *union* of the models
 - ▶ and *model selection* is the process of choosing
- Why?
 - ▶ trade off simplicity vs. expressiveness — reduce overfitting by taking simplest model that fits

Hyper-parameters

Distinction between parameters vs. hyperparameters is *our* choice: what's important enough to call a new model? What naturally fits in our training/optimization method?

- Often, we call the distinguishing factors among models ***hyperparameters***
 - ▶ e.g., depth of decision tree, k in k -NN, or how many attention heads in transformer
 - ▶ in this case, model selection is ***hyperparameter tuning***
- Not just model structure: learning algorithm can have hyperparameters
 - ▶ e.g., mutual information vs. Gini split criterion for decision tree learner
 - ▶ e.g., momentum parameter for deep net learning
 - ▶ e.g., early stopping for either decision trees or deep nets

Model selection loop



- Often, two nested loops
 - ▶ inner (learning) loop fits parameters of one model
 - ▶ outer (model selection) loop chooses which one

Train/ validation/ test split for model selection

- Split data $\mathcal{D}$ into three parts:
 - ▶ $\mathcal{D}_{\text{train}}$: learning *within a model class*
 - ▶ $\mathcal{D}_{\text{val}}$: *model selection*
 - ▶ $\mathcal{D}_{\text{test}}$: final evaluation
- Tension: make one bigger to make its result more accurate, at expense of rest

features			labels
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes
No	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Abnormal	Yes

Model selection loop

- For each model in $\mathcal{H}$:
 - ▶ train on $\mathcal{D}_{\text{train}}$
 - ▶ evaluate on $\mathcal{D}_{\text{val}}$; track best

good parameters, maybe not best, and optimistic performance estimate

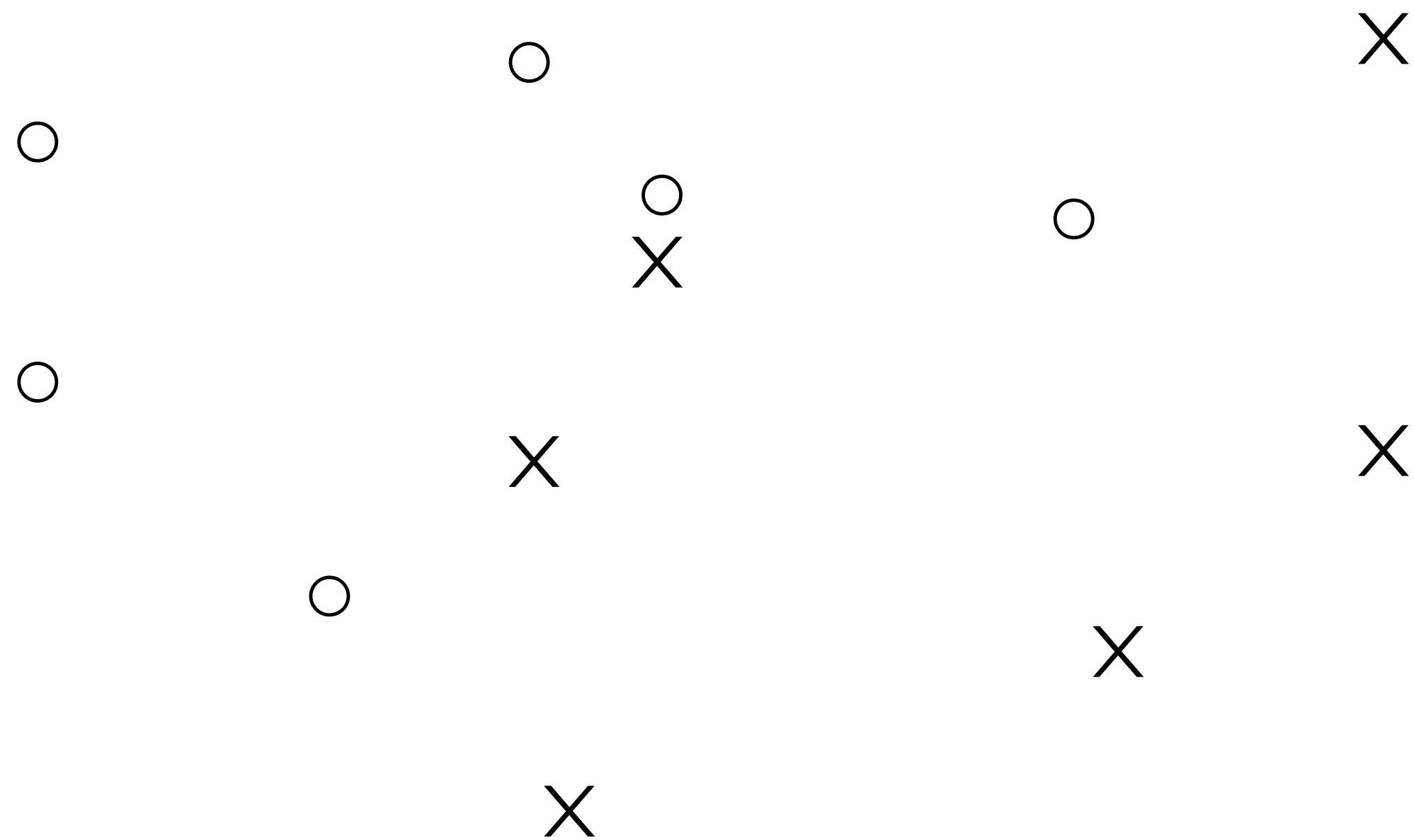
each individual evaluation is unbiased, but max is optimistic
- Finally, evaluate single best model on $\mathcal{D}_{\text{test}}$

unbiased estimate of the selected model's performance

Model selection loop

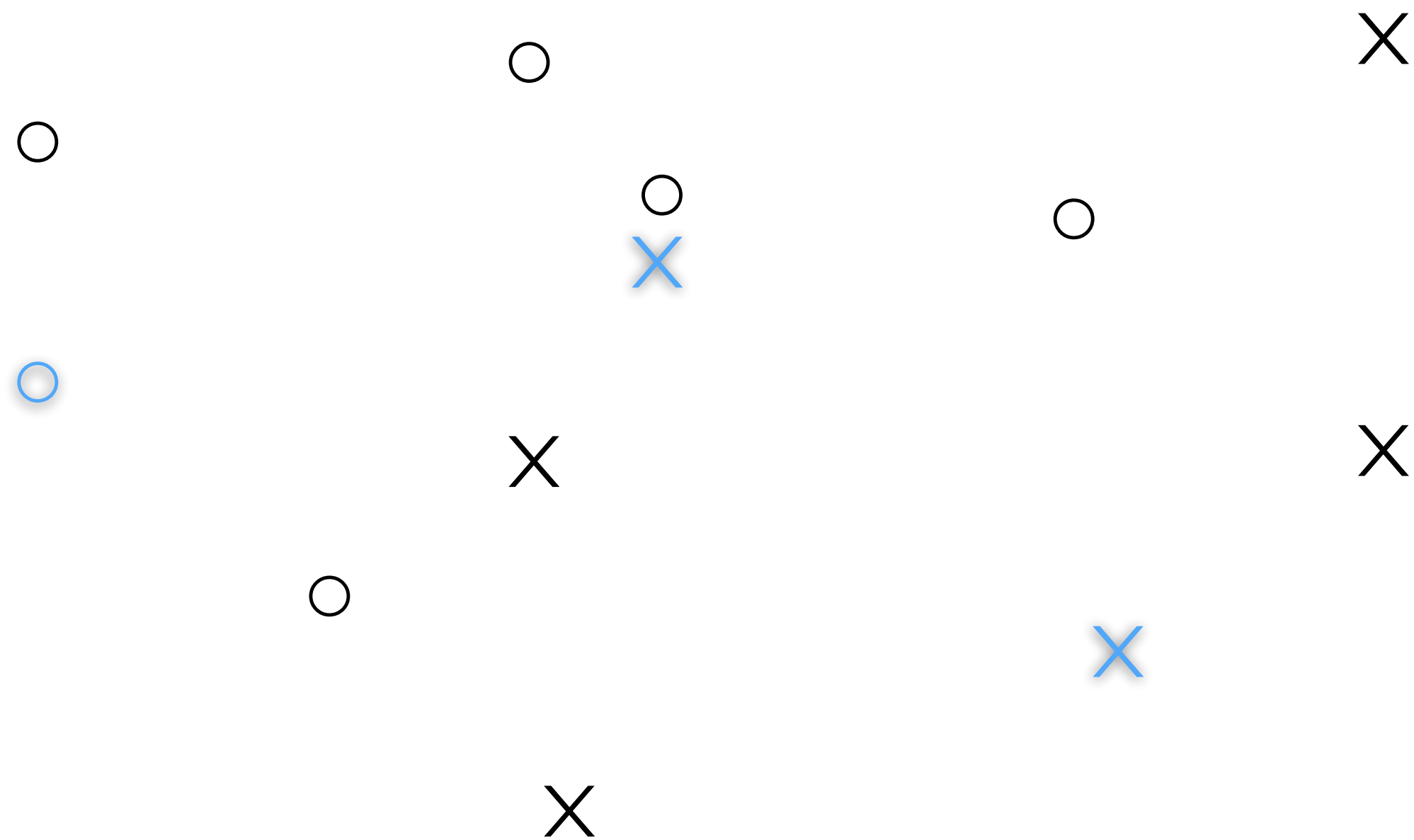
- For each model in $\mathcal{H}$:
 - ▶ train on $\mathcal{D}_{\text{train}}$
 - ▶ evaluate on $\mathcal{D}_{\text{val}}$; track best
 - [optionally] retrain best model on $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$
 - Finally, evaluate single best model on $\mathcal{D}_{\text{test}}$
- good parameters, maybe not best, and optimistic performance estimate
- each individual evaluation is unbiased, but max is optimistic
- more data
- unbiased estimate of the selected model's performance

***Inner loop:
train and
evaluate a
single model***



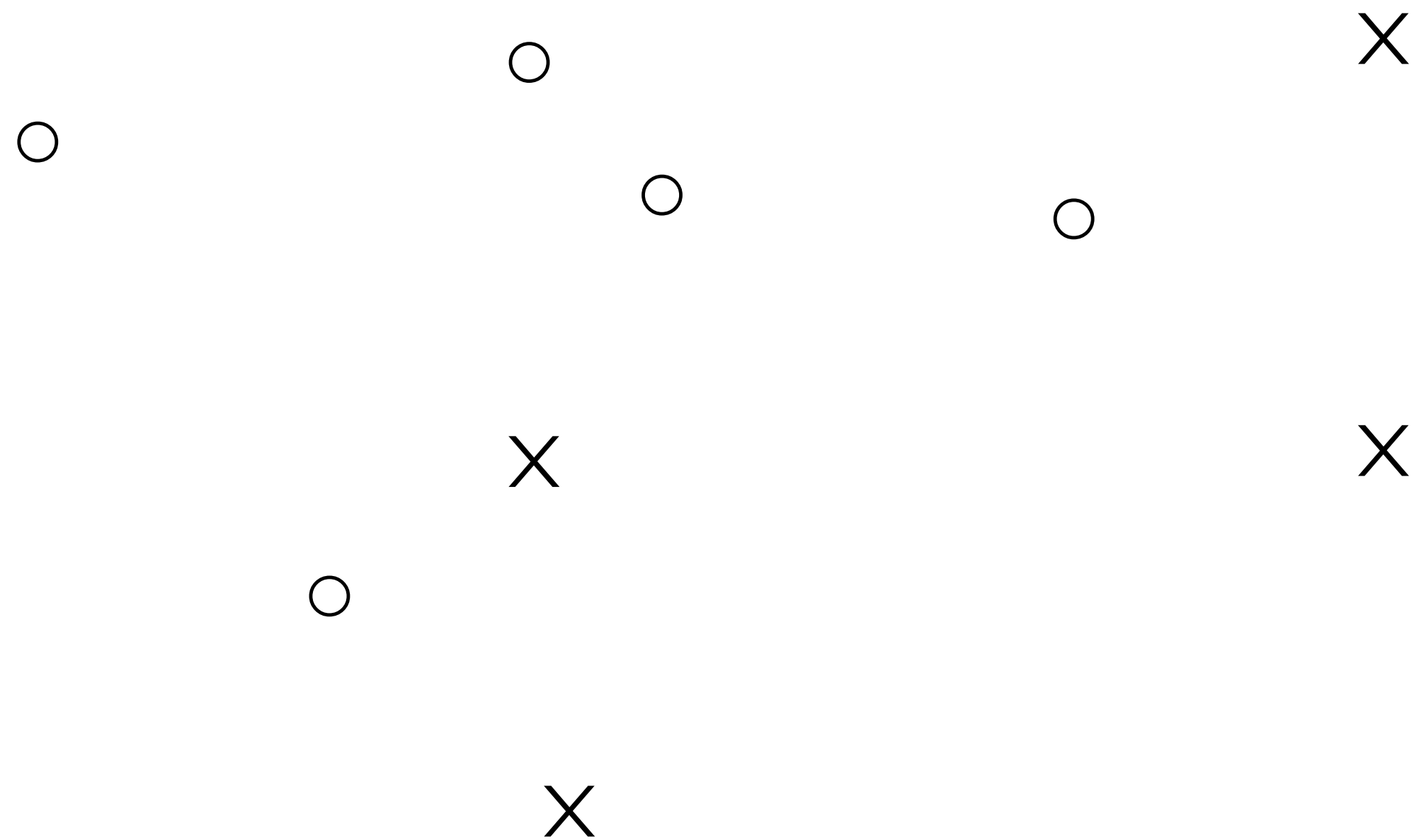
training set

***Inner loop:
train and
evaluate a
single model***



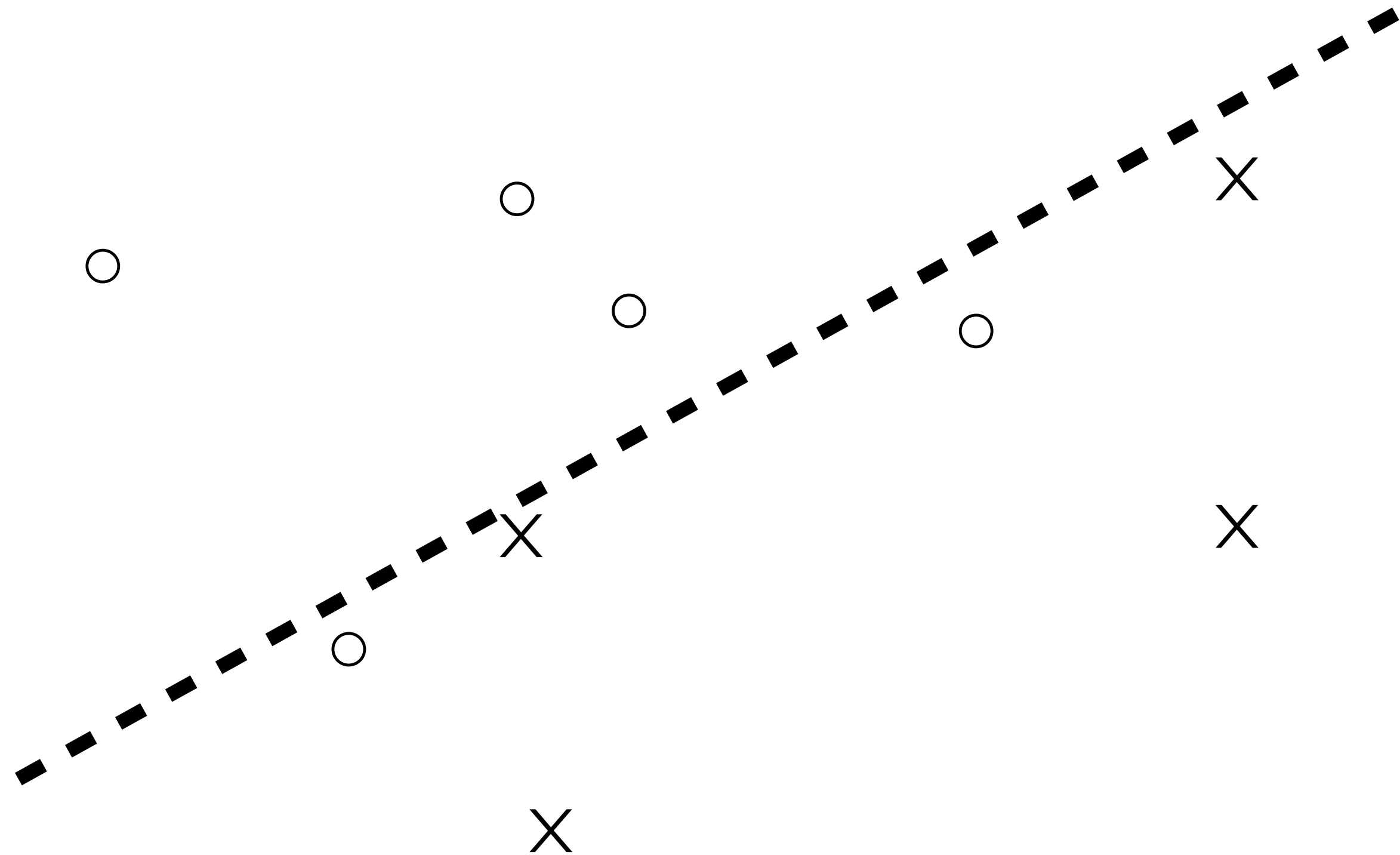
remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***



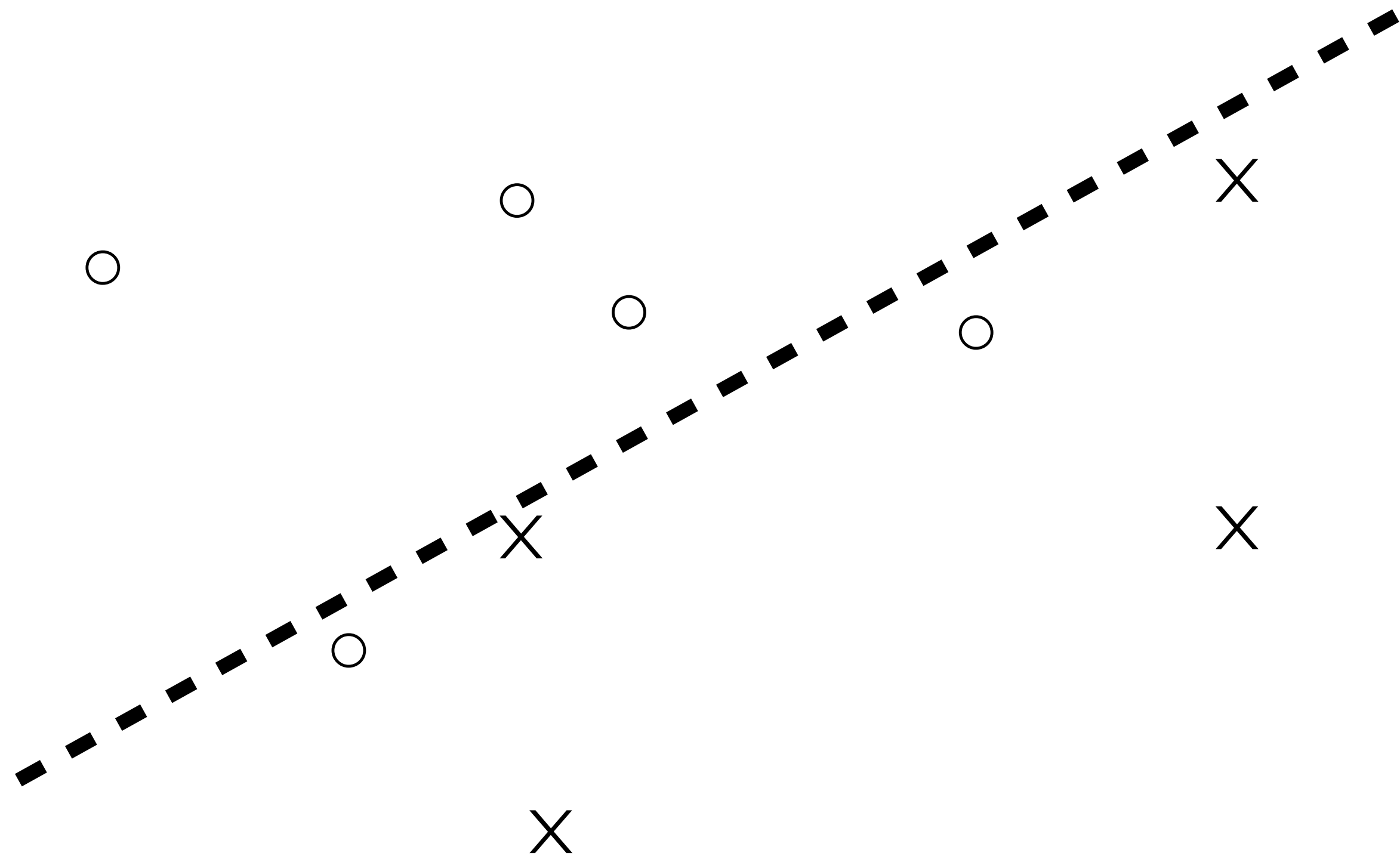
remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***



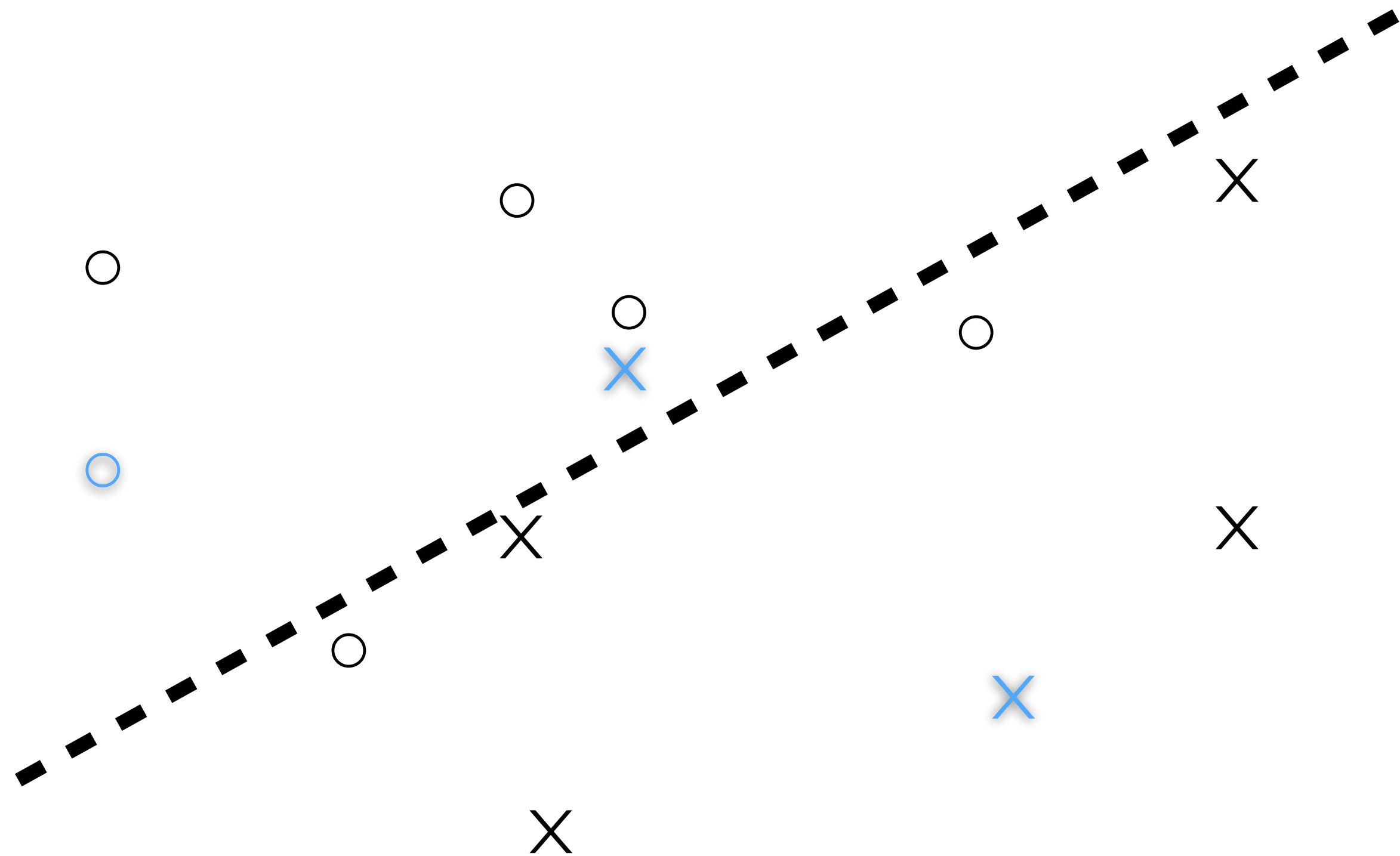
remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***



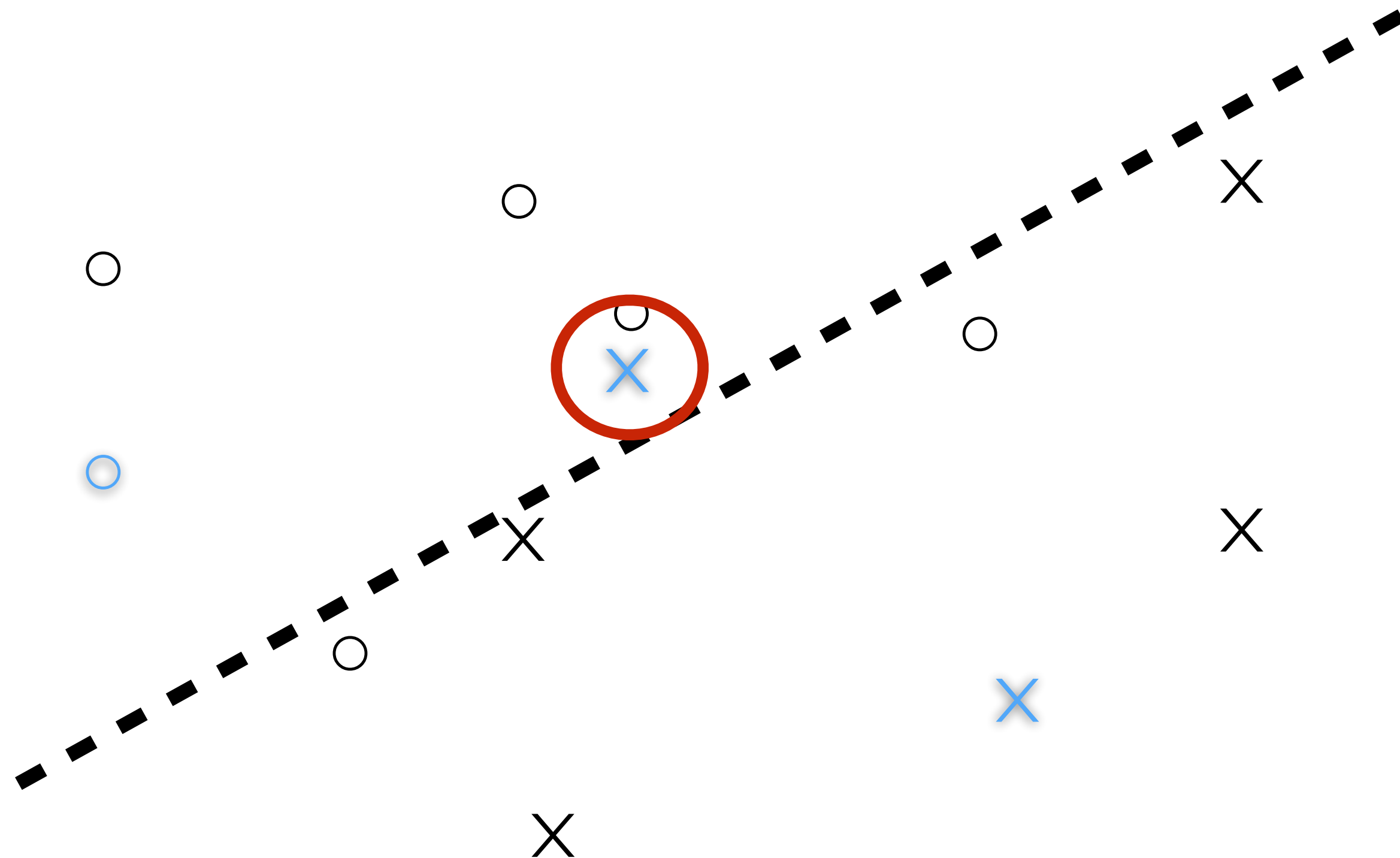
compute error on holdout set

***Inner loop:
train and
evaluate a
single model***

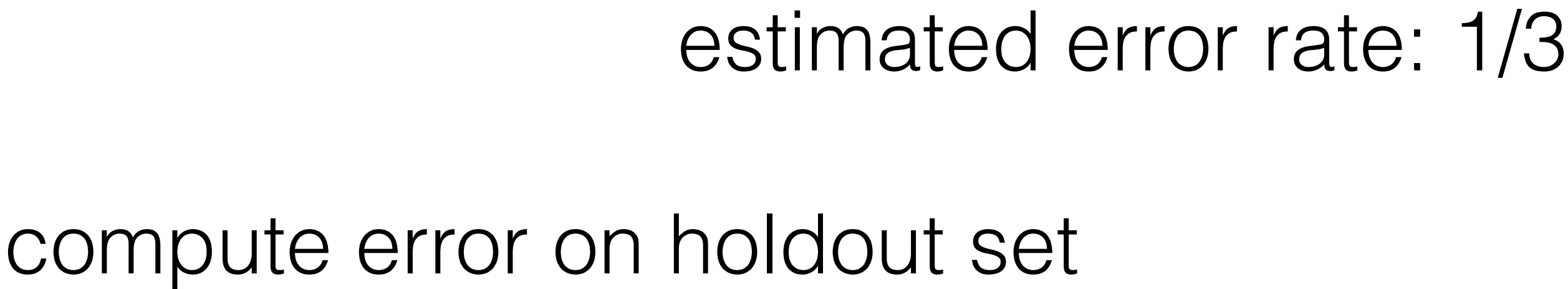


compute error on holdout set

***Inner loop:
train and
evaluate a
single model***



compute error on holdout set

[illegible]

Problem with hold- out set

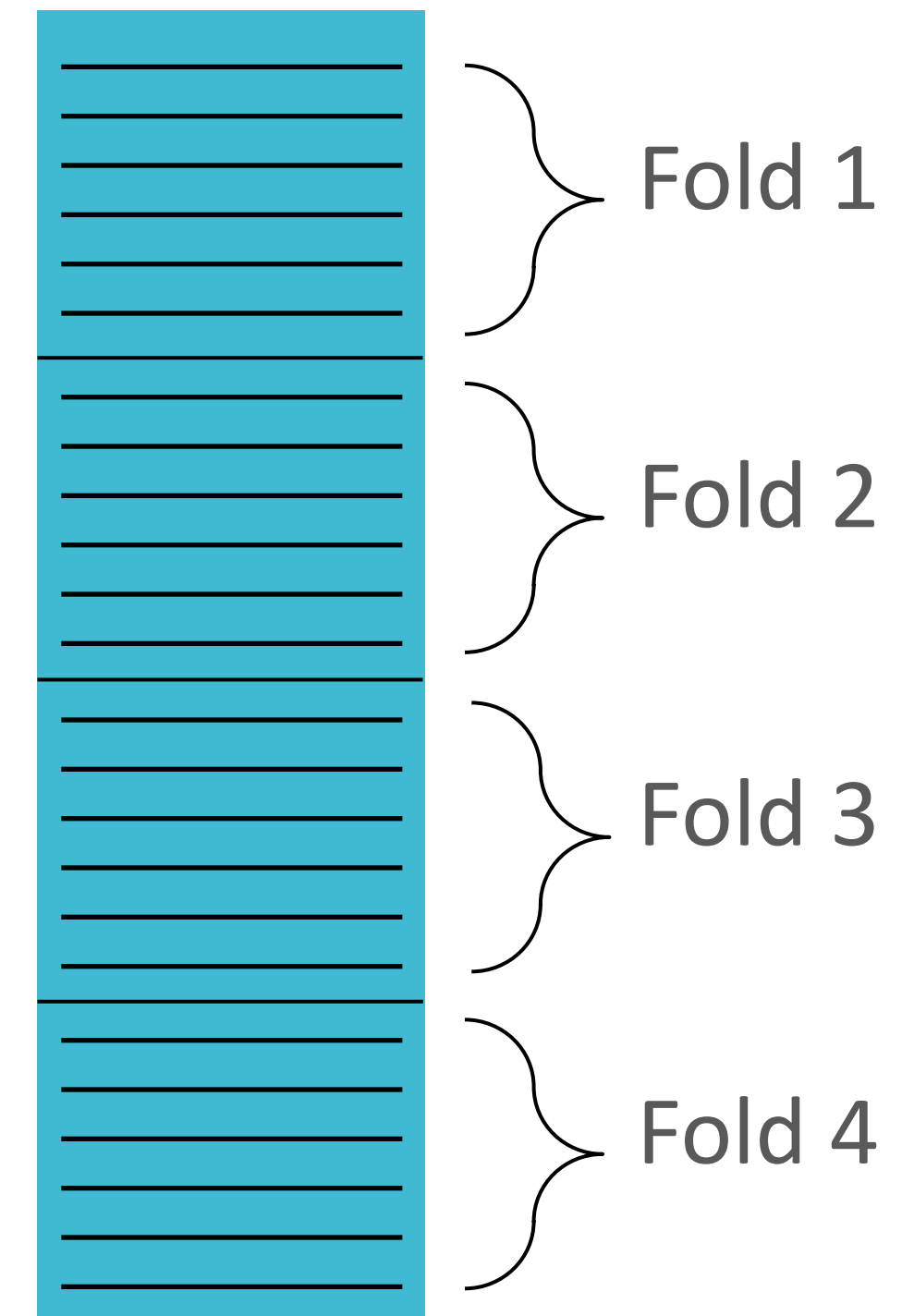
- Tension between size of $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{val}}$
 - ▶ $\mathcal{D}_{\text{train}}$ is slightly too small
 - ▶ often $\mathcal{D}_{\text{val}}$ winds up small enough to be an inaccurate way to select a model
- What if we could put each training example in both sets?

Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}

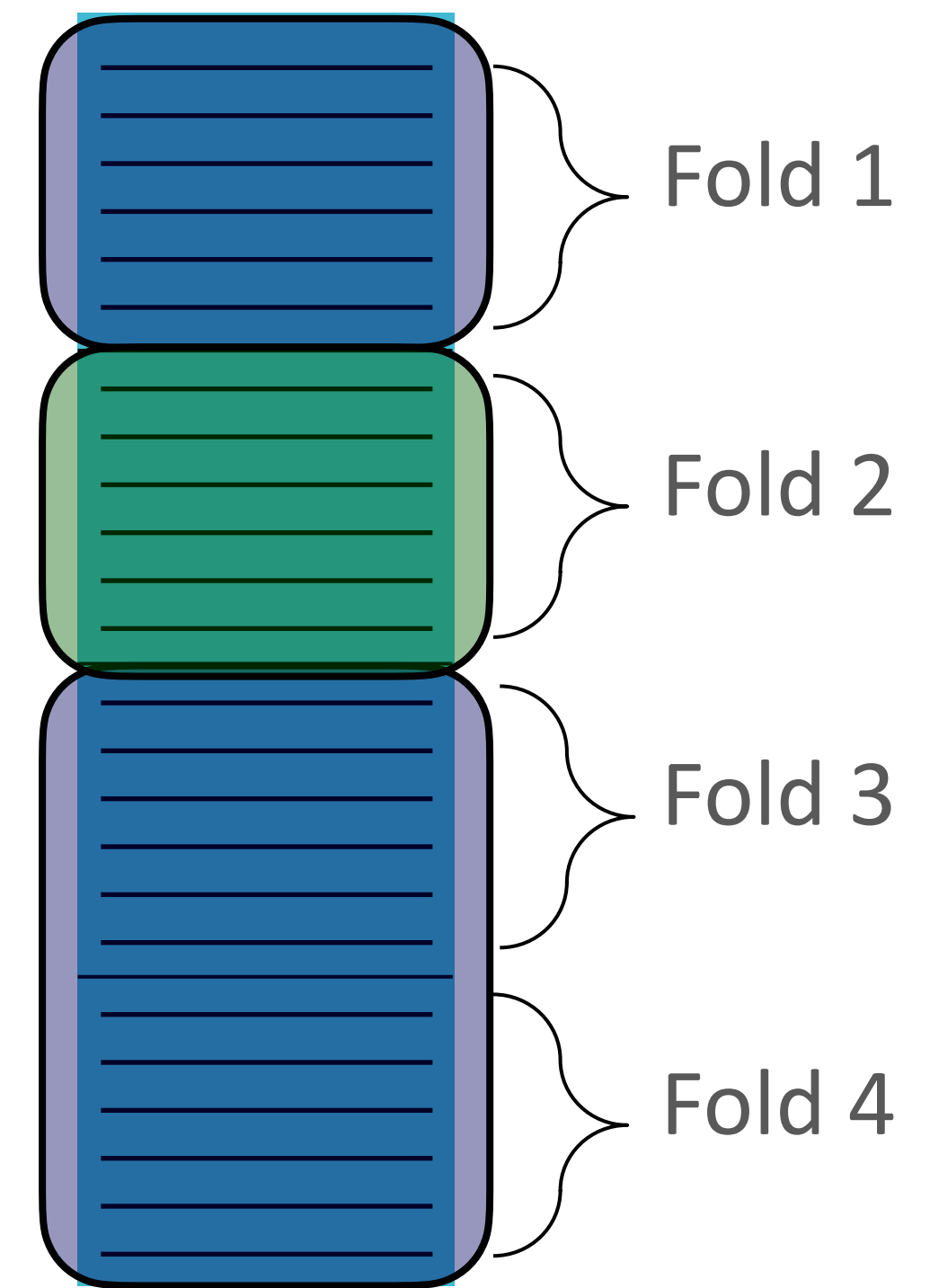
Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}

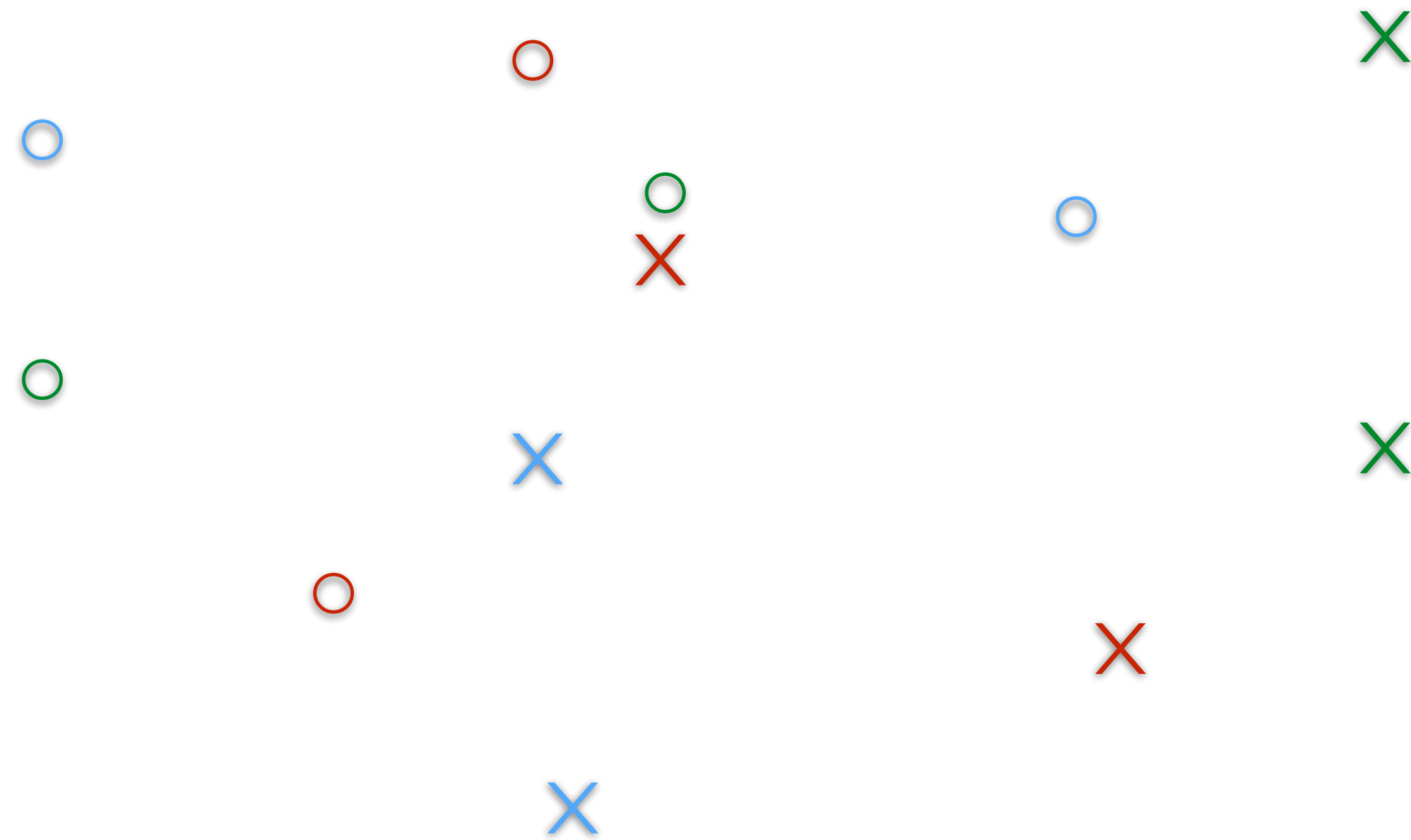


Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}

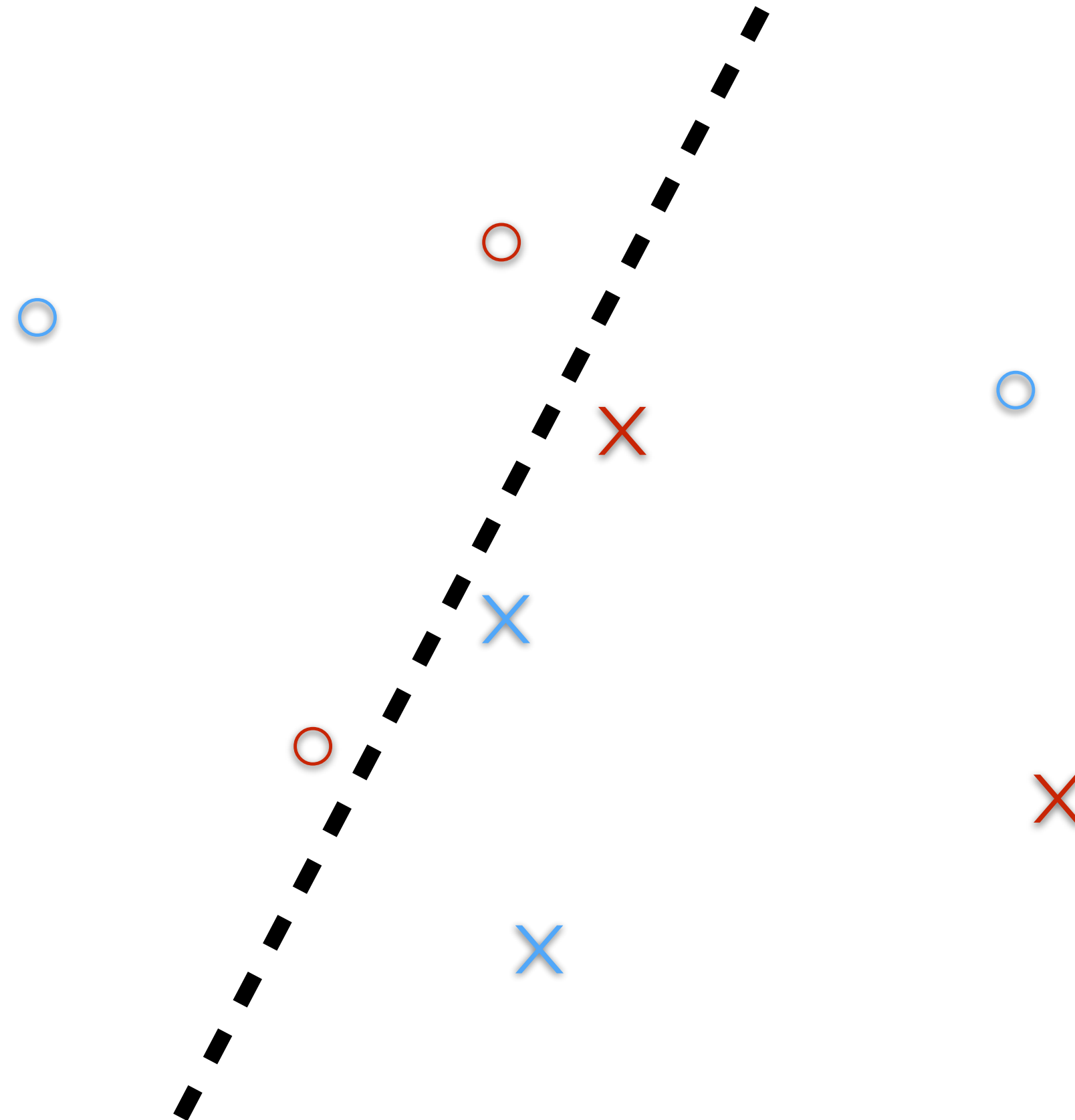


Cross- validation



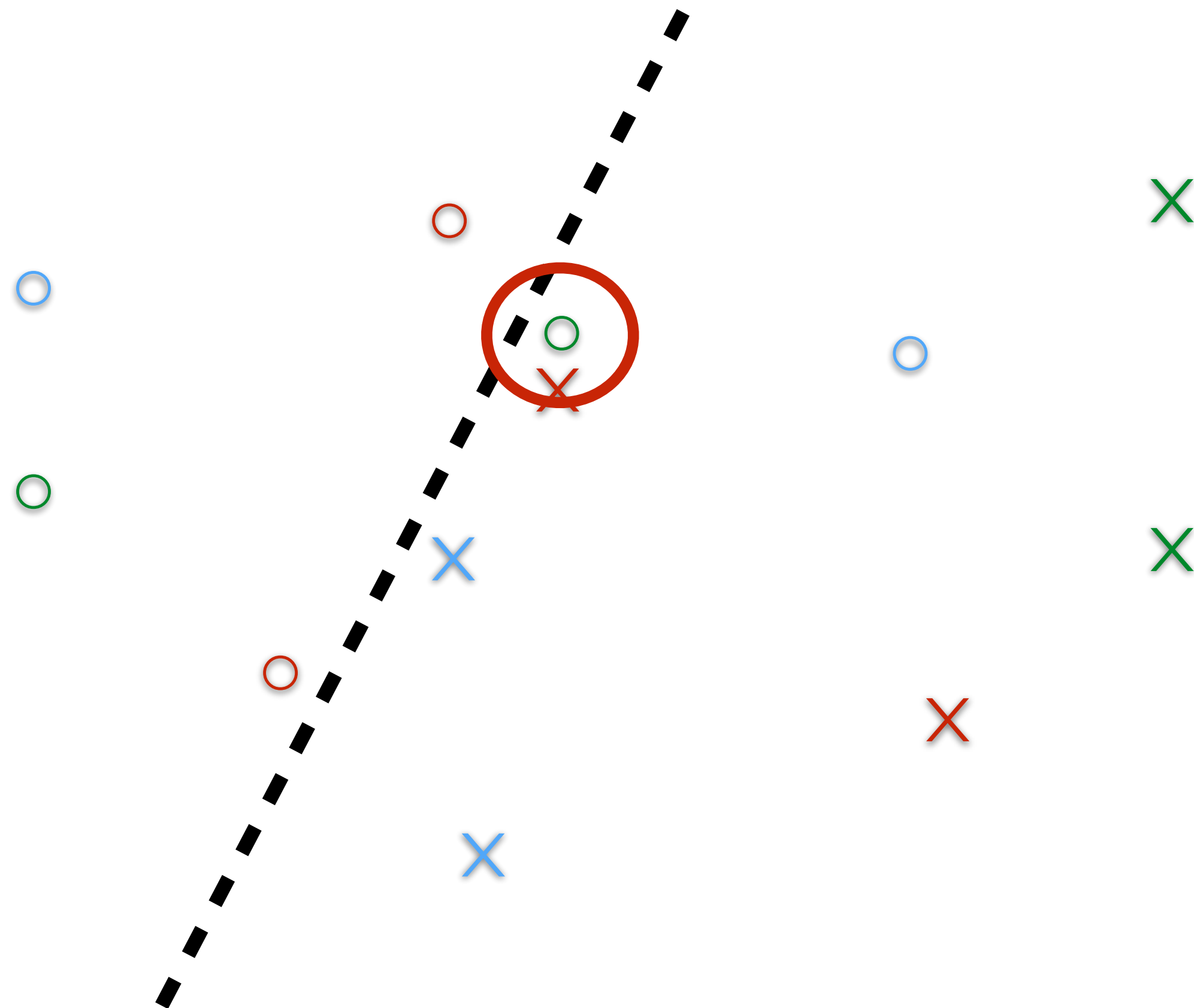
split data evenly into groups (“folds”)

Cross- validation



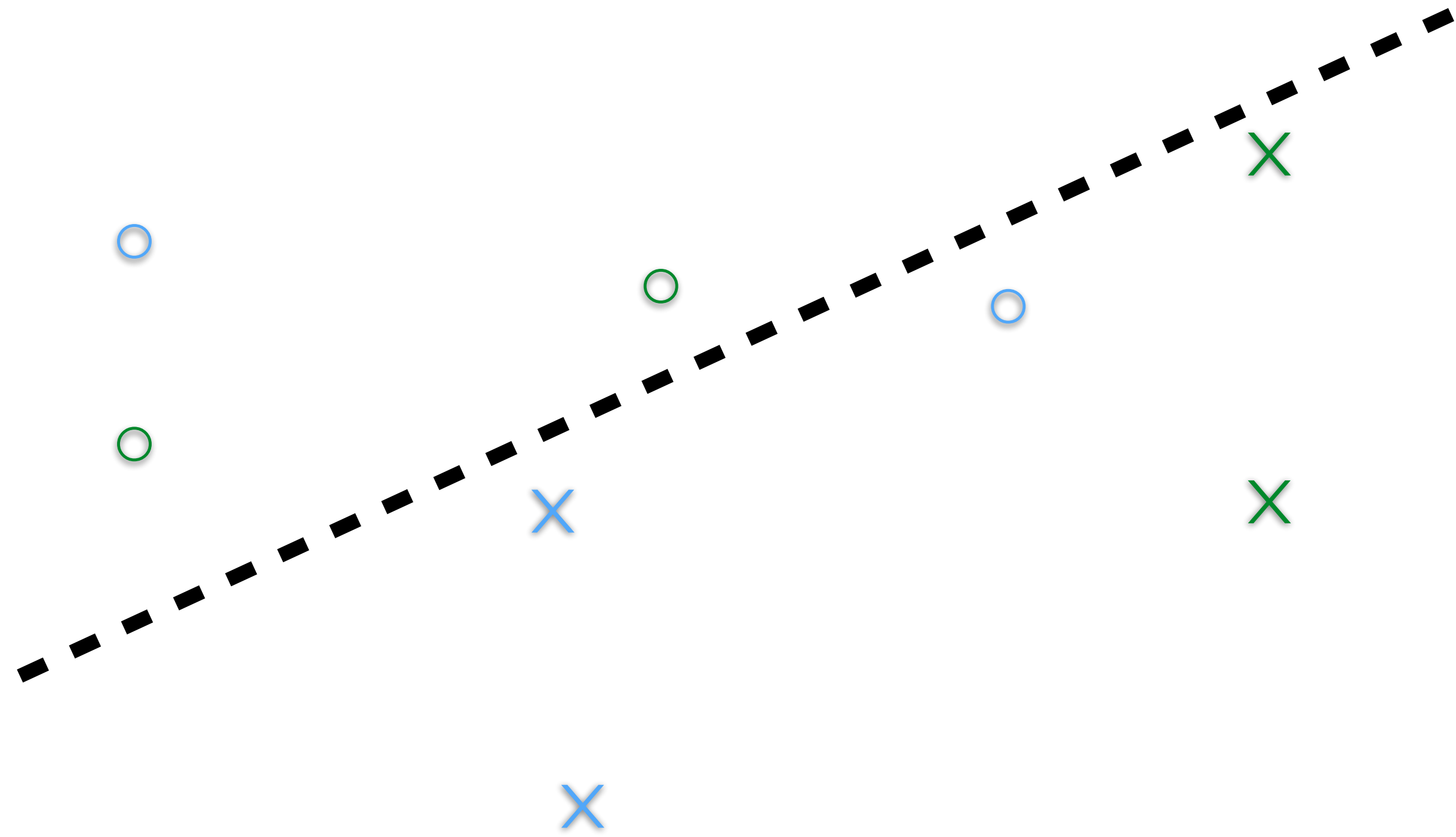
remove green group, fit on rest

Cross-validation



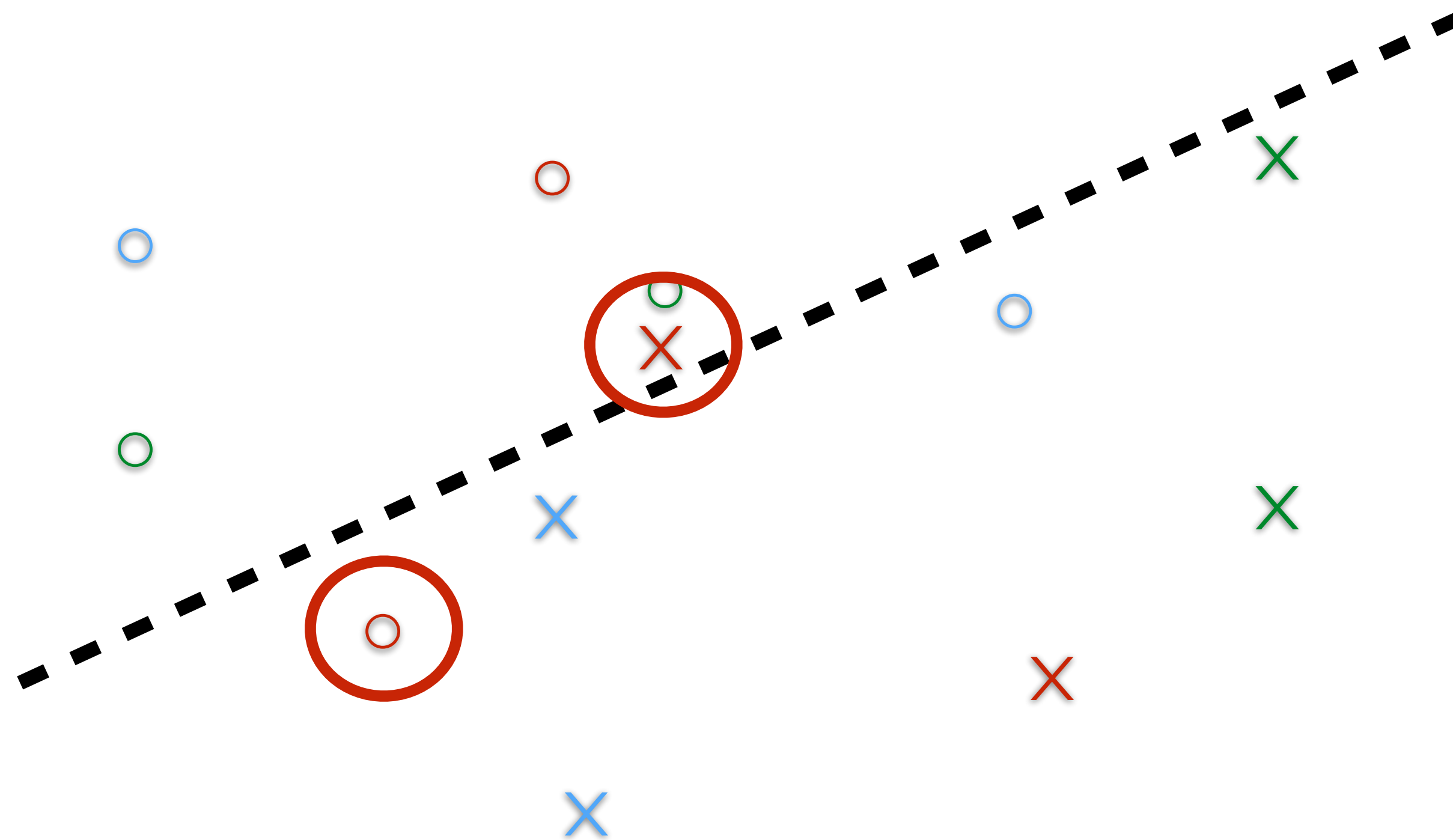
add back green group: error $1/4$

Cross- validation



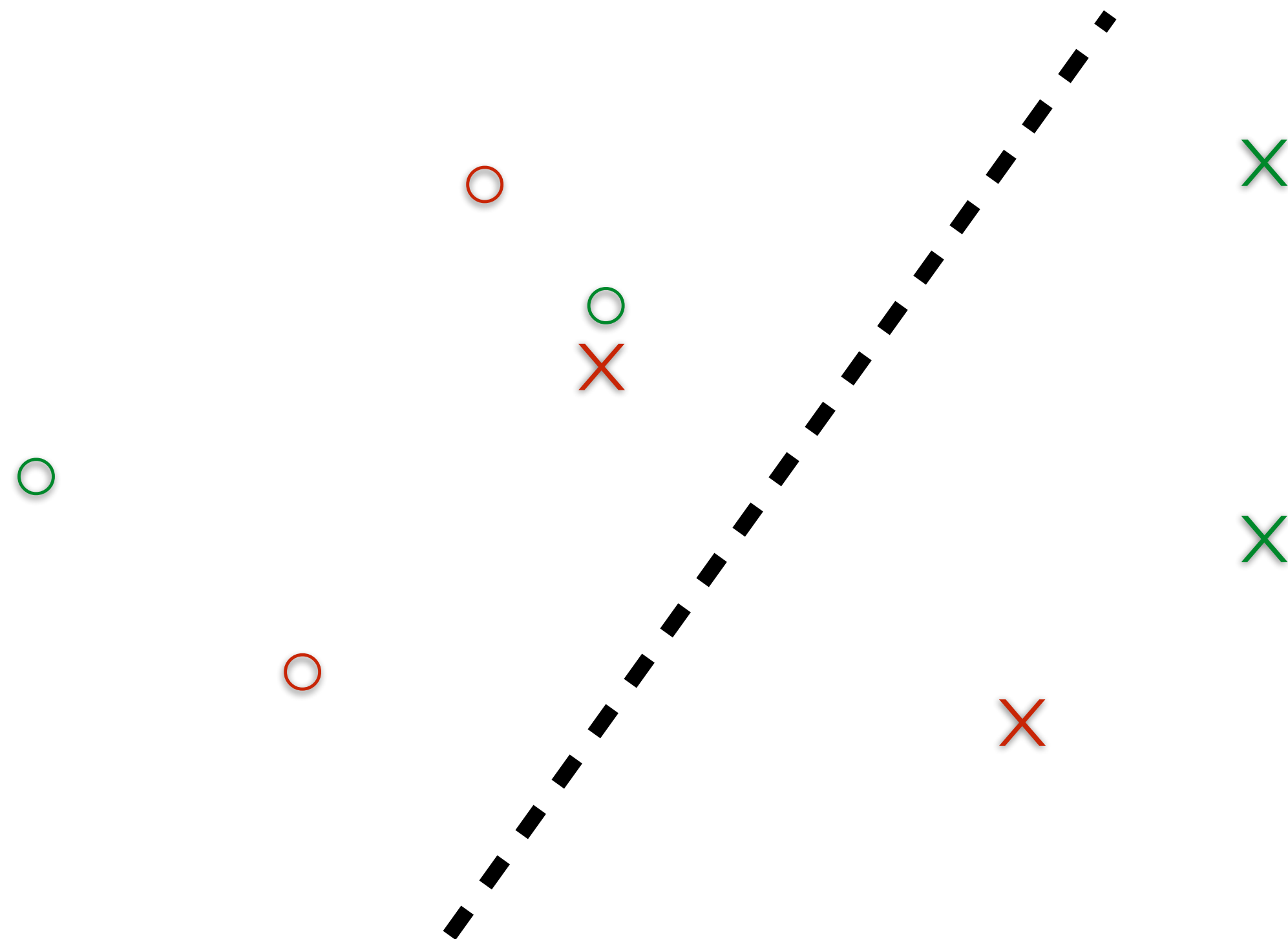
remove red group, fit on rest

Cross-validation



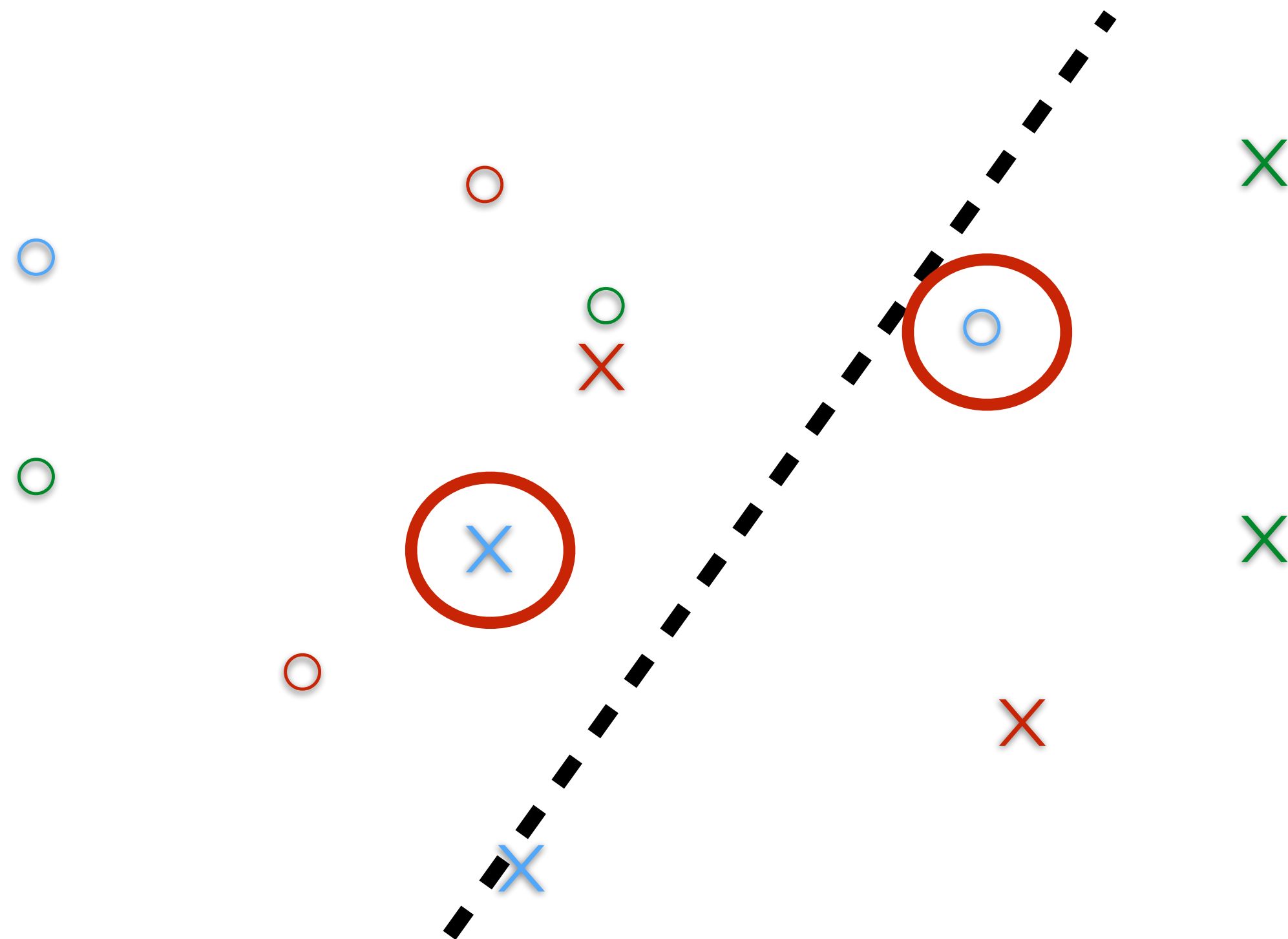
add back red group: error 2/4

Cross- validation



remove blue group, fit on rest

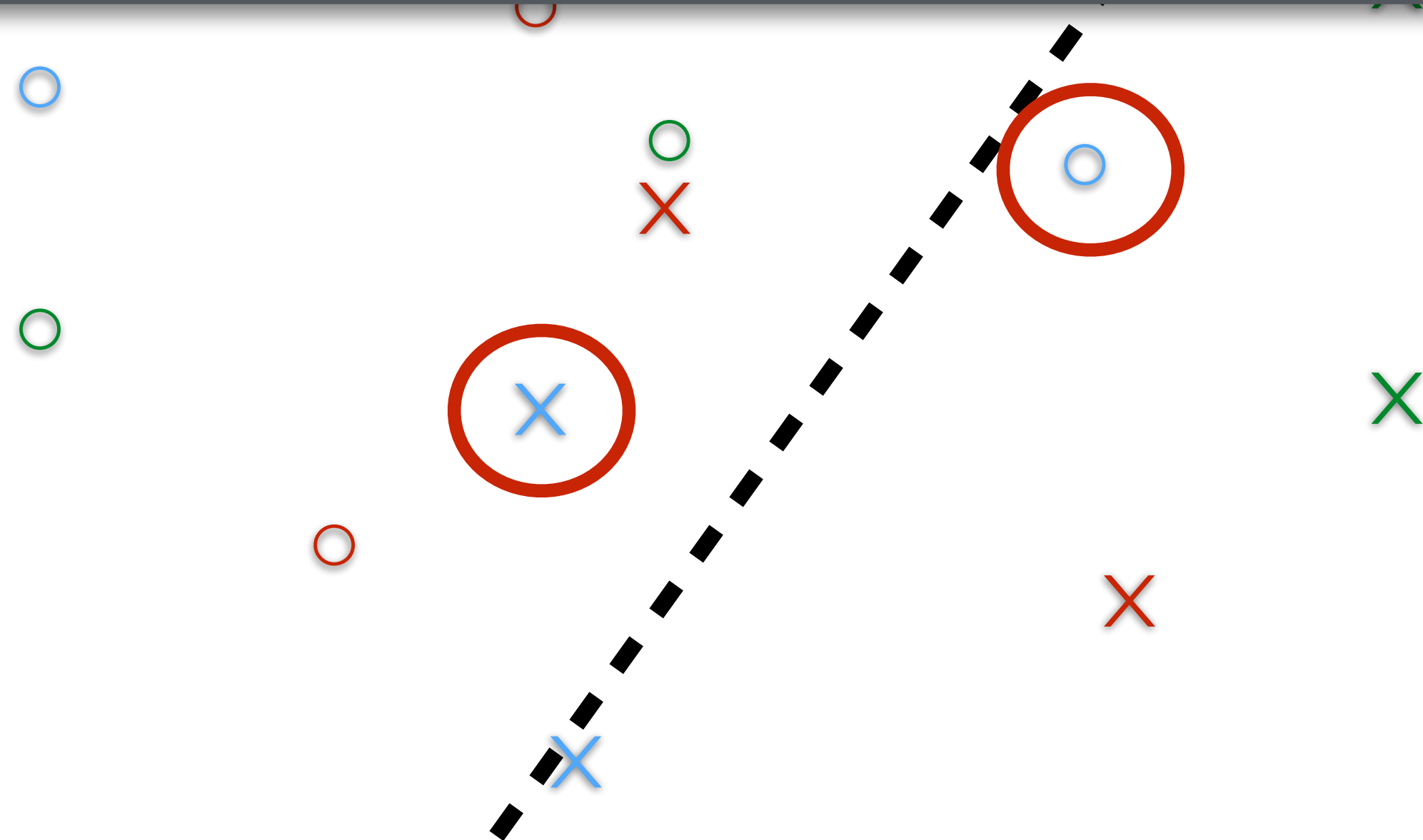
Cross-validation



add back blue group: error 2/4

Cross-validation

Overall: $(1+2+2)/12 = 42\%$ error rate



add back blue group: error 2/4

Model selection with cross- validation

- Split into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$
- Split $\mathcal{D}_{\text{train}}$ into folds
- For each model:
 - ▶ for each fold F
 - ▶ train on $\mathcal{D}_{\text{train}} \setminus F$
 - ▶ evaluate on F
 - ▶ average performance over folds — track best model
- Retrain single best model on all folds
- Finally, evaluate resulting trained model on $\mathcal{D}_{\text{test}}$

Model selection with cross- validation

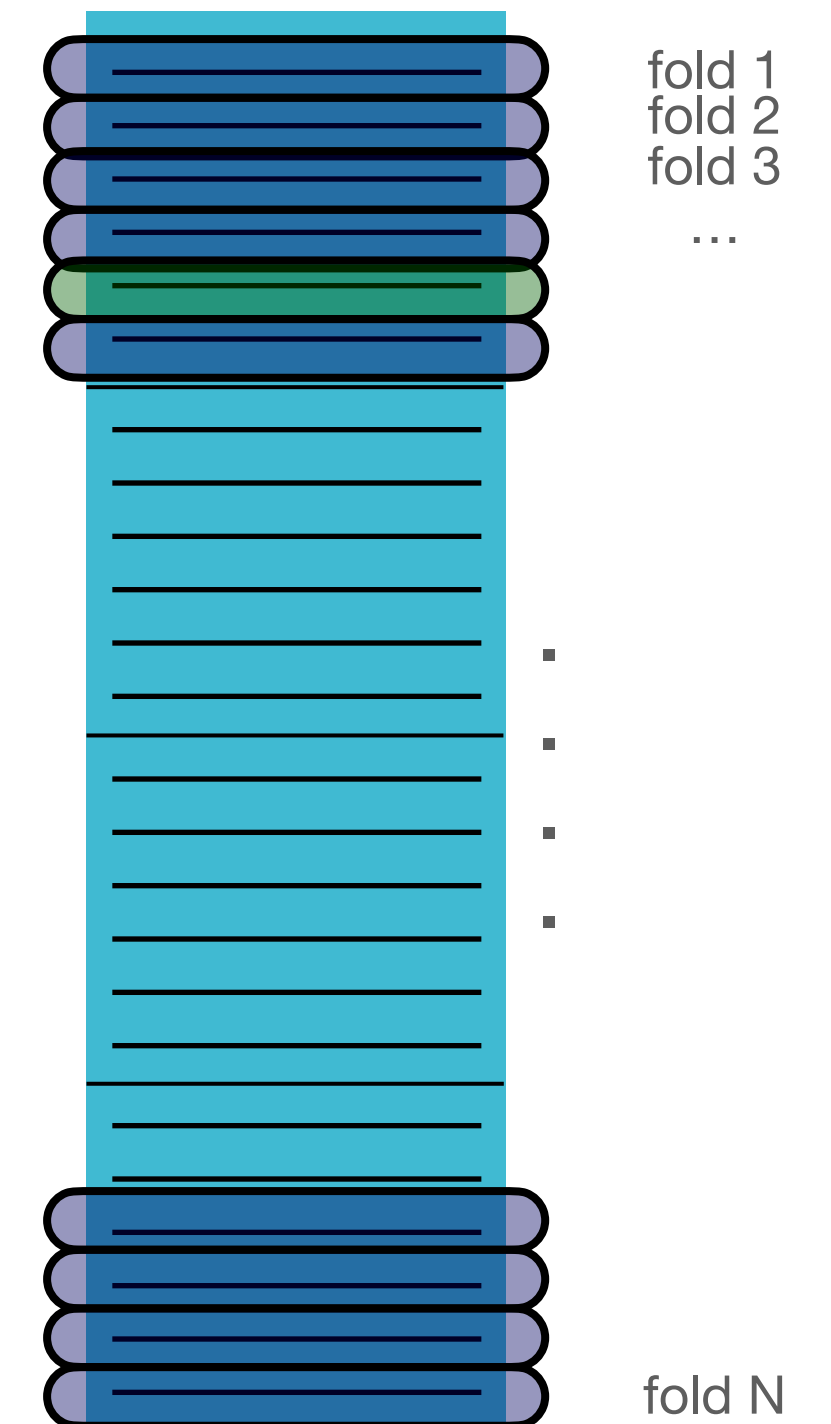
- Split into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$
- Split $\mathcal{D}_{\text{train}}$ into folds
- For each model:
 - ▶ for each fold F
 - ▶ train on $\mathcal{D}_{\text{train}} \setminus F$
 - ▶ evaluate on F
 - ▶ average performance over folds — track best model
- Retrain single best model on all folds
- Finally, evaluate resulting trained model on $\mathcal{D}_{\text{test}}$

could also return ***all*** trained models, use them in an ***ensemble*** (e.g., by averaging their predictions)

LOOCV

special case: each
point is its own fold

- Useful special case: *leave-one-out CV*
 - ▶ each point is its own fold: $k = N$
- Keeps training set size as large as possible ($N - 1$)
- Each validation sample is as close as possible to independent
- Expensive — unless there are shortcuts



Learning objectives— kNN and model selection

- How do we train a kNN classifier? How do we predict?
- What is a Voronoi diagram?
- How do k and the distance function influence kNN?
- What is model selection? Why might we want to do it?
- What is a hyperparameter? For example, what are the hyperparameters of kNN?
- How can we do model selection with a train/validation split? How can we do it with cross-validation?